Neural Language Models

Tanmoy Chakraborty
Associate Professor, IIT Delhi
https://tanmoychak.com/

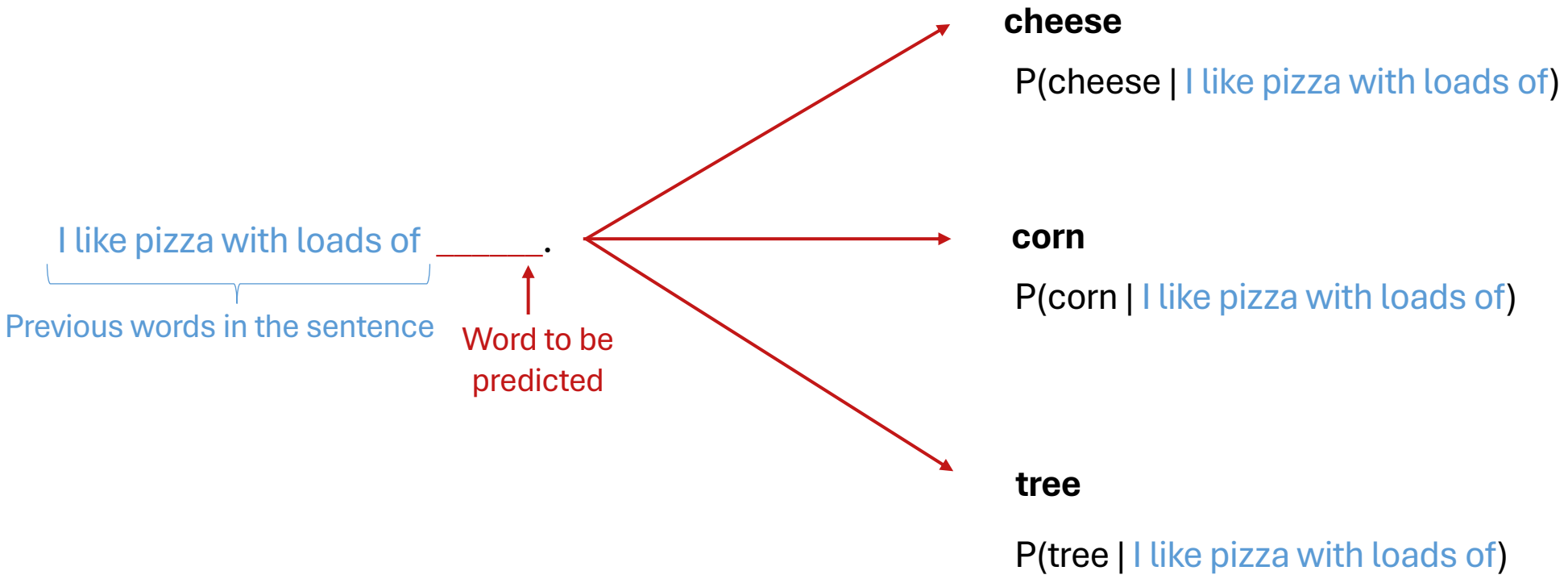
Introduction to Large Language Models

Pre-requisite for this chapter

- Loss function, backpropagation
- CNN
- RNN (LSTM/GRU)

Recall: Language Modeling

- **Language Modeling** is the task of predicting what word comes next



Recall: Language Modeling

- You can also think of a Language Model as a system that assigns a probability to a piece of text.

- For example, if we have some text $x^{(1)}, \ldots, x^{(T)}$, then the probability of this text (according to the Language Model) is:

$$P(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(T)}) = P(\boldsymbol{x}^{(1)}) \times P(\boldsymbol{x}^{(2)} | \boldsymbol{x}^{(1)}) \times \cdots \times P(\boldsymbol{x}^{(T)} | \boldsymbol{x}^{(T-1)}, \ldots, \boldsymbol{x}^{(1)})$$

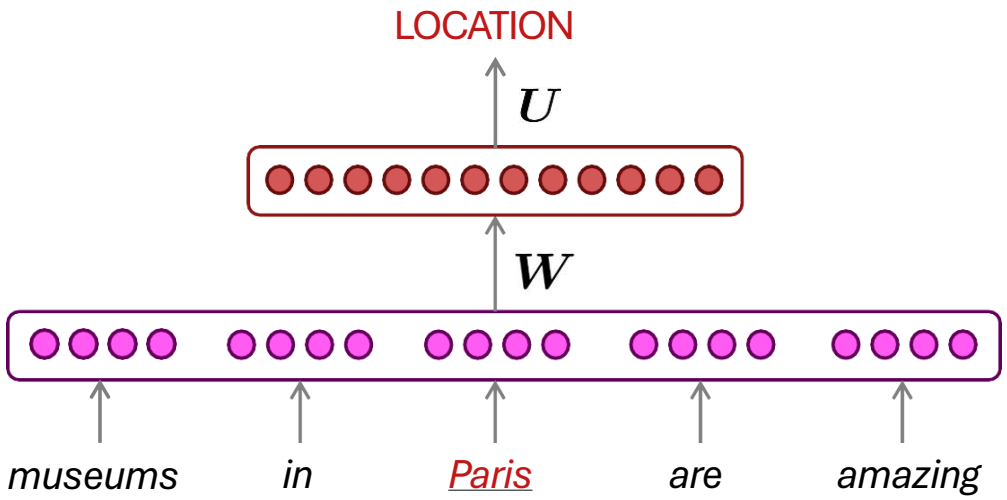
$$= \prod_{t=1}^{T} \underbrace{P(\boldsymbol{x}^{(t)} | \boldsymbol{x}^{(t-1)}, \ldots, \boldsymbol{x}^{(1)})}$$

This is what our LM provides

How to Build a *Neural* Language Model?

- Recall the Language Modeling task:
  - **Input:** sequence of words $\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \ldots, \boldsymbol{x}^{(t)}$
  - **Output:** probability distribution of the next word $\boldsymbol{P}\left(\boldsymbol{x}^{(t+1)} \middle| \boldsymbol{x}^{(t)}, \ldots, \boldsymbol{x}^{(1)}\right)$
- How about a window-based neural model?

Example: NER Task



A Fixed-window Neural Language Model

~~as the proctor started the clock~~ the students opened their _____

discard

fixed window

A Fixed-window Neural Language Model

output distribution

$$\hat{\boldsymbol{y}} = \text{softmax}(\boldsymbol{U}\boldsymbol{h} + \boldsymbol{b}_2) \in \mathbb{R}^{|V|}$$

hidden layer

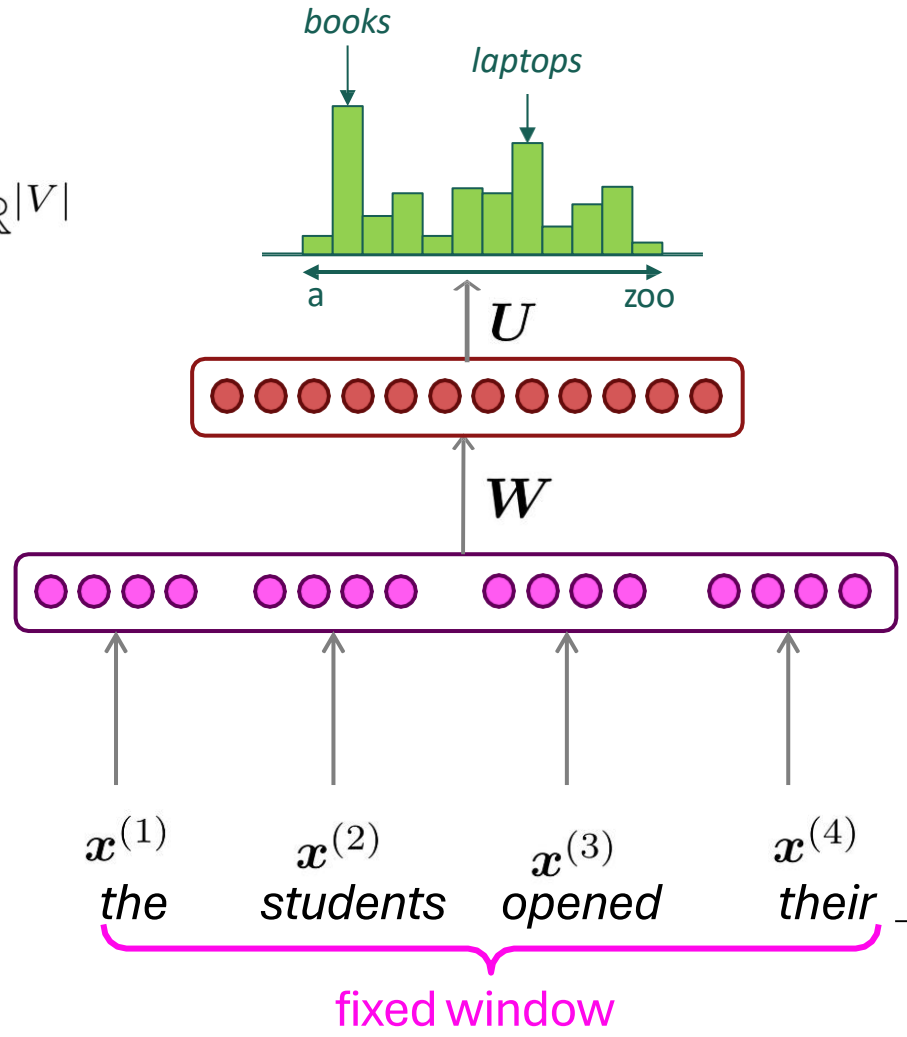
$$\boldsymbol{h} = f(\boldsymbol{W}\boldsymbol{e} + \boldsymbol{b}_1)$$

concatenated word embeddings

$$\boldsymbol{e} = [\boldsymbol{e}^{(1)}; \boldsymbol{e}^{(2)}; \boldsymbol{e}^{(3)}; \boldsymbol{e}^{(4)}]$$

words / one-hot vectors

$$\boldsymbol{x}^{(1)}, \boldsymbol{x}^{(2)}, \boldsymbol{x}^{(3)}, \boldsymbol{x}^{(4)}$$



~~as the proctor started the clock~~

discard

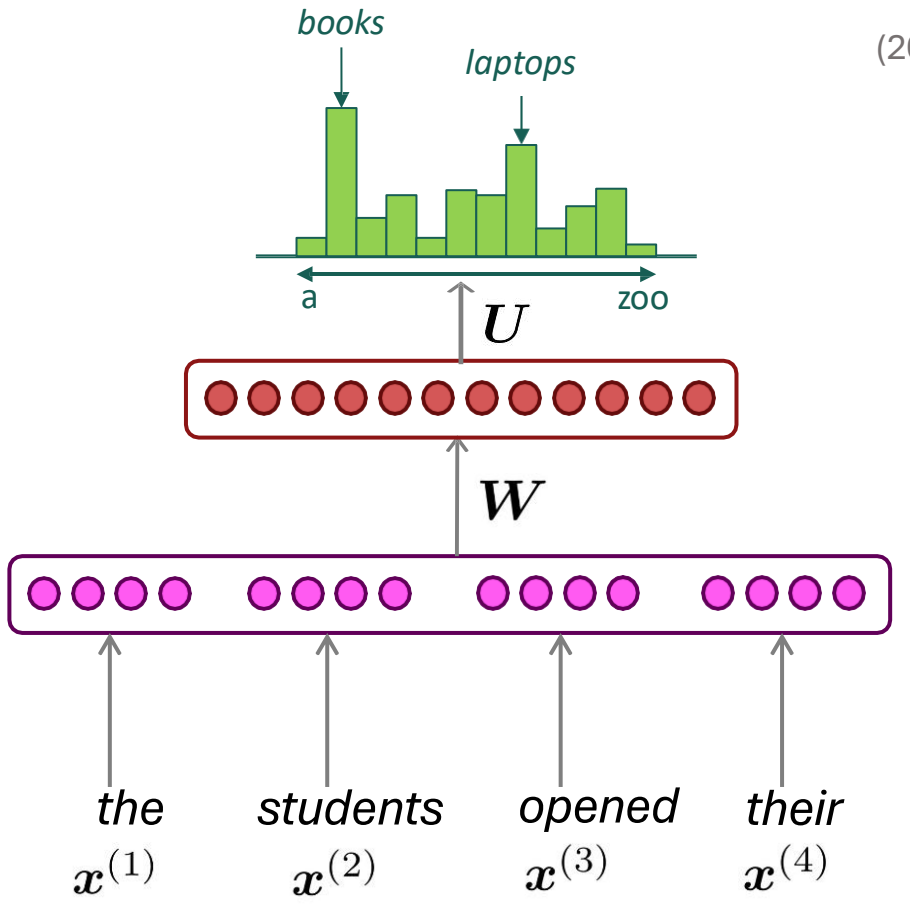
A Fixed-window Neural Language Model

Improvements over *n*-gram LM:

- No sparsity problem
- Don't need to store all observed *n*-grams

Remaining problems:

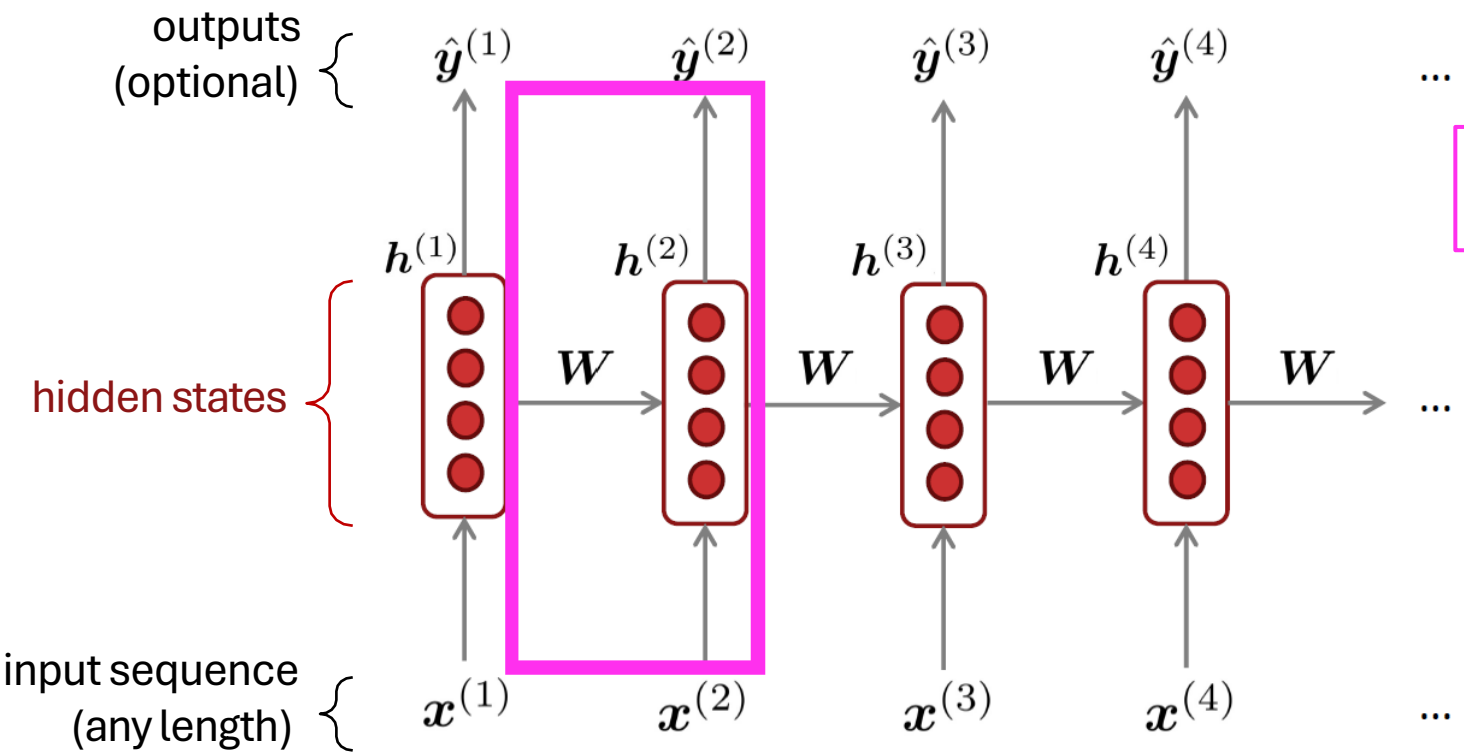
- Fixed window is too small
- Enlarging window enlarges W
- $x^{(1)}$ and $x^{(2)}$ are multiplied by completely different weights in W. No symmetry in how the inputs are processed.



Approximately: Y. Bengio, et al. (2000/2003): A Neural Probabilistic Language Model

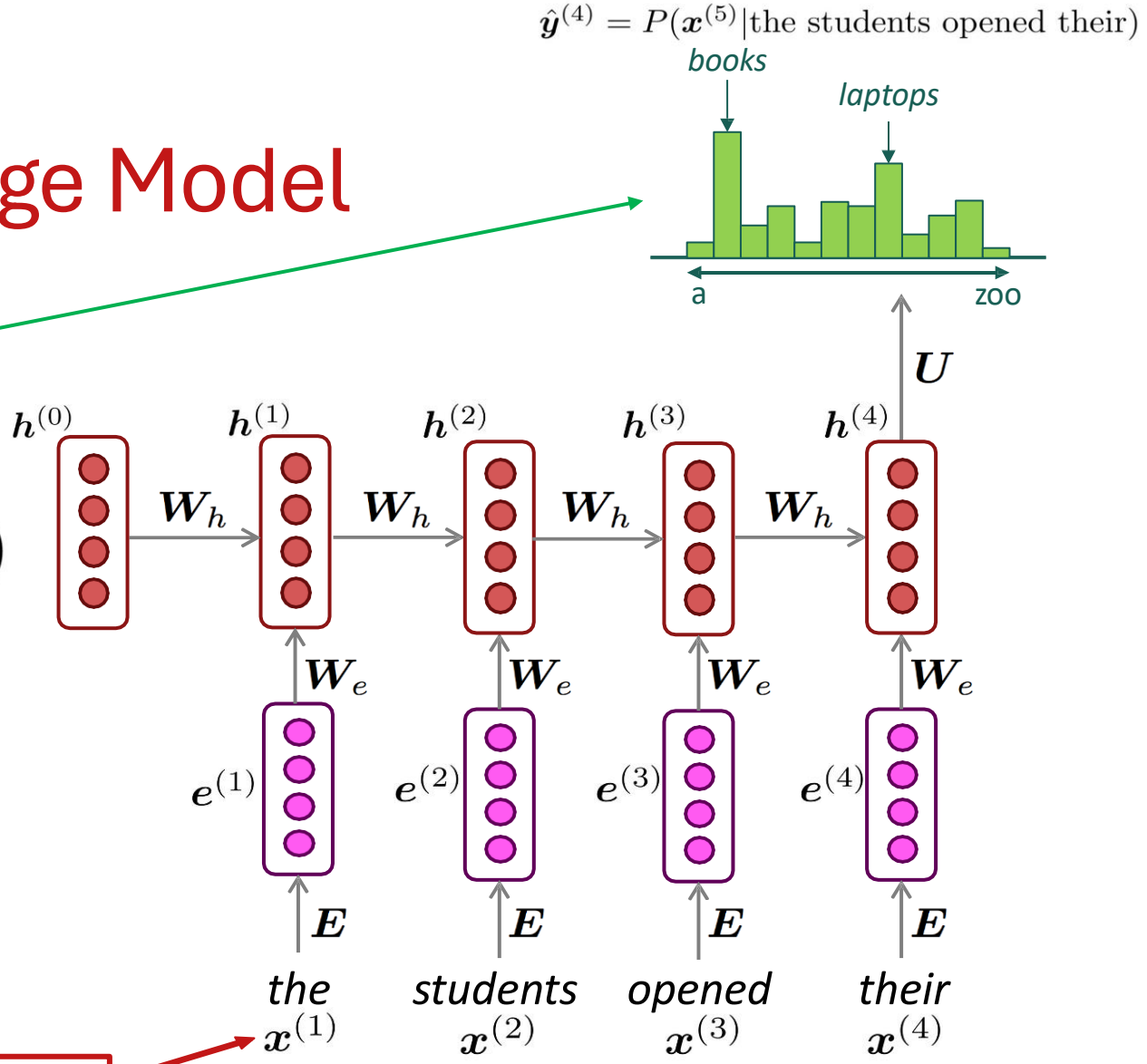
We need a neural architecture that can process ***any length input***

Recurrent Neural Networks (RNN)



Core idea: Apply the same weights *W repeatedly*

A Simple RNN Language Model



output distribution

$$\hat{\boldsymbol{y}}^{(t)} = \text{softmax}\left(\boldsymbol{U}\boldsymbol{h}^{(t)} + \boldsymbol{b}_2\right) \in \mathbb{R}^{|V|}$$

hidden states

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h\boldsymbol{h}^{(t-1)} + \boldsymbol{W}_e\boldsymbol{e}^{(t)} + \boldsymbol{b}_1\right)$$

$\boldsymbol{h}^{(0)}$ is the initial hidden state

word embeddings

$$\boldsymbol{e}^{(t)} = \boldsymbol{E}\boldsymbol{x}^{(t)}$$

words / one-hot vectors

$$\boldsymbol{x}^{(t)} \in \mathbb{R}^{|V|}$$

***Note**: this input sequence could be much longer now!*

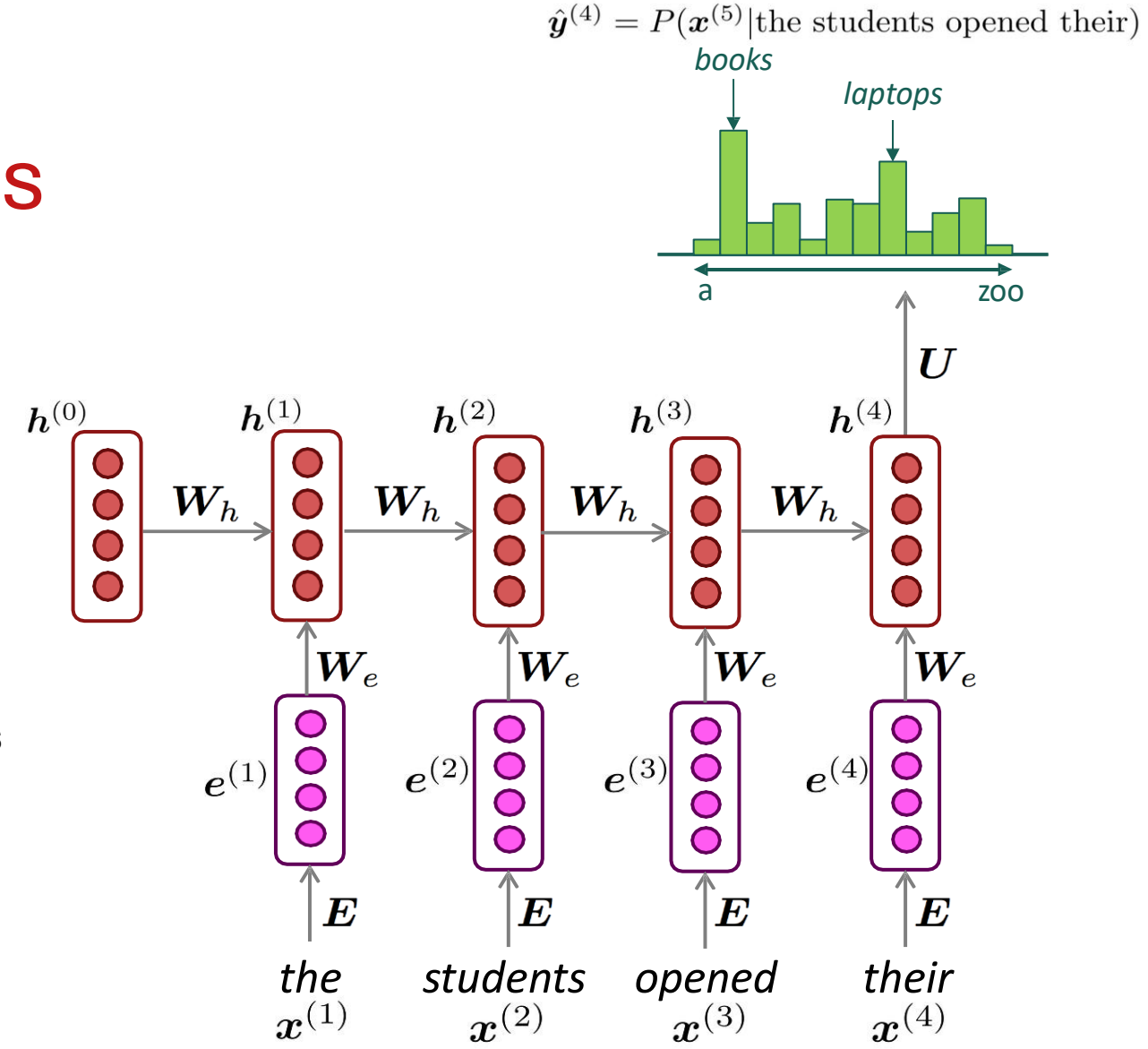
RNN Language Models

RNN Advantages:

- Can process any length input
- Computation for step *t* can (in theory) use information from many steps back
- Model size doesn't increase for longer input context
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back



Training an RNN Language Model

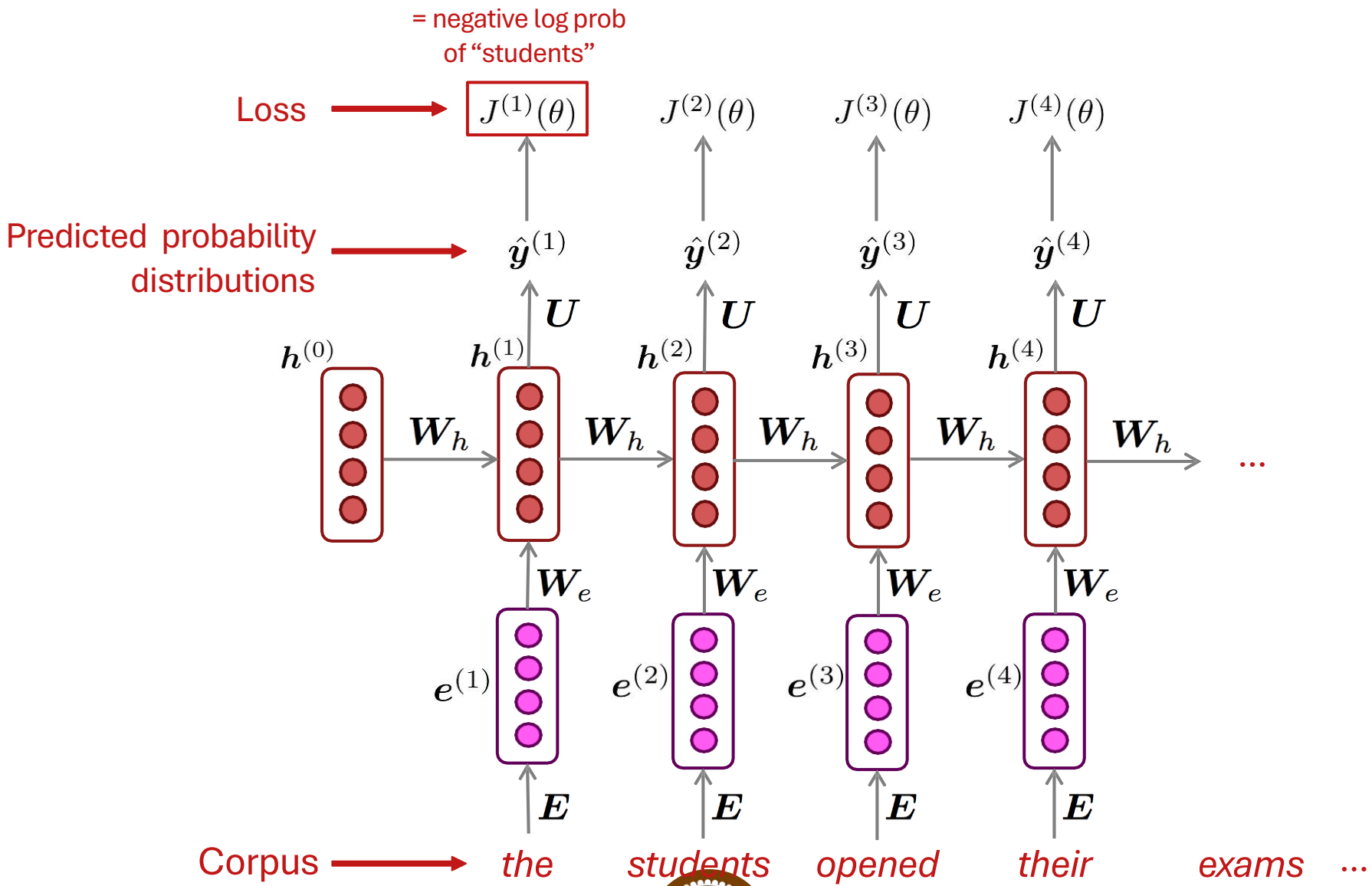
Training an RNN Language Model

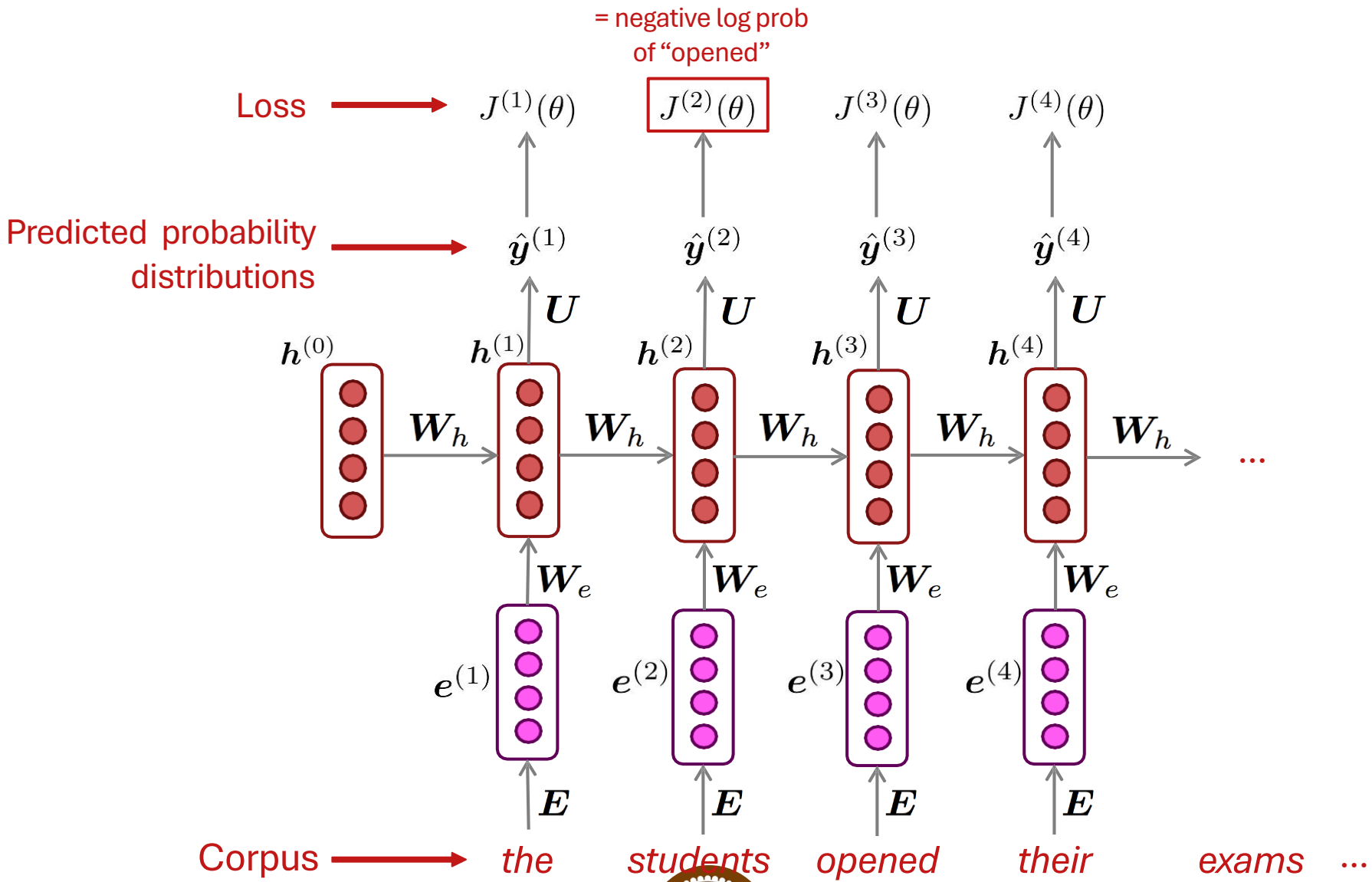
- Get a big corpus of text which is a sequence of words $x^{(1)}, x^{(2)}, \ldots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ for every step *t*.
  - i.e., predict probability distribution of every word, given words so far
- Loss function on step *t* is cross-entropy between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

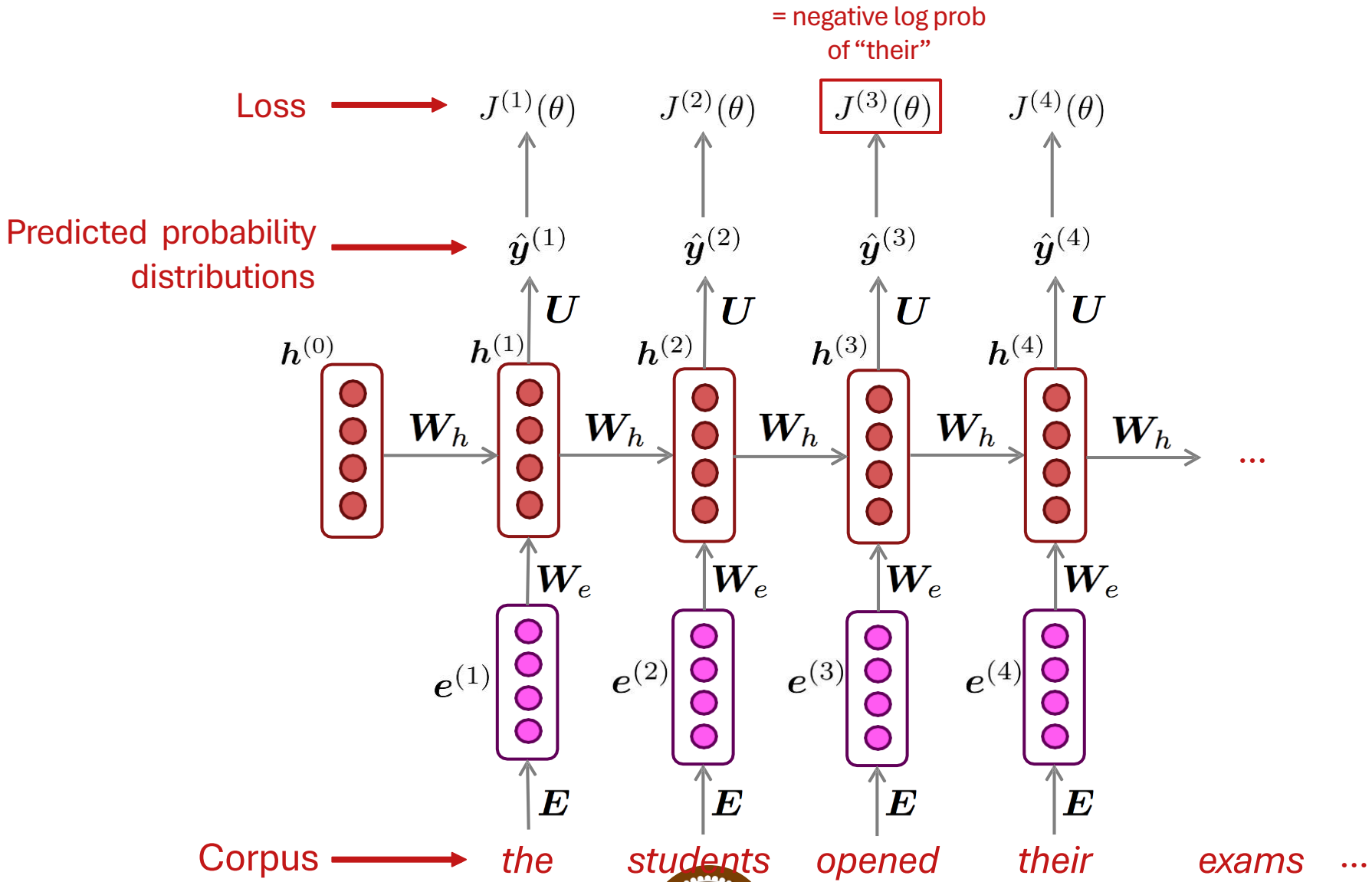
$$J^{(t)}(\theta) = CE(\boldsymbol{y}^{(t)}, \hat{\boldsymbol{y}}^{(t)}) = -\sum_{w \in V} \boldsymbol{y}_w^{(t)} \log \hat{\boldsymbol{y}}_w^{(t)} = -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

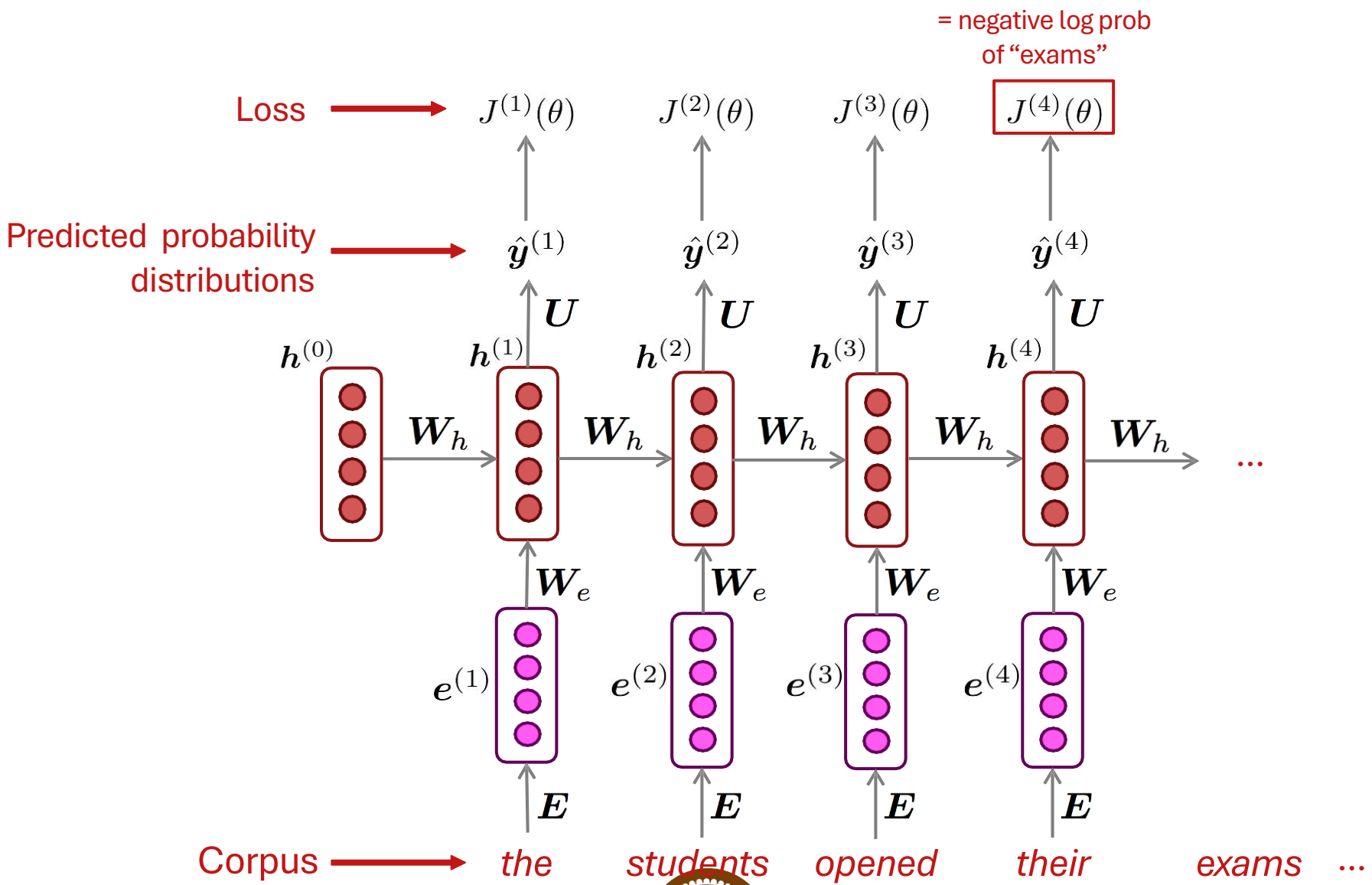
- Average this to get overall loss for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^{T} J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^{T} -\log \hat{\boldsymbol{y}}_{\boldsymbol{x}_{t+1}}^{(t)}$$

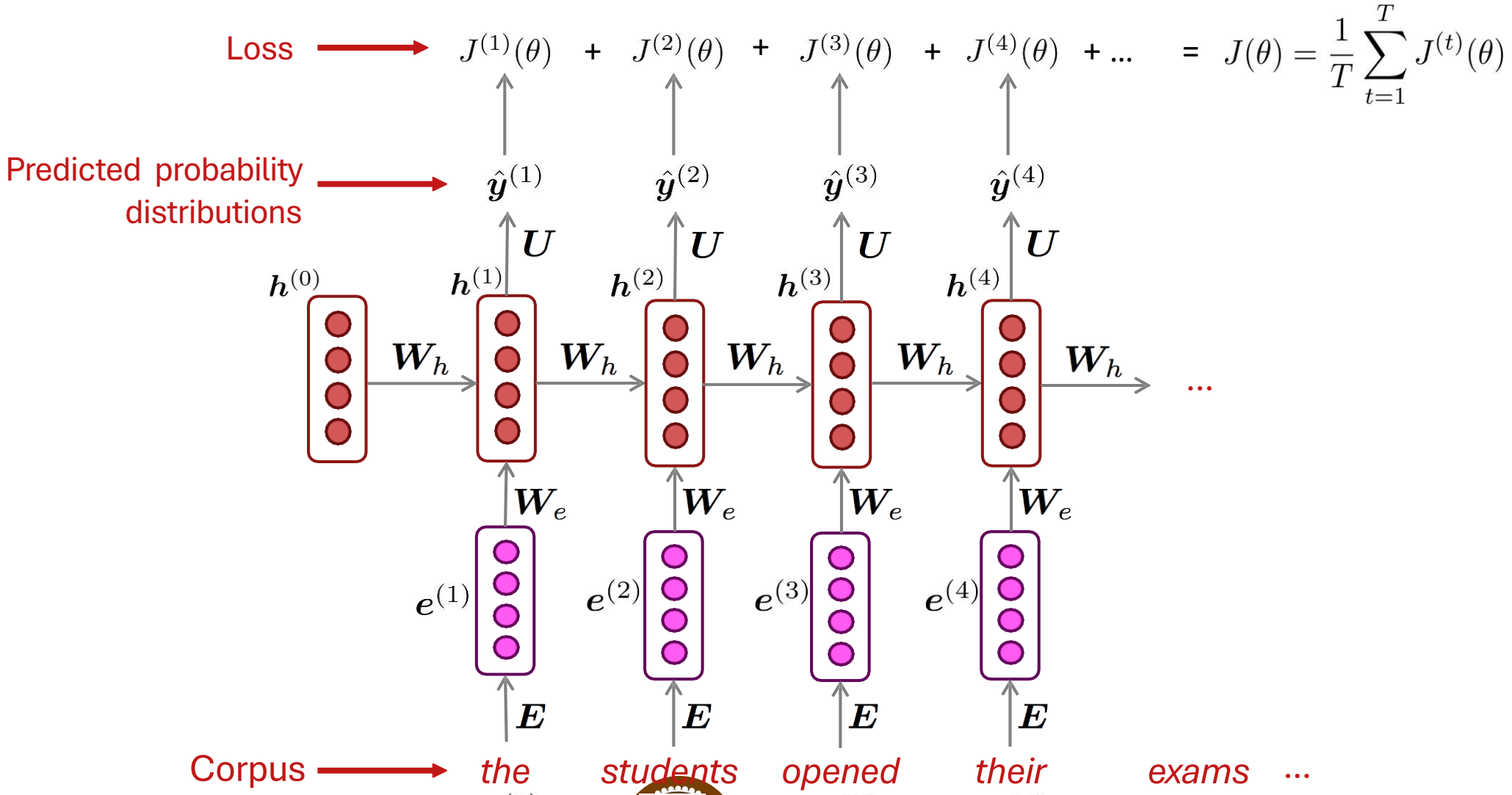

= negative log prob of "students"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
U
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
W_h
W_e
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
E
the
students
opened
their
exams
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$


= negative log prob of "opened"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
U
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
W_h
...
W_e
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
E
the
students
opened
their
exams
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$


= negative log prob of "their"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
U
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
W_h
...
W_e
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
E
the
students
opened
their
exams
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$


= negative log prob of "exams"
Loss
Predicted probability distributions
Corpus
$J^{(1)}(\theta)$
$J^{(2)}(\theta)$
$J^{(3)}(\theta)$
$J^{(4)}(\theta)$
$\hat{y}^{(1)}$
$\hat{y}^{(2)}$
$\hat{y}^{(3)}$
$\hat{y}^{(4)}$
U
$h^{(0)}$
$h^{(1)}$
$h^{(2)}$
$h^{(3)}$
$h^{(4)}$
W_h
...
W_e
$e^{(1)}$
$e^{(2)}$
$e^{(3)}$
$e^{(4)}$
E
the
students
opened
their
exams
...
$x^{(1)}$
$x^{(2)}$
$x^{(3)}$
$x^{(4)}$

"Teacher forcing"



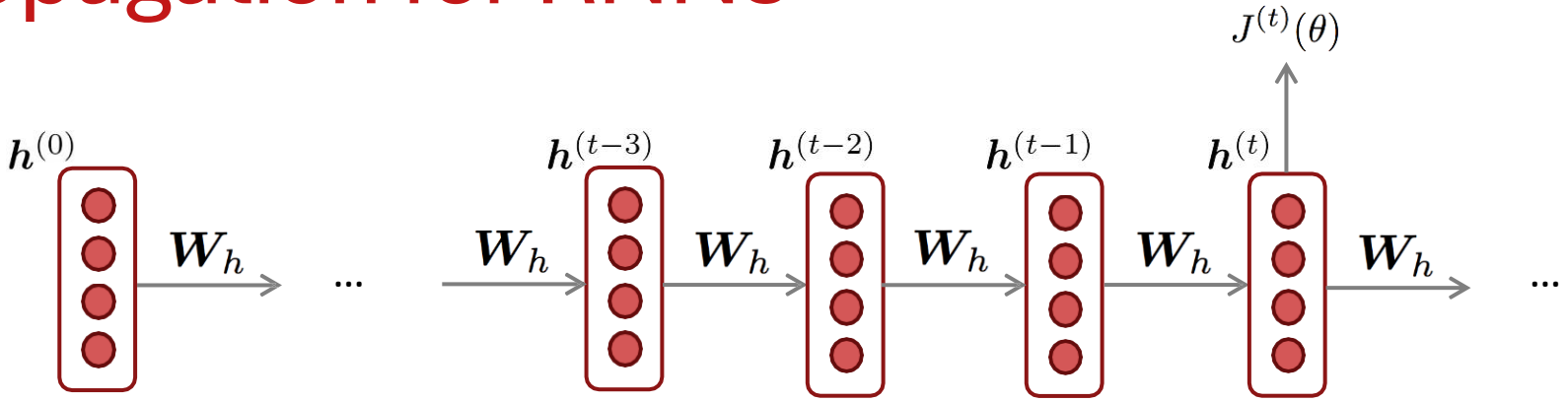
Training a RNN Language Model

- However: Computing loss and gradients across entire corpus $x^{(1)}, x^{(2)}, \ldots, x^{(T)}$ at once is too expensive (memory-wise)!

$$J(\theta) = \frac{1}{T}\sum_{t=1}^{T} J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, x^{(2)}, \ldots, x^{(T)}$ as a sentence (or a document)
- Recall: Stochastic Gradient Descent allows us to compute loss and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights. Repeat on a new batch of sentences.

Backpropagation for RNNs



Question: What's the derivative of $J^{(t)}(\theta)$ w.r.t the repeated weight matrix W_h ?

Answer:

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \left. \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} \right|_{(i)}$$

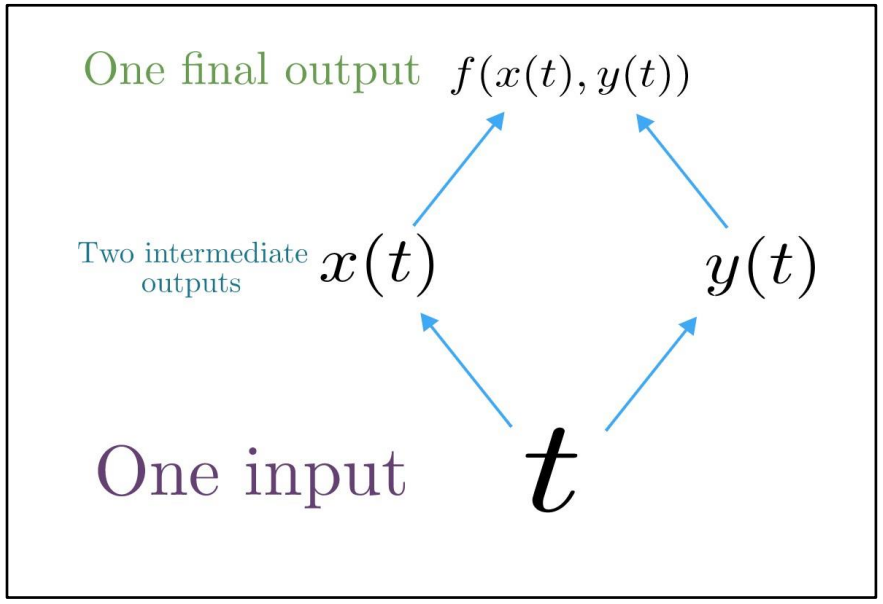
"The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears"

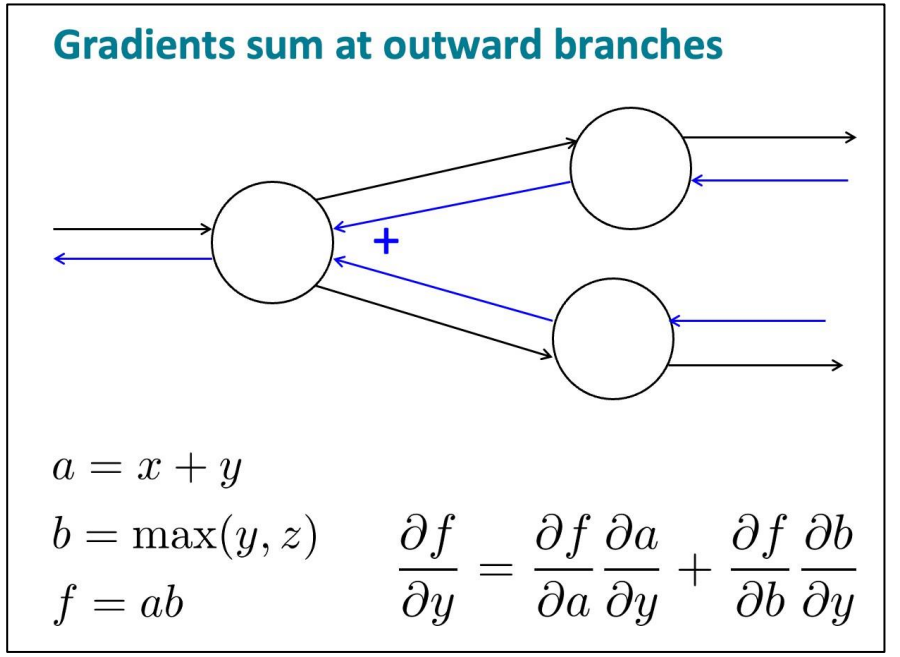
Why?

Multivariable Chain Rule

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

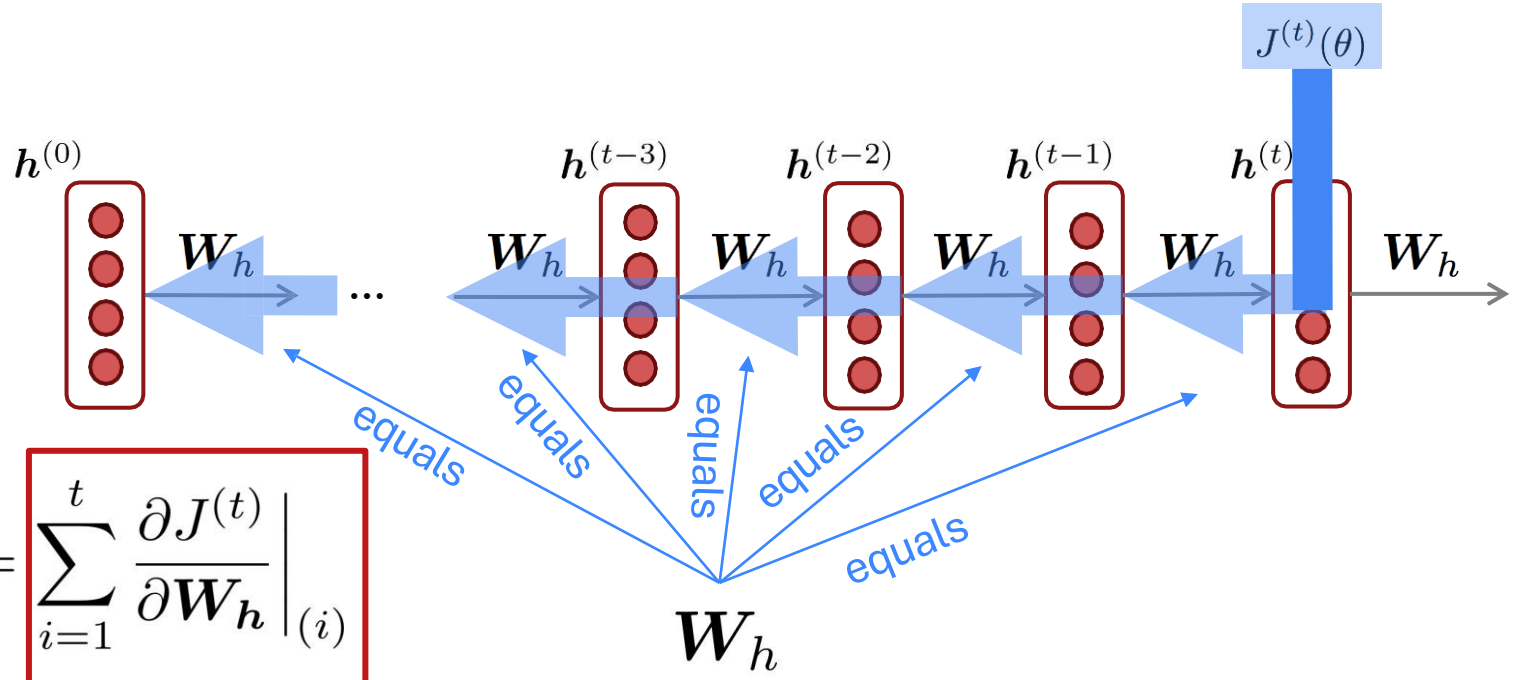
$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x}\frac{dx}{dt} + \frac{\partial f}{\partial y}\frac{dy}{dt}$$





Source:
https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/differentiating-vector-valued-functions/a/multivariable-chain-rule-simple-version

Training The Parameters of RNNs: Backpropagation for RNNs



In practice, often "truncated" after ~20 timesteps for training efficiency reasons

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}$$

Question: How do we calculate this?

Answer: Backpropagate over timesteps $i = t, \ldots, 0$, summing gradients as you go. This algorithm is called **"backpropagation through time"**

[Werbos, P.G., 1988, *Neural Networks* **1**, and others]

Apply the multivariable chain rule:

$$\frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}} = \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)} \frac{\partial \boldsymbol{W_h}|_{(i)}}{\partial \boldsymbol{W_h}}$$

= 1

$$= \sum_{i=1}^{t} \frac{\partial J^{(t)}}{\partial \boldsymbol{W_h}}\bigg|_{(i)}$$

Neural Language Models

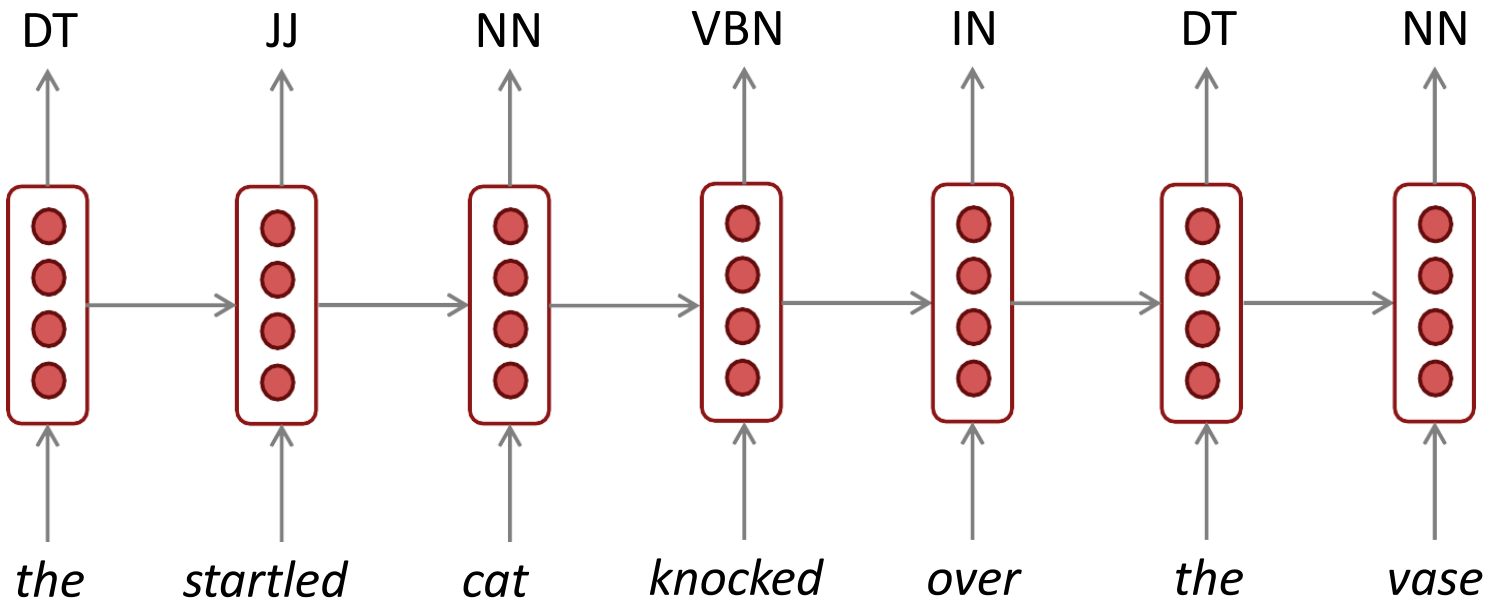
Tanmoy Chakraborty
Associate Professor, IIT Delhi
https://tanmoychak.com/

Introduction to Large Language Models

Uses of RNNs

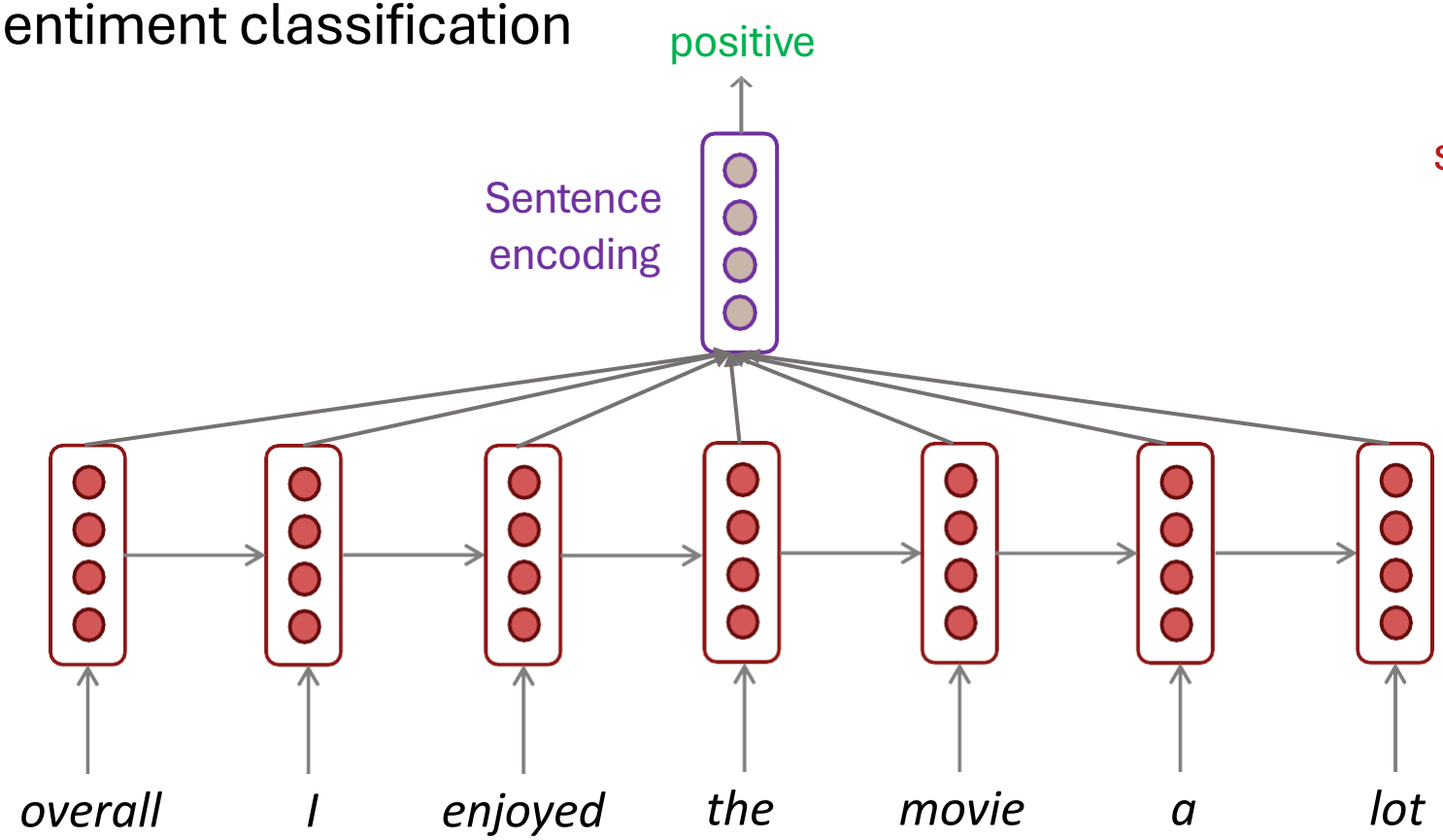
RNNs Can Be Used for Sequence Tagging

- **Example:** for part-of-speech tagging, named entity recognition



RNNs Can Be Used for Sentence Classification

- **Example:** for sentiment classification

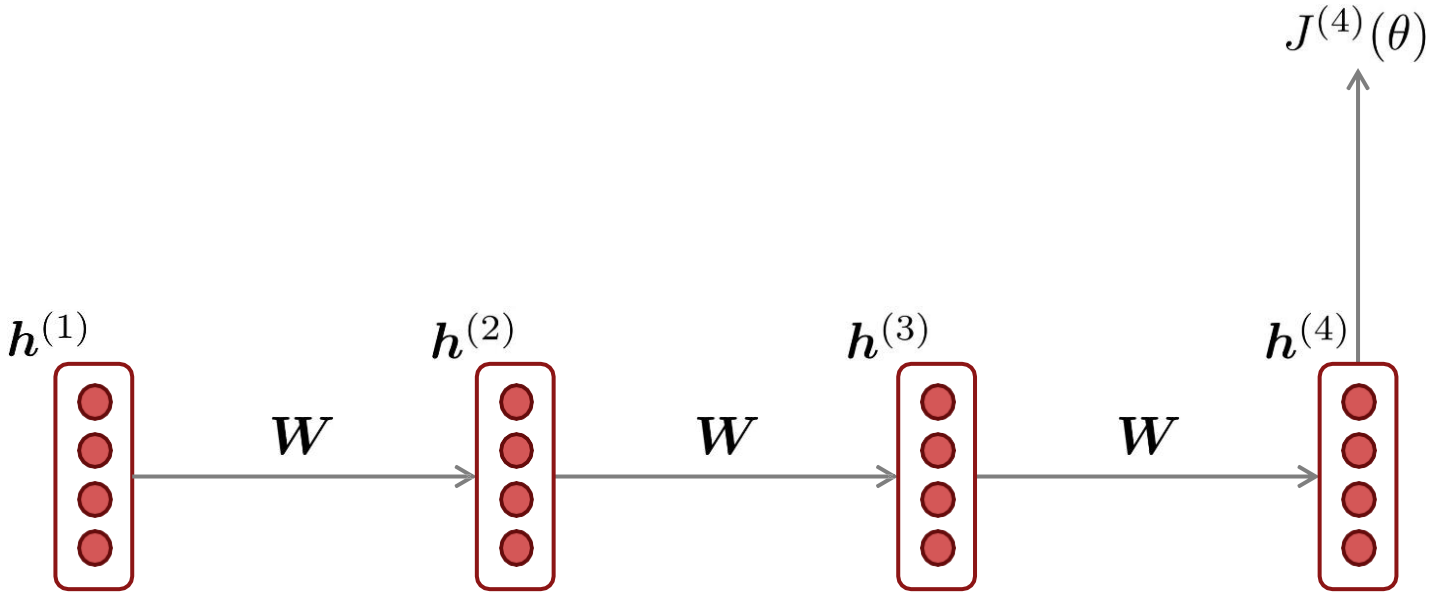


How to compute sentence encoding?

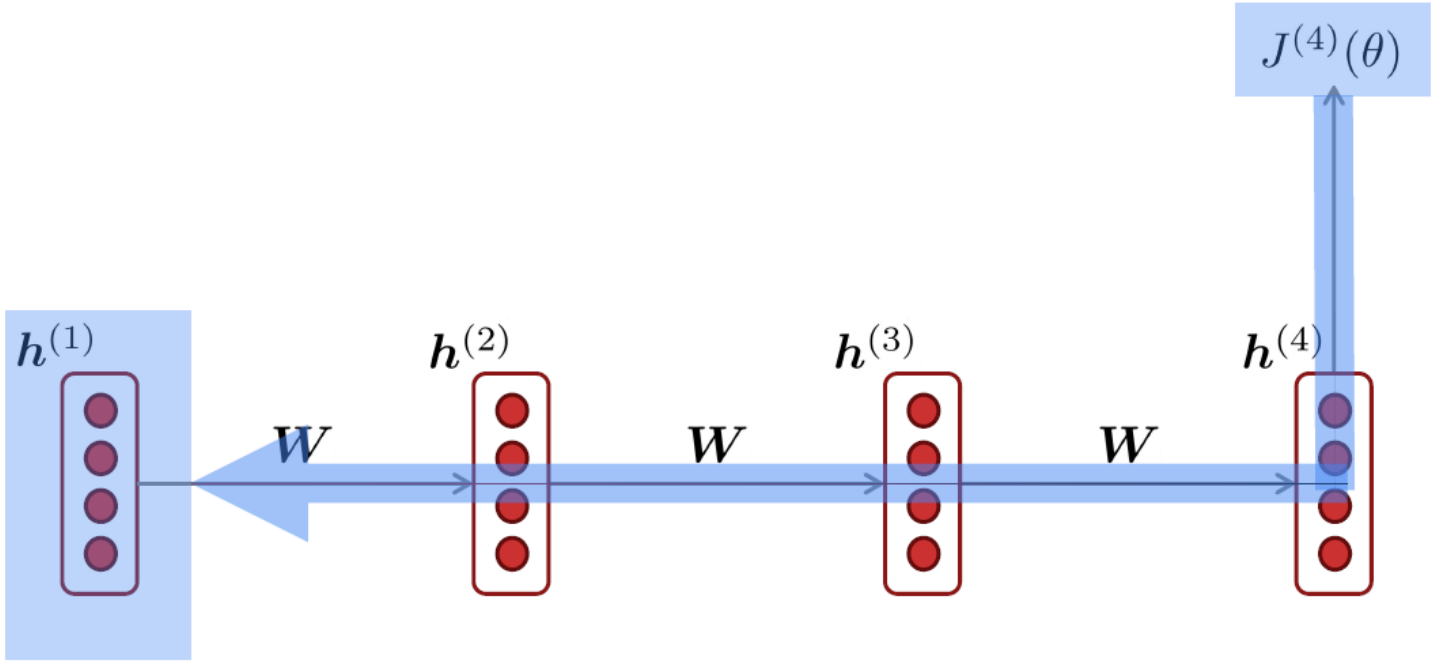
Usually better: Take element-wise max or mean of all hidden states

Problems with RNNs

Vanishing and Exploding Gradients

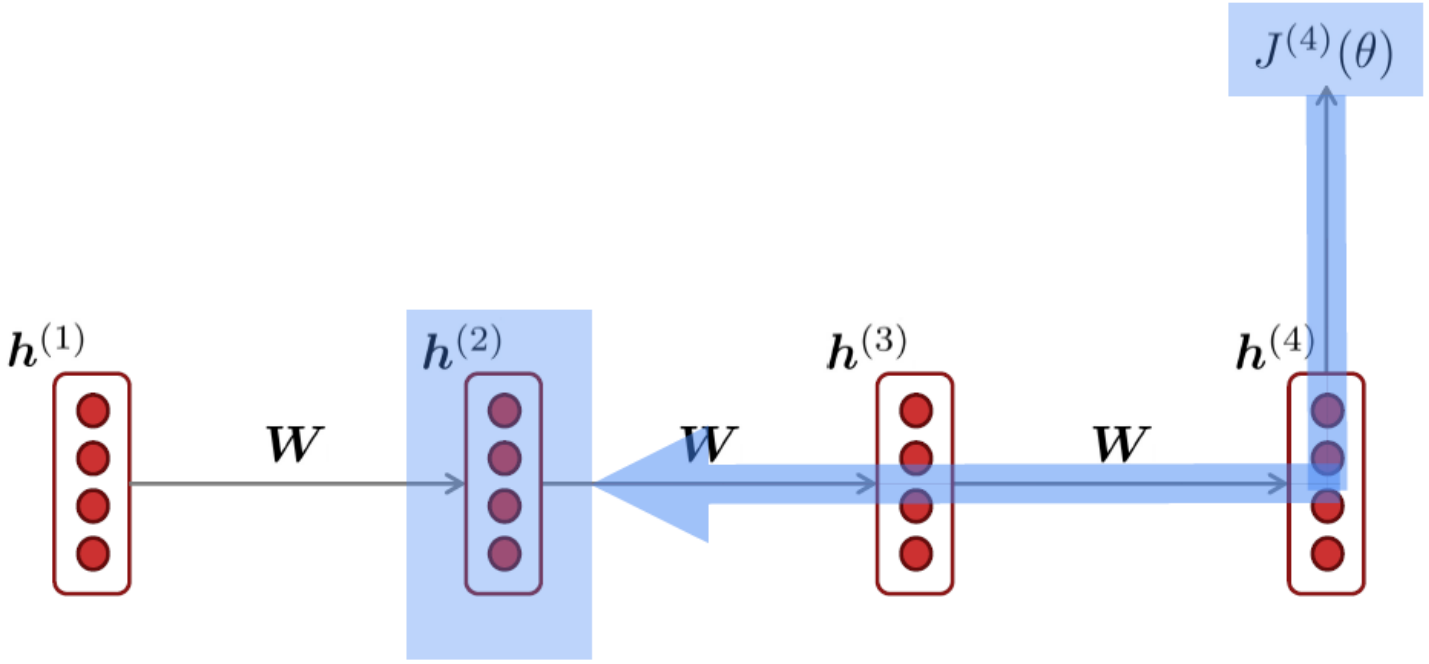


Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \ ?$$

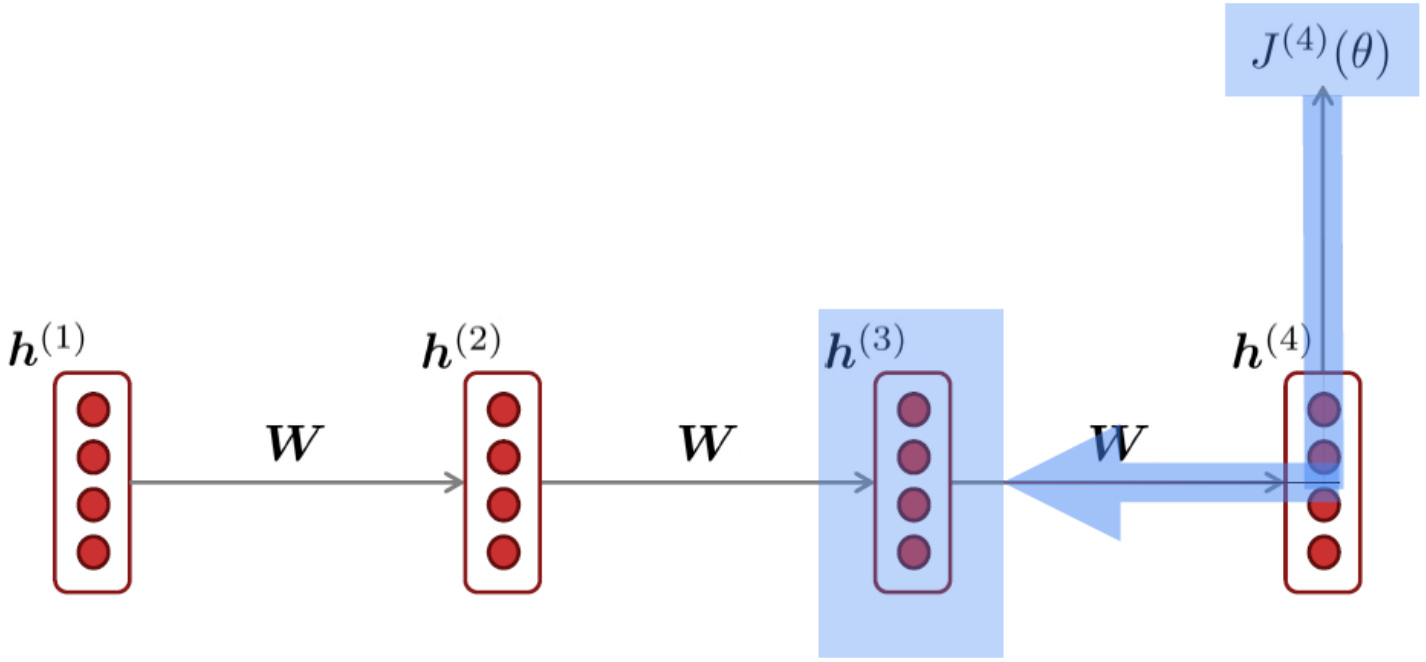
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$

Chain Rule!

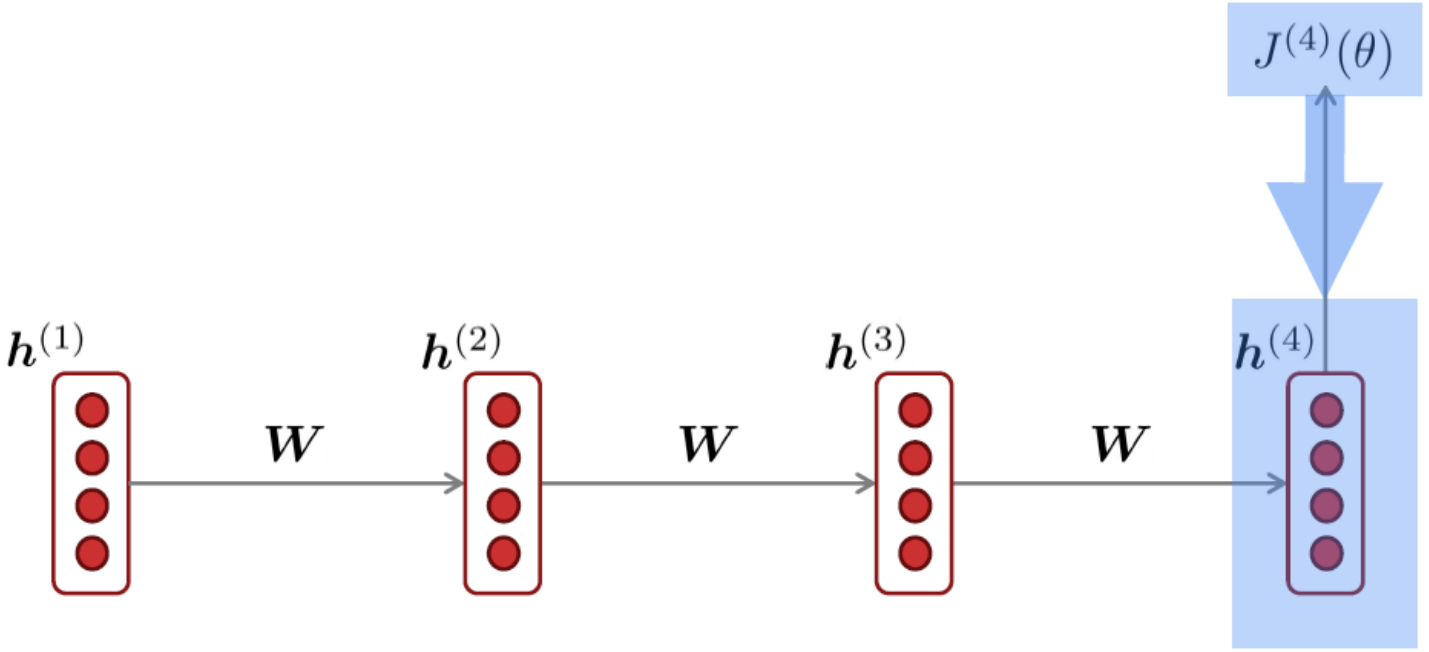
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial J^{(4)}}{\partial h^{(3)}}$$

Chain Rule!

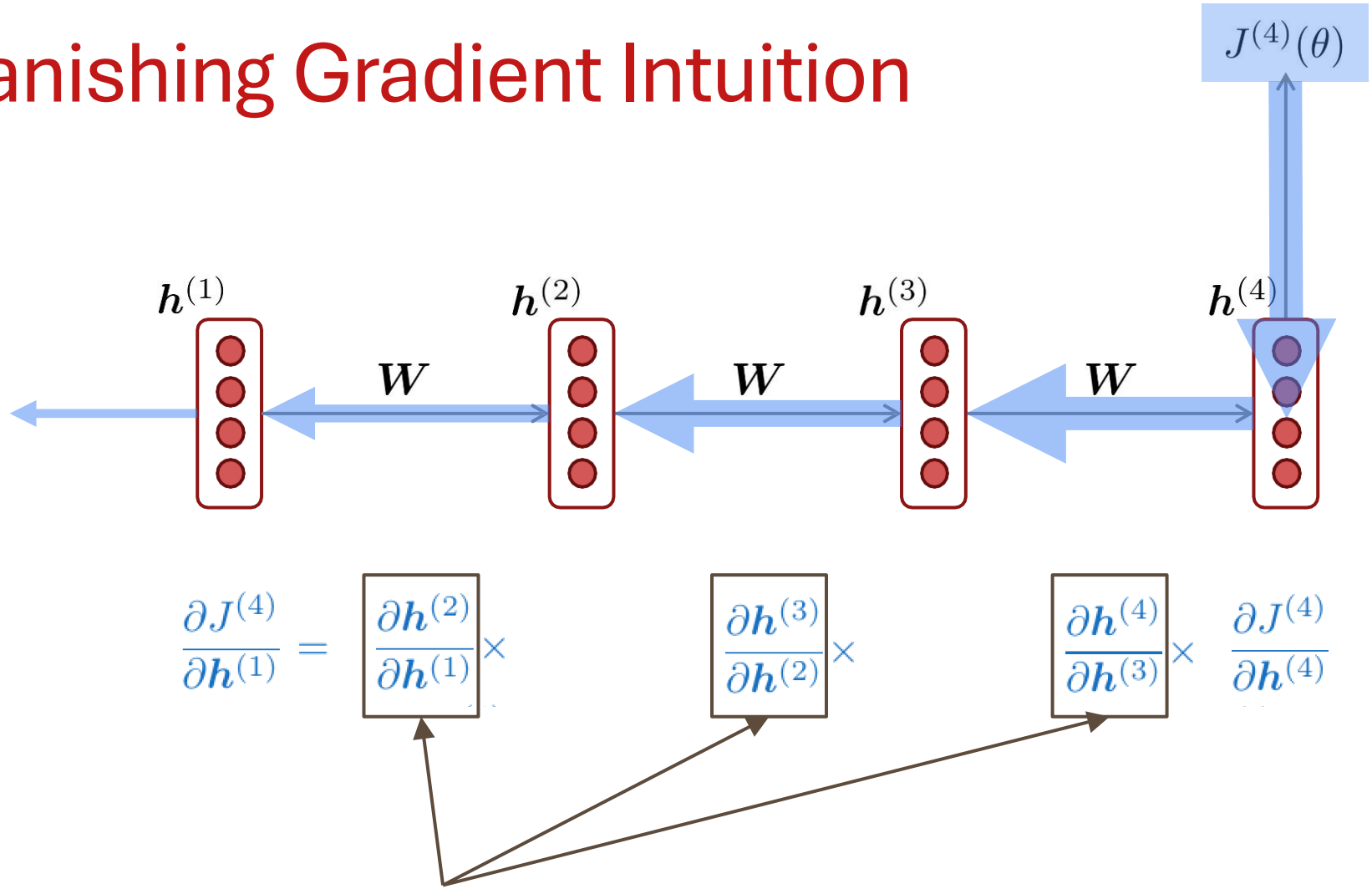
Vanishing Gradient Intuition



$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

Chain Rule!

Vanishing Gradient Intuition



Vanishing gradient problem: When these are small, the gradient signal gets smaller and smaller as it backpropagates further

Why is Vanishing Gradient a Problem?

Effect of Vanishing Gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her* _____
- To learn from this training example, the RNN-LM needs to model the dependency between *"tickets"* on the 7th step and the target word *"tickets"* at the end.
- But if the gradient is small, the model can't learn this dependency
  - So, the model is unable to predict similar long-distance dependencies at test time

How to Fix the Vanishing Gradient Problem?

- The main problem is that *it's too difficult for the RNN to learn to preserve information over many timesteps*.
- In a vanilla RNN, the hidden state is constantly being rewritten

$$\boldsymbol{h}^{(t)} = \sigma\left(\boldsymbol{W}_h \boldsymbol{h}^{(t-1)} + \boldsymbol{W}_x \boldsymbol{x}^{(t)} + \boldsymbol{b}\right)$$

- How about an RNN with separate memory which is added to?
  - LSTMs
- And then: Creating more direct and linear pass-through connections in model
  - Attention, residual connections, etc.

LSTMs & GRUs

Long Short-Term Memory (LSTM)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step *t*, there is a hidden state $h^{(t)}$ and a cell state $c^{(t)}$
  - Both are vectors length *n*
  - The cell stores long-term information
  - The LSTM can erase, write and read information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding gates (gates are calculated things whose values are probabilities)
  - The gates are also vectors length *n*
  - On each timestep, each element of the gates can be open (1), closed (0), or somewhere in-between.
  - The gates are dynamic: their value is computed based on the current context

LSTM

We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep *t*:

Sigmoid function: all gate values are between 0 and 1

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

$$\boldsymbol{f}^{(t)} = \sigma\left(\boldsymbol{W}_f \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_f \boldsymbol{x}^{(t)} + \boldsymbol{b}_f\right)$$

$$\boldsymbol{i}^{(t)} = \sigma\left(\boldsymbol{W}_i \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_i \boldsymbol{x}^{(t)} + \boldsymbol{b}_i\right)$$

$$\boldsymbol{o}^{(t)} = \sigma\left(\boldsymbol{W}_o \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_o \boldsymbol{x}^{(t)} + \boldsymbol{b}_o\right)$$

New cell content: this is the new content to be written to the cell

Cell state: erase ("forget") some content from last cell state, and write ("input") some new cell content

Hidden state: read ("output") some content from the cell

$$\tilde{\boldsymbol{c}}^{(t)} = \tanh\left(\boldsymbol{W}_c \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_c \boldsymbol{x}^{(t)} + \boldsymbol{b}_c\right)$$

$$\boldsymbol{c}^{(t)} = \boldsymbol{f}^{(t)} \odot \boldsymbol{c}^{(t-1)} + \boldsymbol{i}^{(t)} \odot \tilde{\boldsymbol{c}}^{(t)}$$

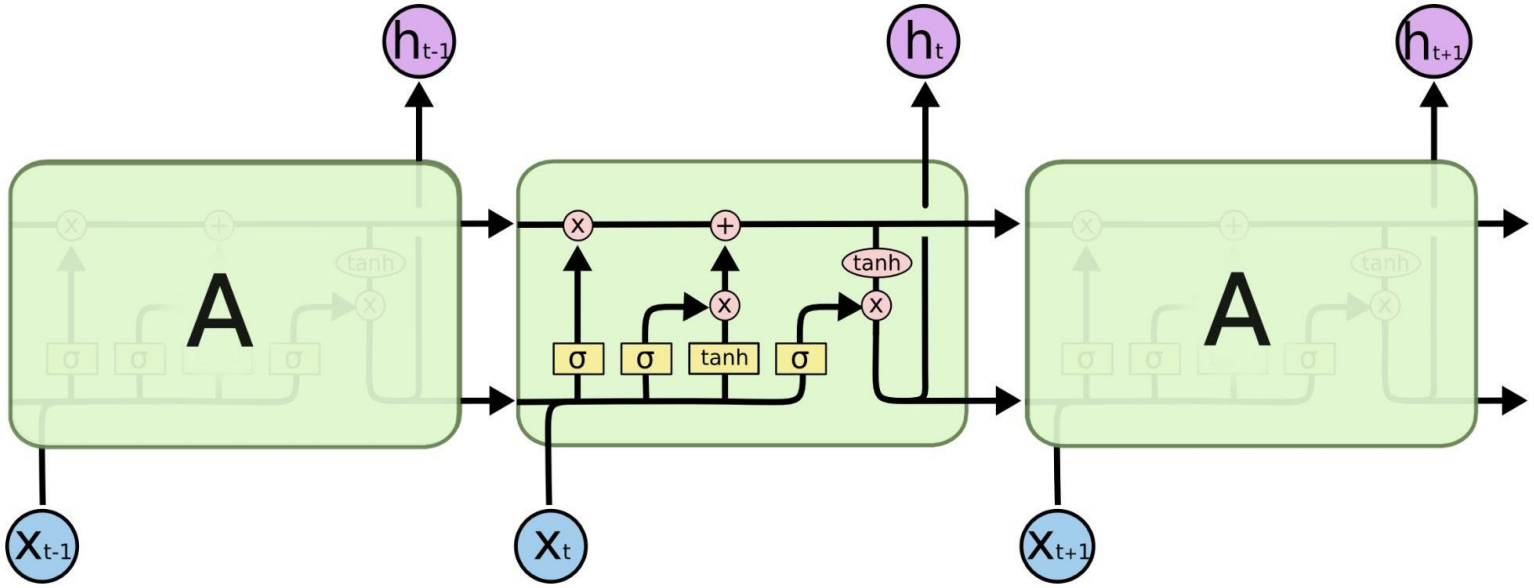
$$\boldsymbol{h}^{(t)} = \boldsymbol{o}^{(t)} \odot \tanh \boldsymbol{c}^{(t)}$$

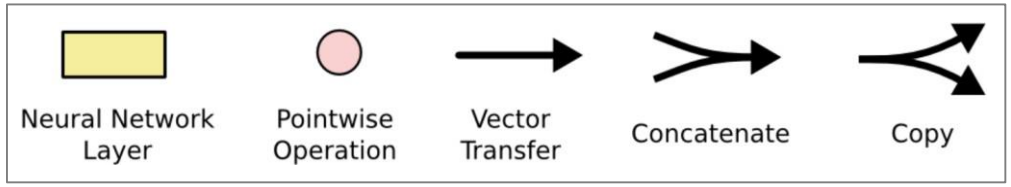
All these are vectors of same length *n*

Gates are applied using element-wise (or Hadamard) product: $\odot$

Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:





Source: http://colah.github.io/posts/2015-08-Understanding-LSTMs/

How Does LSTM Solve Vanishing Gradients?

- The LSTM architecture makes it much easier for an RNN to preserve information over many timesteps
  - For example, if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely.
  - In contrast, it's harder for a vanilla RNN to learn a recurrent weight matrix W_h that preserves info in the hidden state
  - In practice, you get about 100 timesteps rather than about 7
- LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies.
- There are also alternative ways of creating more direct and linear pass-through connections in models for long distance dependencies.

Is Vanishing/Exploding Gradient Just an RNN Problem?

- No! It can be a problem for all neural architectures (including feed-forward and convolutional neural networks), especially very deep ones.
  - Due to chain rule / choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
  - Thus, lower layers are learned very slowly (i.e., are hard to train)
- **Another solution:** lots of new deep feedforward/convolutional architectures add more direct connections (thus allowing the gradient to flow)

For example:

- Residual connections aka "ResNet"
- Also known as skip-connections
- The identity connection preserves information by default
- This makes deep networks much easier to train

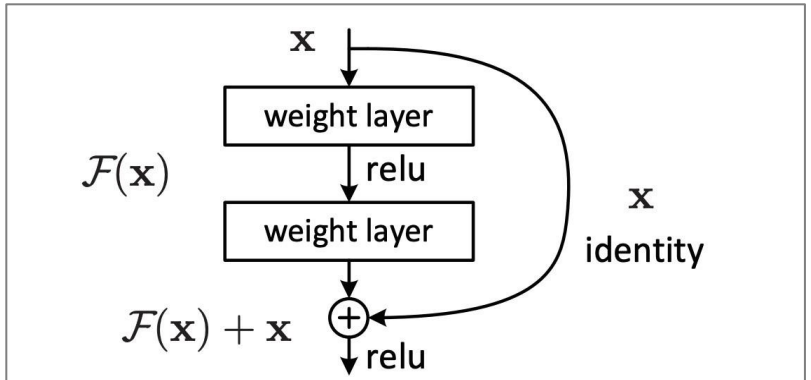


Figure 2. Residual learning: a building block.

"Deep Residual Learning for Image Recognition", He et al, 2015. https://arxiv.org/pdf/1512.03385.pdf

Is Vanishing/Exploding Gradient Just an RNN Problem?

Other Methods:

- Dense connections aka "DenseNet"
- Directly connect each layer to all future layers!

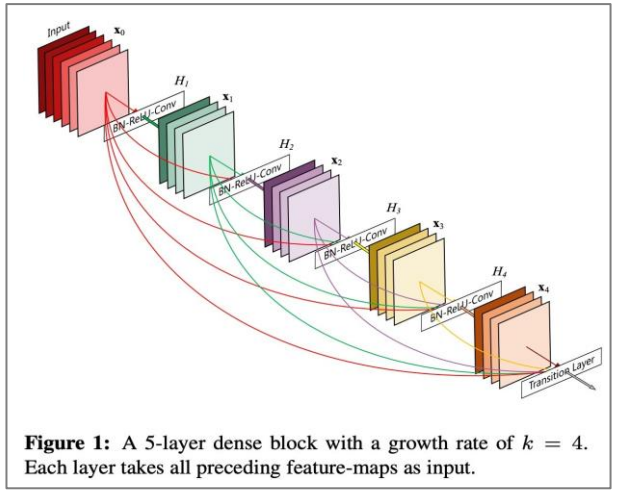
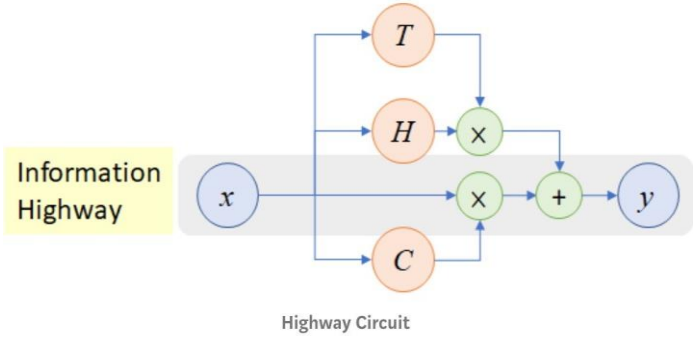


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

- Highway connections aka "HighwayNet"
- Similar to residual connections, but the identity connection vs the transformation layer is controlled by a dynamic gate
- Inspired by LSTMs, but applied to deep feedforward/convolutional networks



Highway Circuit

Conclusion: Though vanishing/exploding gradients are a general problem, RNNs are particularly unstable due to the repeated multiplication by the same weight matrix.

Gated Recurrent Units (GRUs)

- Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM.
- On each timestep t, we have input $x^{(t)}$ and hidden state $h^{(t)}$ (no cell state).

Update gate: controls what parts of hidden state are updated vs preserved

Reset gate: controls what parts of previous hidden state are used to compute new content

New hidden state content: reset gate selects useful parts of prev hidden state. Use this and current input to compute new hidden content.

Hidden state: update gate simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

How does this solve vanishing gradient?
Like LSTM, GRU makes it easier to retain info long-term (e.g. by setting update gate to 0)

$$\boldsymbol{u}^{(t)} = \sigma\left(\boldsymbol{W}_u \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_u \boldsymbol{x}^{(t)} + \boldsymbol{b}_u\right)$$

$$\boldsymbol{r}^{(t)} = \sigma\left(\boldsymbol{W}_r \boldsymbol{h}^{(t-1)} + \boldsymbol{U}_r \boldsymbol{x}^{(t)} + \boldsymbol{b}_r\right)$$

$$\tilde{\boldsymbol{h}}^{(t)} = \tanh\left(\boldsymbol{W}_h(\boldsymbol{r}^{(t)} \circ \boldsymbol{h}^{(t-1)}) + \boldsymbol{U}_h \boldsymbol{x}^{(t)} + \boldsymbol{b}_h\right)$$

$$\boldsymbol{h}^{(t)} = (1 - \boldsymbol{u}^{(t)}) \circ \boldsymbol{h}^{(t-1)} + \boldsymbol{u}^{(t)} \circ \tilde{\boldsymbol{h}}^{(t)}$$

"Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation", Cho et al. 2014, https://arxiv.org/pdf/1406.1078v3.pdf

LSTM vs GRU

- Researchers have proposed many gated RNN variants, but LSTM and GRU are the most widely-used.
- The biggest difference is that GRU is quicker to compute and has fewer parameters.
- There is no conclusive evidence that one consistently performs better than the other.
- LSTM is a good default choice (especially if your data has particularly long dependencies, or you have lots of training data).
- **Rule of thumb:** start with LSTM, but switch to GRU if you want something more efficient.

Bidirectional RNNs

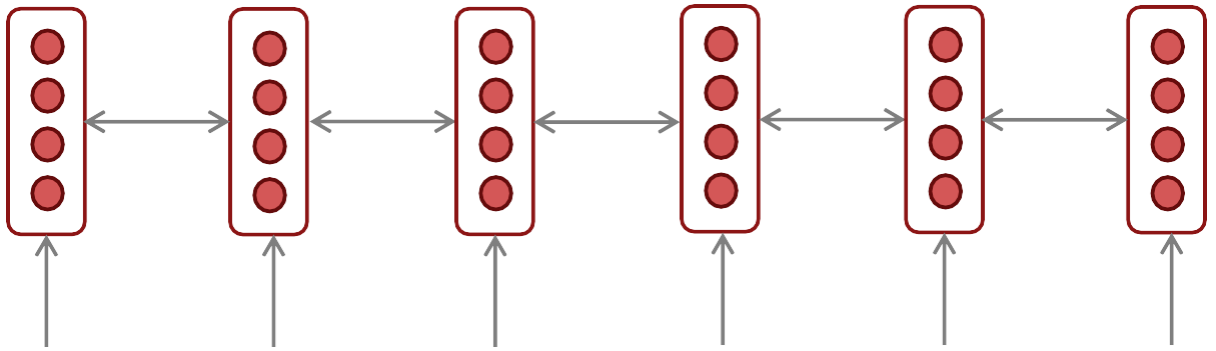
On timestep *t* :

Forward RNN $\overrightarrow{\boldsymbol{h}}^{(t)} = \boxed{\mathrm{RNN}_{\mathrm{FW}}}(\overrightarrow{\boldsymbol{h}}^{(t-1)}, \boldsymbol{x}^{(t)})$

Backward RNN $\overleftarrow{\boldsymbol{h}}^{(t)} = \mathrm{RNN}_{\mathrm{BW}}(\overleftarrow{\boldsymbol{h}}^{(t+1)}, \boldsymbol{x}^{(t)})$

Generally, these two RNNs have separate weights

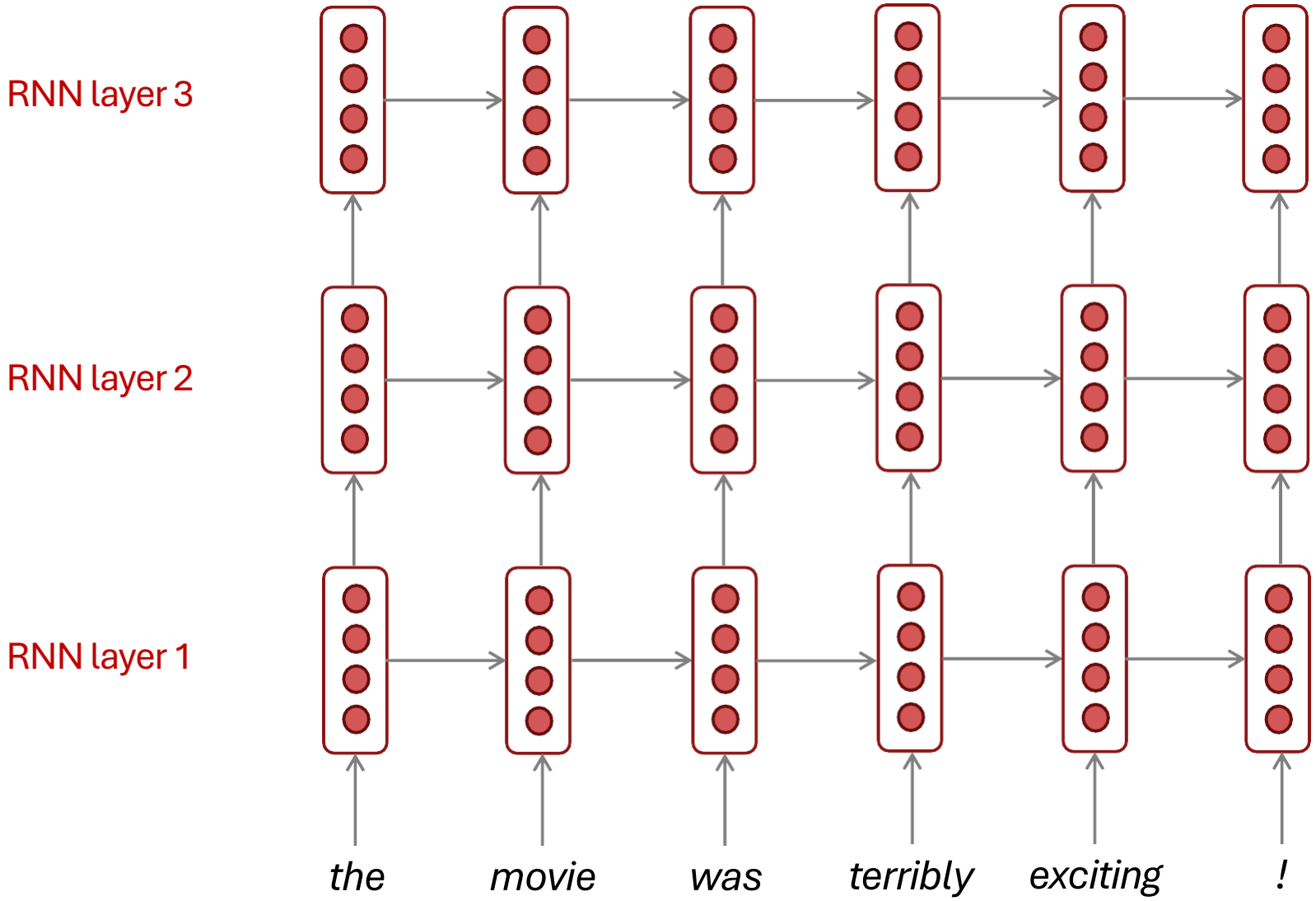
Concatenated hidden states $\boxed{\boldsymbol{h}^{(t)}} = [\overrightarrow{\boldsymbol{h}}^{(t)}; \overleftarrow{\boldsymbol{h}}^{(t)}]$



Multi-layer RNNs

- RNNs are already “deep” on one dimension (they unroll over many timesteps)
- We can also make them “deep” in another dimension by applying multiple RNNs – this is a multi-layer RNN.
- This allows the network to compute more complex representations
  - The lower RNNs should compute lower-level features and the higher RNNs should compute higher-level features.
- Multi-layer RNNs are also called *stacked RNNs*.

The hidden states from RNN layer *i* are the inputs to RNN layer *i*+1



LSTMs: Real-world Success

- In 2013–2015, LSTMs started achieving state-of-the-art results
  - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
  - LSTMs became the dominant approach for most NLP tasks
- Now (2019–2024), Transformers have become dominant for all tasks
  - For example, in WMT (a Machine Translation conference + competition):
    - In WMT 2014, there were 0 neural machine translation systems (!)
    - In WMT 2016, the summary report contains "RNN" 44 times (and these systems won)
    - In WMT 2019: "RNN" 7 times, "Transformer" 105 times

Source: "Findings of the 2016 Conference on Machine Translation (WMT16)", Bojar et al. 2016, http://www.statmt.org/wmt16/pdf/W16-2301.pdf
Source: "Findings of the 2018 Conference on Machine Translation (WMT18)", Bojar et al. 2018, http://www.statmt.org/wmt18/pdf/WMT028.pdf
Source: "Findings of the 2019 Conference on Machine Translation (WMT19)", Barrault et al. 2019, http://www.statmt.org/wmt18/pdf/WMT028.pdf

Sequence-to-Sequence Modelling

Tanmoy Chakraborty
Associate Professor, IIT Delhi
https://tanmoychak.com/

Introduction to Large Language Models

Sequence-to-Sequence Modeling

Neural Machine Translation?

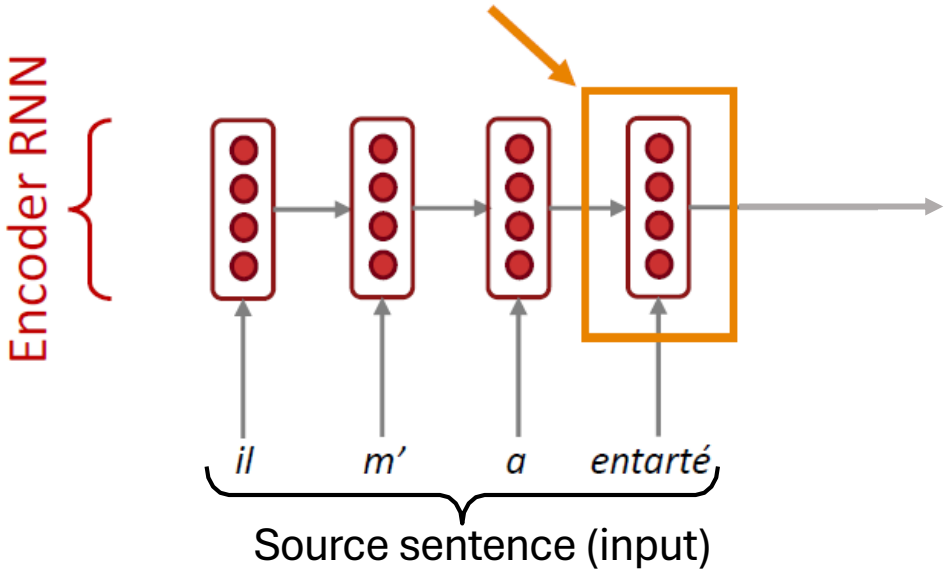
- Neural Machine Translation (NMT) is a way to do Machine Translation with a *single neural network*.

- The neural network architecture is called sequence-to-sequence (aka seq2seq) and it involves *two* RNNs.

Neural Machine Translation (NMT)

The Sequence-to-Sequence Model

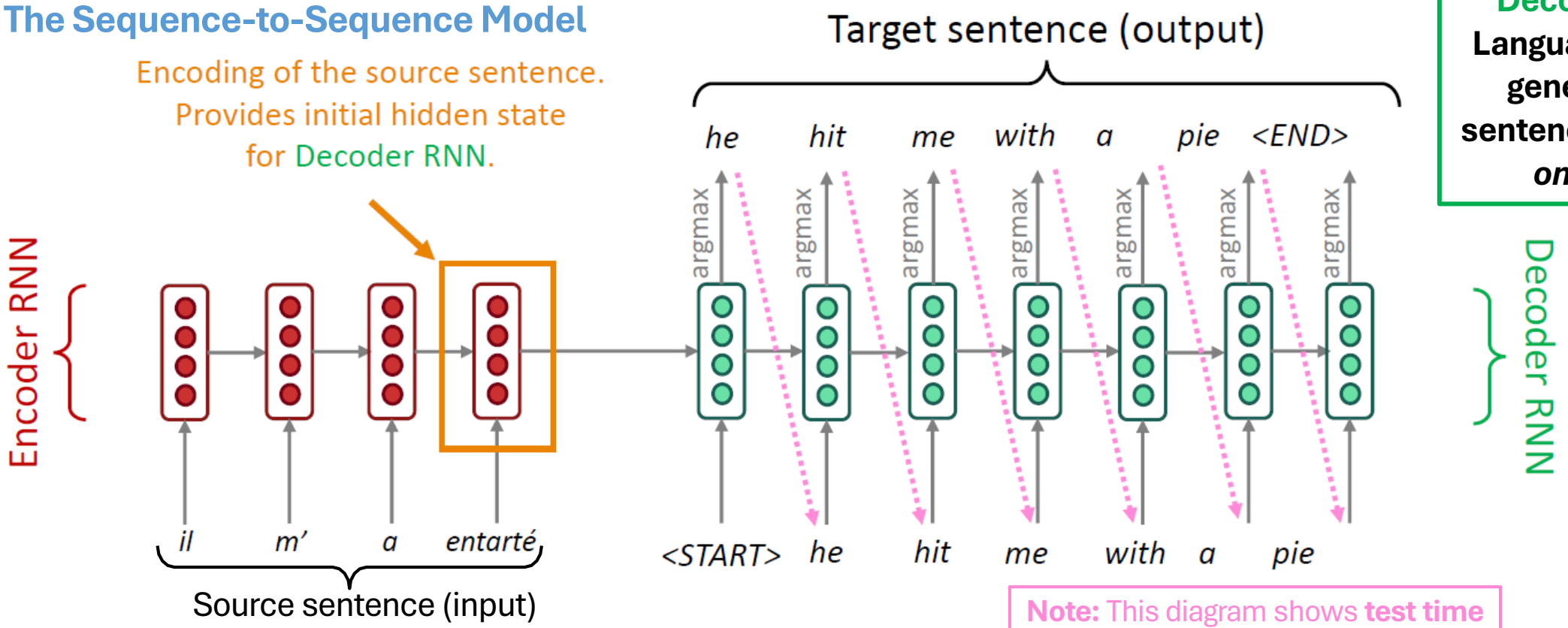
Encoding of the source sentence.
Provides initial hidden state
for Decoder RNN.



Encoder RNN produces an encoding of the source sentence.

Neural Machine Translation (NMT)

The Sequence-to-Sequence Model



Decoder RNN is a Language Model that generates target sentence, *conditioned on encoding*.

Encoder RNN produces an encoding of the source sentence.

Note: This diagram shows **test time** behavior: decoder output is fed in as next step's input

Sequence-to-Sequence is Versatile!

- The general notion here is an encoder-decoder model
  - One neural network takes input and produces a neural representation
  - Another network produces output based on that neural representation
  - If the input and output are sequences, we call it a seq2seq model
- Sequence-to-sequence is useful for *more than just MT*
- Many NLP tasks can be phrased as sequence-to-sequence:
  - **Summarization** (long text → short text)
  - **Dialogue** (previous utterances → next utterance)
  - **Parsing** (input text → output parse as sequence)
  - **Code generation** (natural language → Python code)

Neural Machine Translation (NMT)

- The sequence-to-sequence model is an example of a **Conditional Language Model**
  - **Language Model** because the decoder is predicting the next word of the target sentence *y*
  - **Conditional** because its predictions are also conditioned on the source sentence *x*
- NMT directly calculates $P(y|x)$

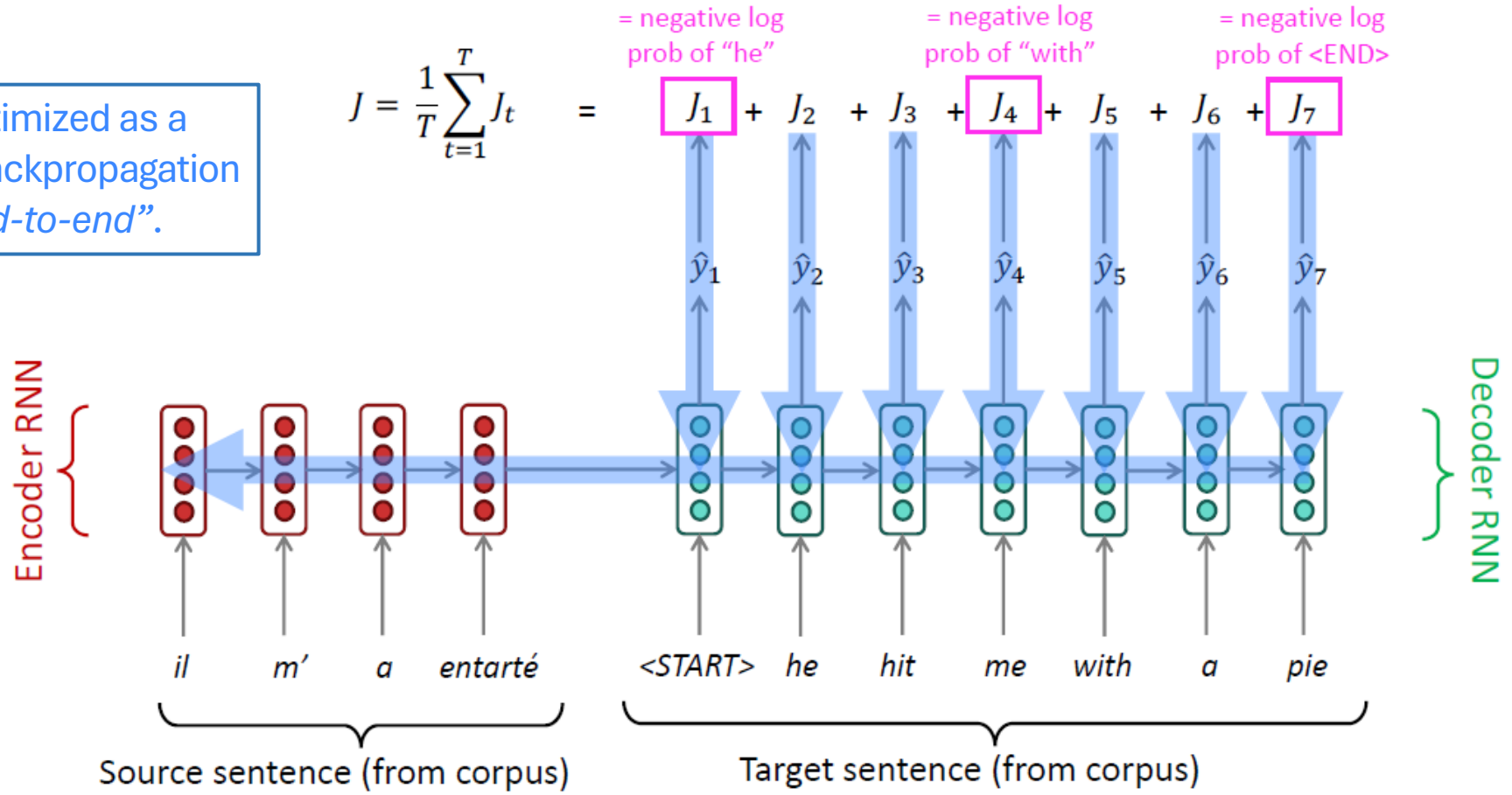
$$P(y|x) = P(y_1|x)\, P(y_2|y_1, x)\, P(y_3|y_1, y_2, x) \ldots \underbrace{P(y_T|y_1, \ldots, y_{T-1}, x)}$$

Probability of next target word, given target words so far and source sentence x

- How to train an NMT system?

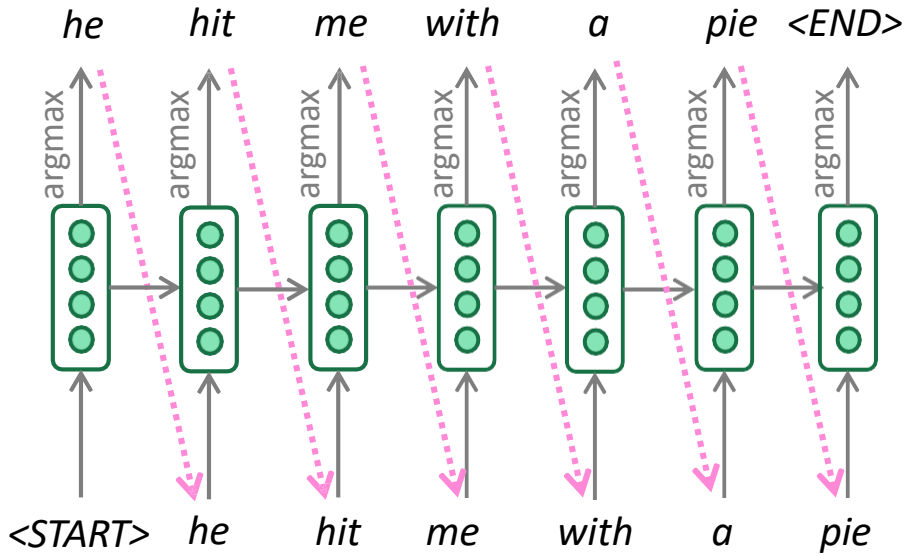
Training an NMT System

Seq2seq is optimized as a **single system.** Backpropagation operates "*end-to-end*".



Greedy decoding

- We saw how to generate (or "decode") the target sentence by taking argmax on each step of the decoder.



- This is greedy decoding (take most probable word on each step)
- **Problems with this method?**

Problems With Greedy Decoding

- Greedy decoding has no way to undo decisions!
- Input: *il a m'entarté* *(he hit me with a pie)*
- → *he* _____
- → *he hit* _____
- → *he hit a* _____ (whoops! no going back now…)

How to fix this?

Exhaustive Search Decoding

- Ideally we want to find a (length *T*) translation *y* that maximizes

$$P(y|x) = P(y_1|x)\,P(y_2|y_1,x)\,P(y_3|y_1,y_2,x)\ldots,P(y_T|y_1,\ldots,y_{T-1},x)$$
$$= \prod_{t=1}^{T} P(y_t|y_1,\ldots,y_{t-1},x)$$

- We could try computing all possible sequences *y*
- This means that on each step *t* of the decoder, we're tracking V^t possible partial translations, where *V* is vocab size
- This $O(V^T)$ complexity is **far too expensive**!

Beam Search Decoding

- **Core idea:** On each step of decoder, keep track of the *k most probable* partial translations (which we call *hypotheses*)
  - *k* is the beam size (in practice around 5 to 10)
- A hypothesis $y_1, \ldots, y_t$ has a score which is its log probability:

$$\text{score}(y_1, \ldots, y_t) = \log P_{\text{LM}}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

  - Scores are all negative, and higher score is better
  - We search for high-scoring hypotheses, tracking top *k* on each step
- Beam search is not guaranteed to find optimal solution
  - But much more efficient than exhaustive search!

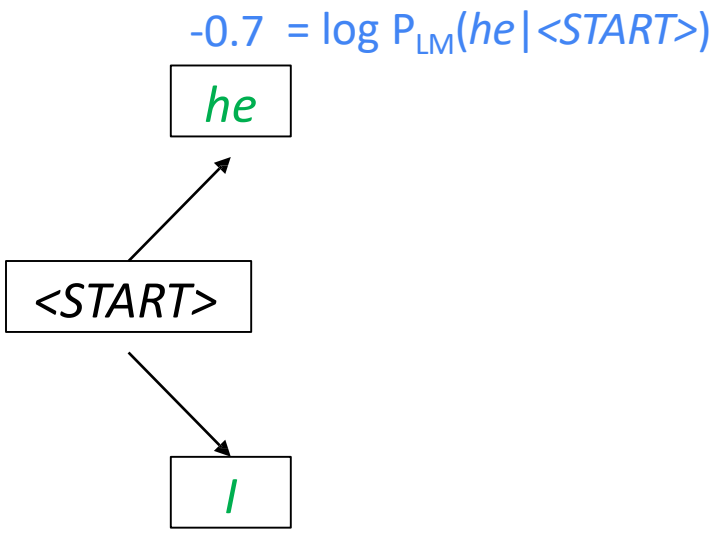
Beam Search Decoding: Example

Beam size = k = 2.

<START>

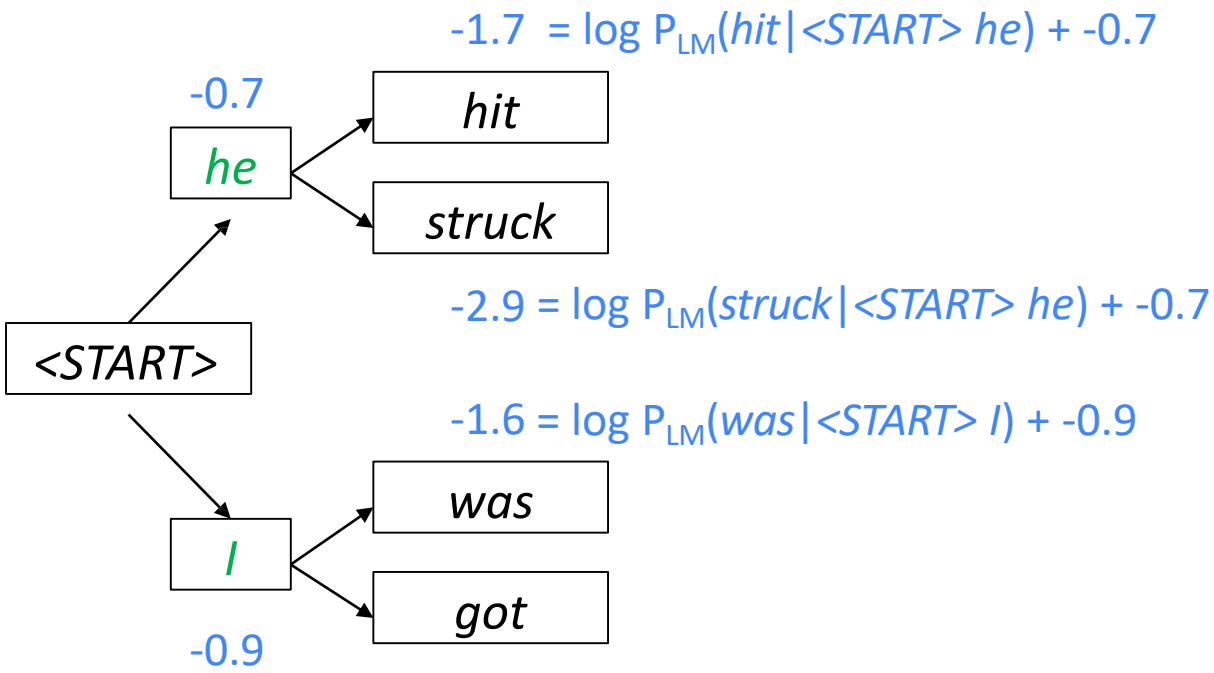
Calculate prob distribution of next word

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



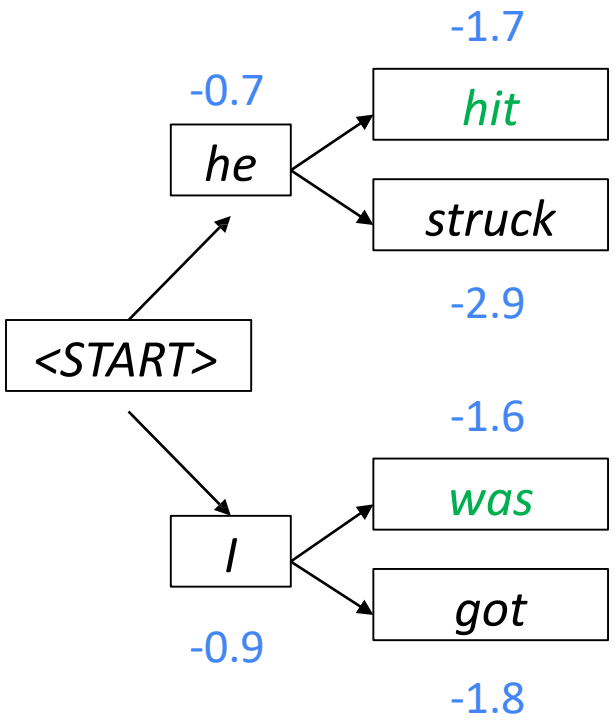
Take top *k* words
and compute scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



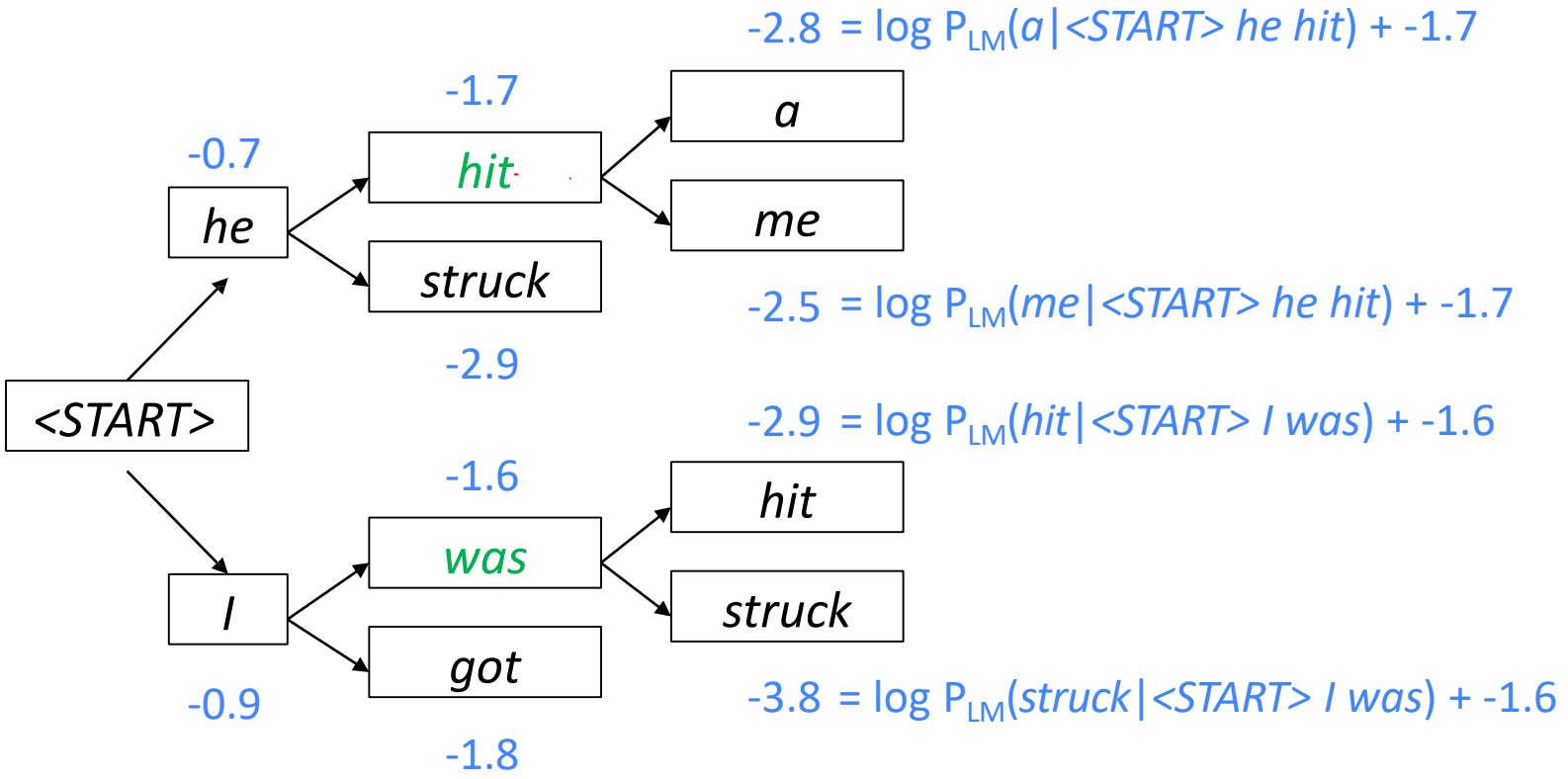
For each of the *k* hypotheses, find top *k* next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



-2.8 = log P_{LM}(*a*|*<START> he hit*) + -1.7

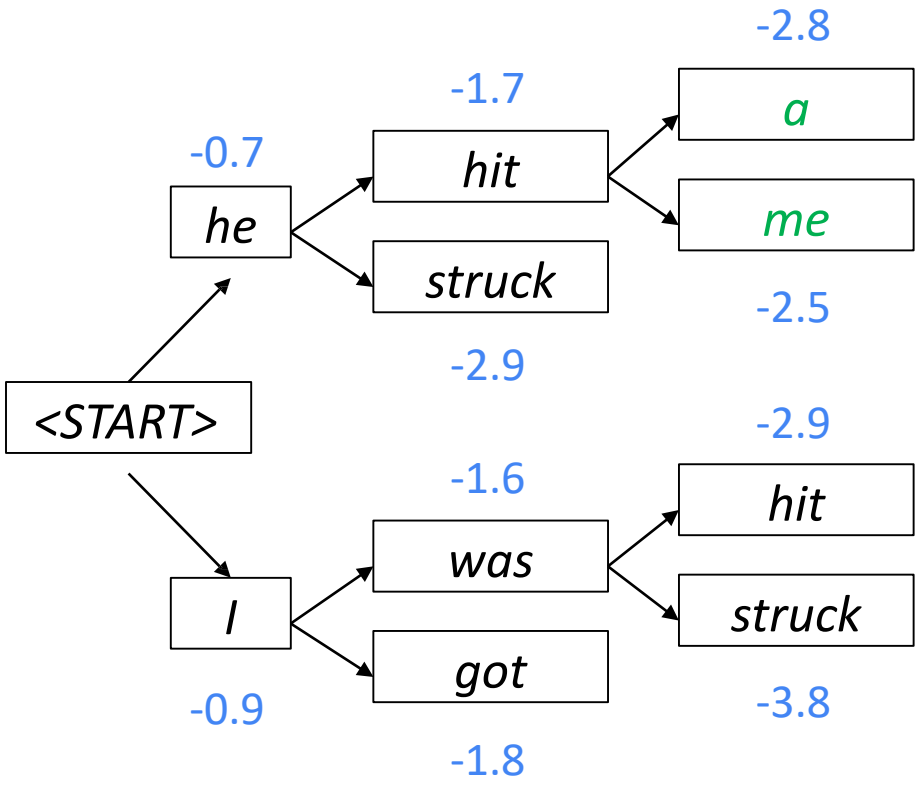
-2.5 = log P_{LM}(*me*|*<START> he hit*) + -1.7

-2.9 = log P_{LM}(*hit*|*<START> I was*) + -1.6

-3.8 = log P_{LM}(*struck*|*<START> I was*) + -1.6

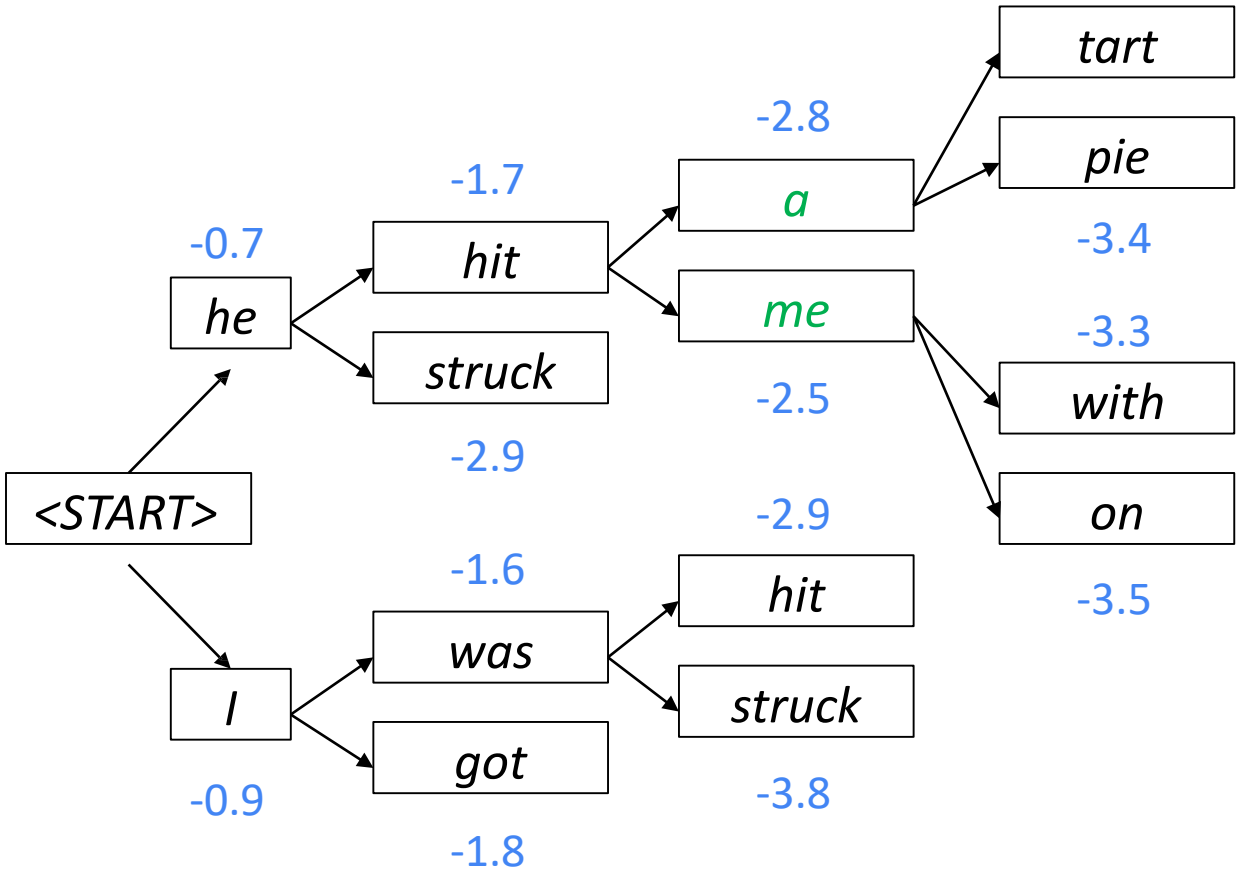
For each of the *k* hypotheses, find top *k* next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



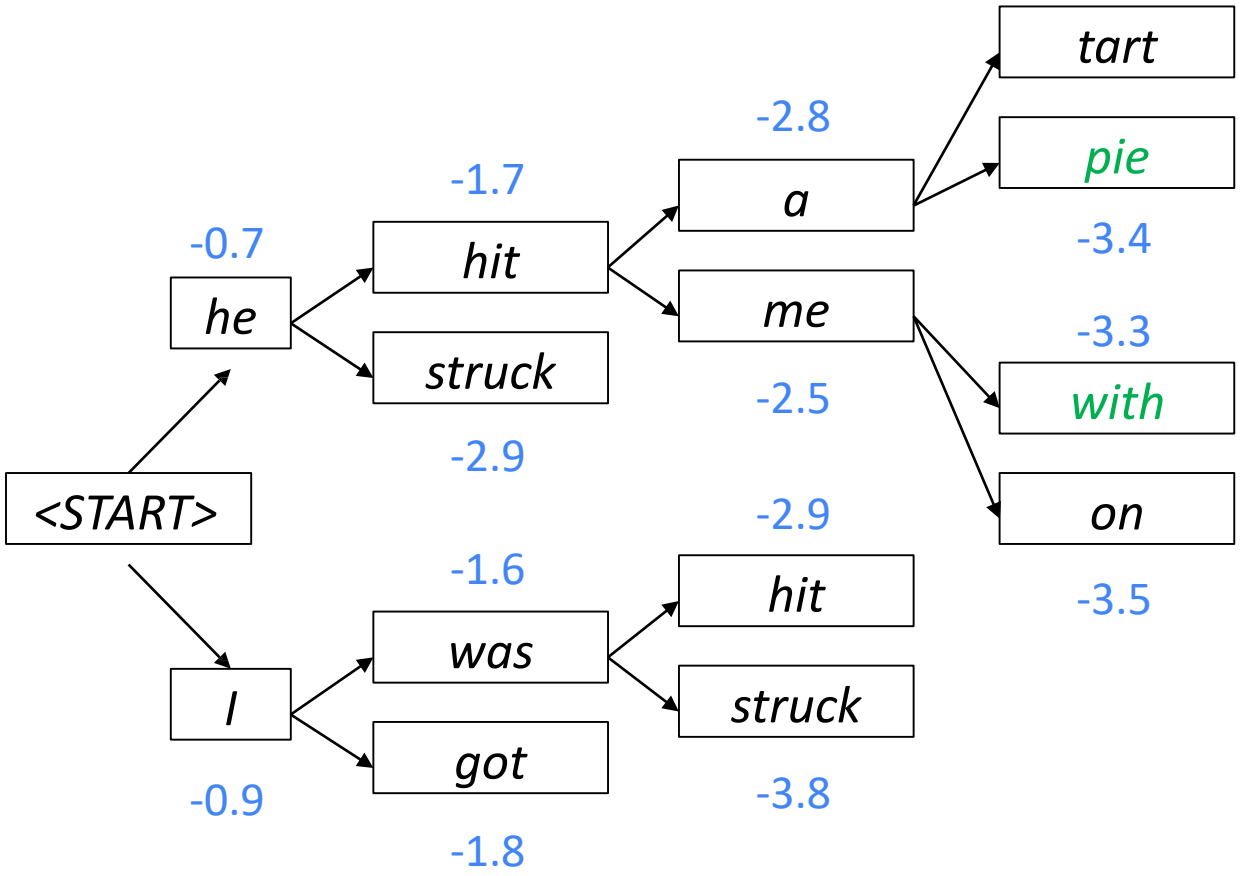
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



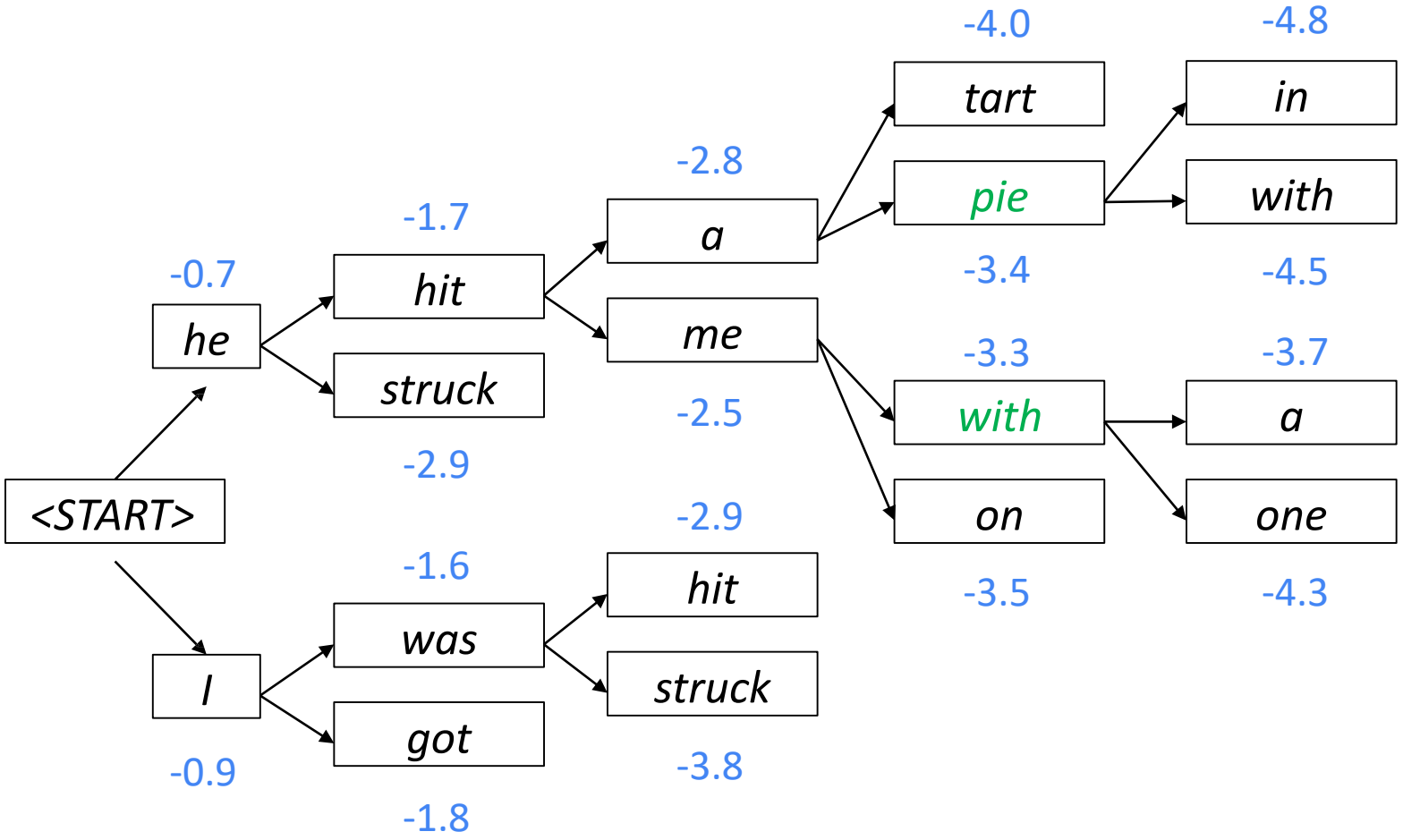
For each of the *k* hypotheses, find top *k* next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



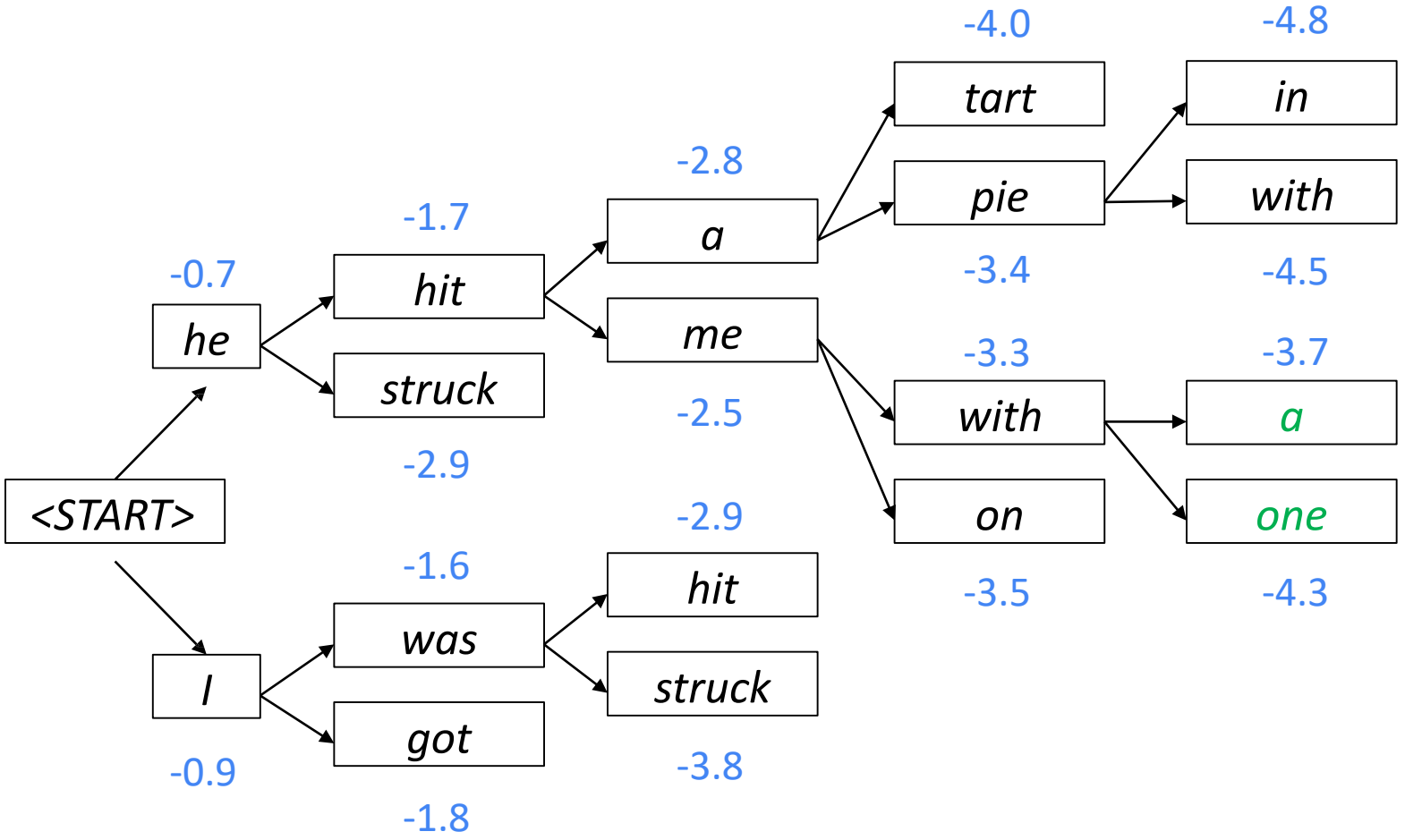
Of these k^2 hypotheses,
just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



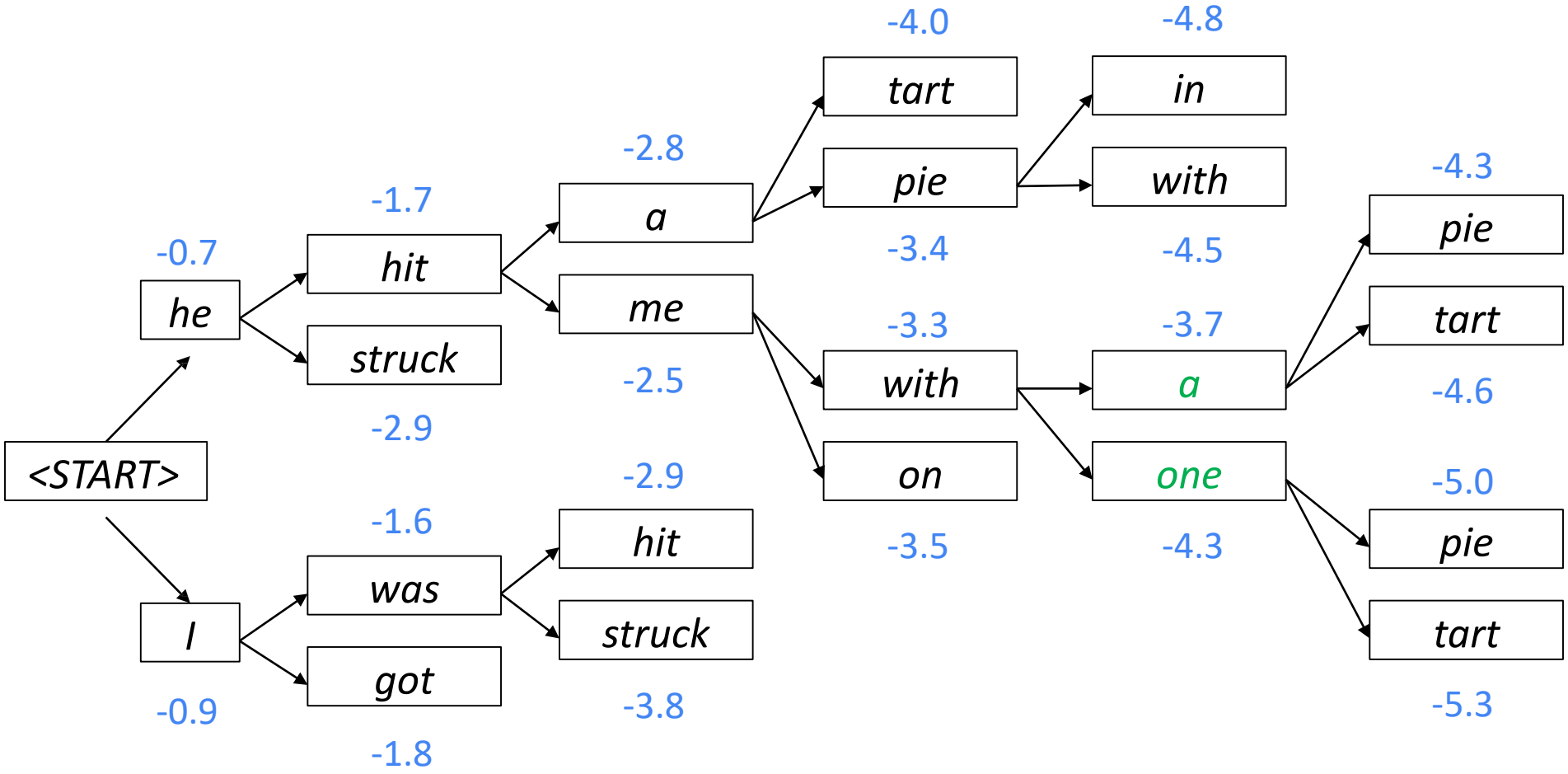
For each of the *k* hypotheses, find top *k* next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



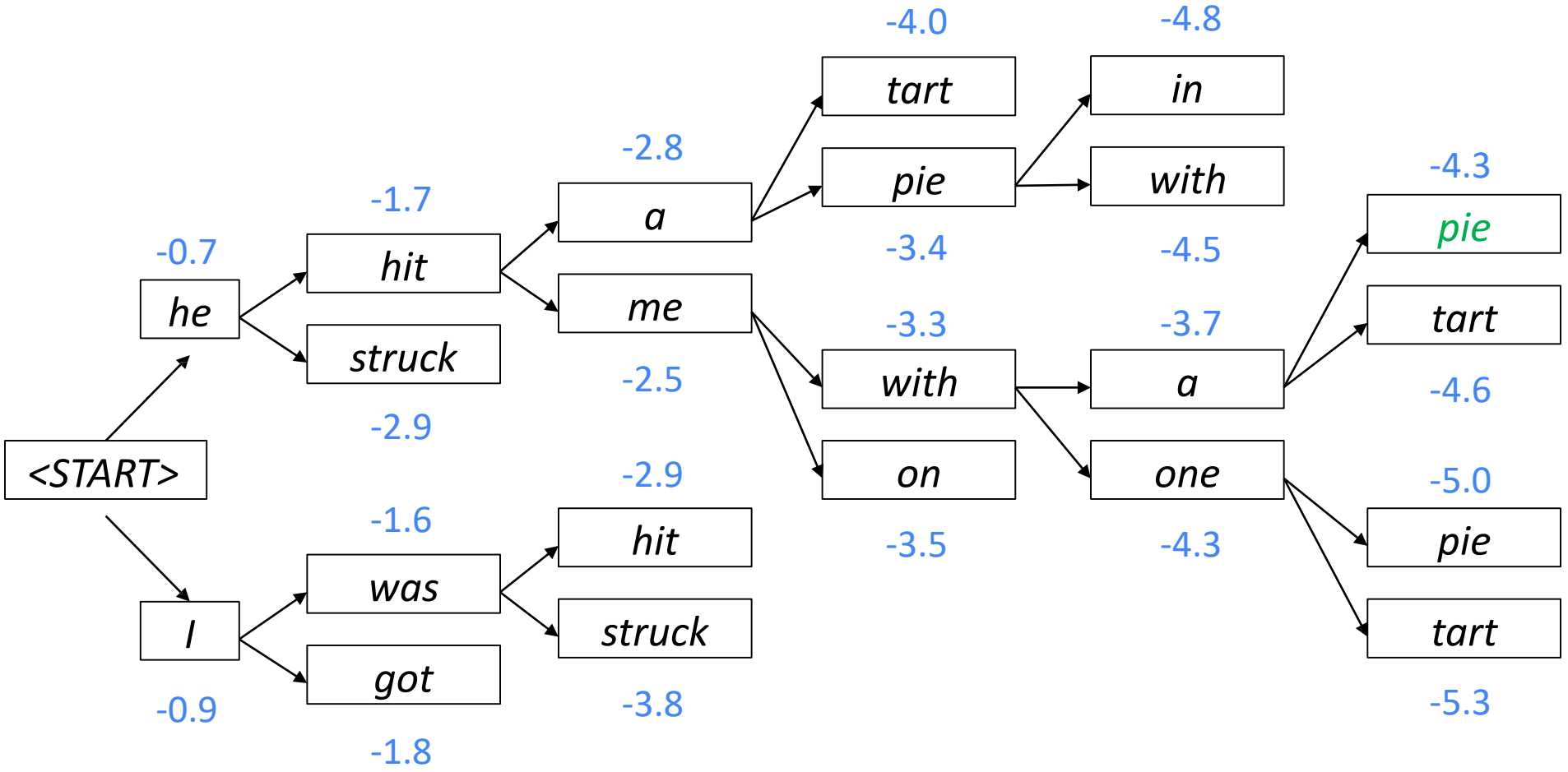
Of these k^2 hypotheses, just keep k with highest scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



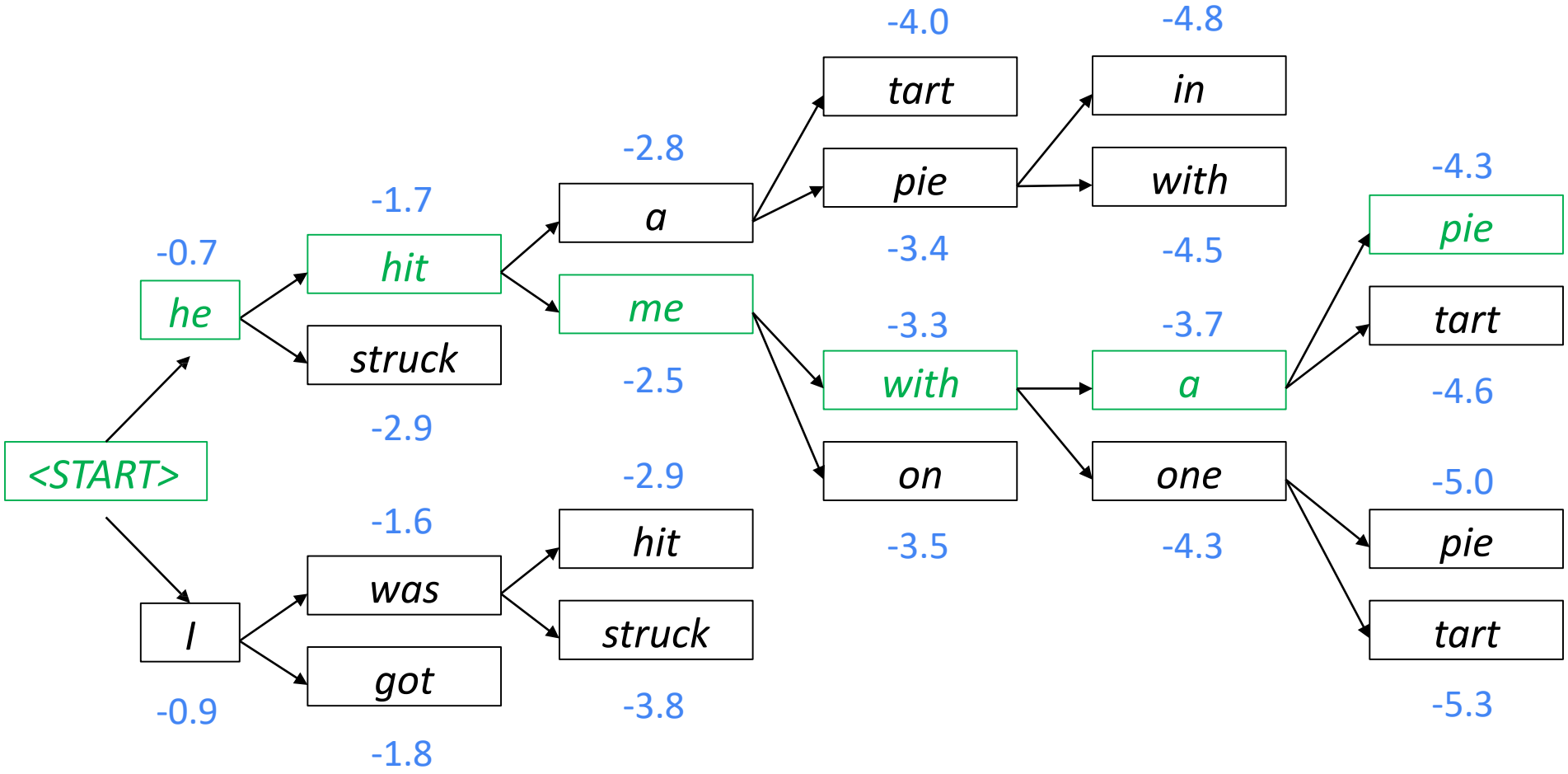
For each of the *k* hypotheses, find top *k* next words and calculate scores

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



This is the top-scoring hypothesis!

Beam size = k = 2. Blue numbers = $\text{score}(y_1, \ldots, y_t) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$



Backtrack to obtain the full hypothesis

Beam Search Decoding: Stopping Criterion

- In greedy decoding, usually we decode until the model produces a <END> token
  - **For example:** <START> he hit me with a pie <END>
- In beam search decoding, different hypotheses may produce <END> tokens on different timesteps
  - When a hypothesis produces <END>, that hypothesis is complete.
  - Place it aside and continue exploring other hypotheses via beam search.
- Usually we continue beam search until:
  - We reach timestep *T* (where *T* is some pre-defined cutoff), or
  - We have at least *n* completed hypotheses (where *n* is pre-defined cutoff)

Beam Search Decoding: Finishing Up

- We have our list of completed hypotheses.
- How to select top one with highest score?
- Each hypothesis $y_1, \ldots, y_t$ on our list has a score

$$\text{score}(y_1, \ldots, y_t) = \log P_{\text{LM}}(y_1, \ldots, y_t | x) = \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

- **Problem:** longer hypotheses have lower scores
- **Fix:** Normalize by length. Use this to select the top one instead:

$$\frac{1}{t} \sum_{i=1}^{t} \log P_{\text{LM}}(y_i | y_1, \ldots, y_{i-1}, x)$$

Decoding Strategies

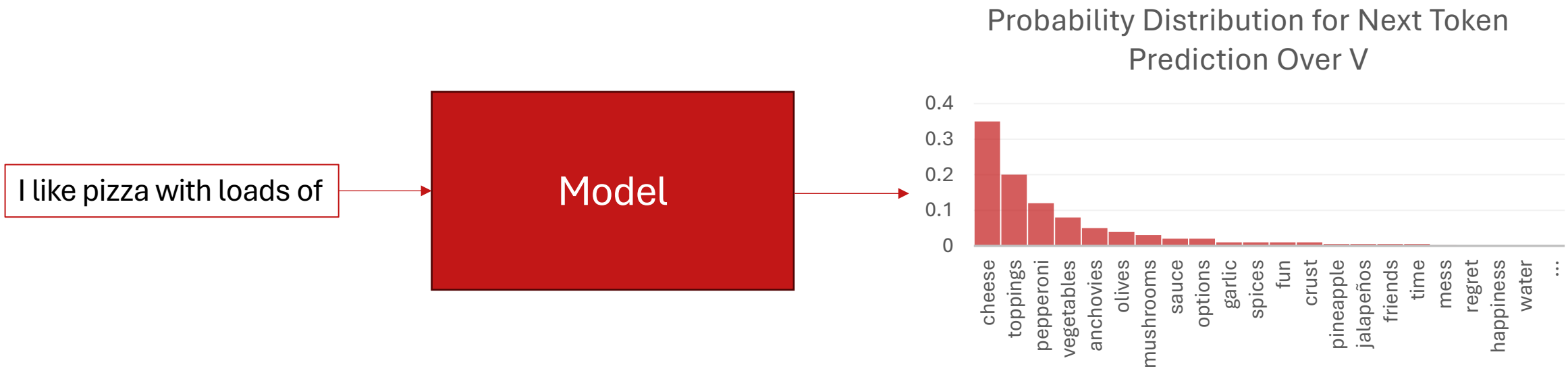
Tanmoy Chakraborty
Associate Professor, IIT Delhi
https://tanmoychak.com/

Introduction to Large Language Models

What is Decoding?

- Recall: In Seq2Seq models (like, NMT systems built with RNNs), we get a probability distribution over the tokens in vocabulary at each timestep.
  - We have $P(w_t | c, w_{<t})$ for $\forall\ w_t \in V$, where c is the input context/sentence/prompt, $w_{<t}$ are the tokens generated till now, and w_t is the token to be generated at the timestep t. V is the vocabulary.
- Decoding is the process of choosing the (next) token to generate based on the probability distribution over the vocabulary which the (language) model outputs.
- While generating the next token based on the output probability distribution, trade-off between two factors need to be handled by the decoding algorithms:
  1. Quality (Generated responses should be coherent and accurate)
  2. Diversity (Generated responses should be 'creative' and diverse, instead of being repetitive)

Recap



Assume, we want to generate N subsequent tokens given the input context c (c is '*I like pizza with loads of*' in the above example).
In the previous lecture, we discussed:

1. Greedy Decoding (generate the token with the highest probability at each step)
2. Exhaustive Search Decoding (calculate the probabilities for all $|V|^N$ possible sequences; finally generate the most probable one)
3. Beam Search Decoding (retain only k most probable sequences at each step)

Deterministic vs. Stochastic Decoding Strategies

- The decoding methods which we discussed in the previous lecture – greedy decoding, exhaustive search and beam search – are all deterministic, i.e., *given the same model and the same input context, they always generate the same output sequence*.

- However, to ensure diversity of the generated responses we need stochastic decoding strategies.
  - The stochastic decoding strategies generate diverse responses even for the same model and same input context.
- In this lecture, we will look into three stochastic decoding strategies:
  1. Top-k Sampling
  2. Top-p / Nucleus Sampling
  3. Temperature Sampling

Random Sampling

- Before moving onto those three stochastic decoding strategies, let's first look at simple random sampling.
- If we want to generate a sequence of N words w_1, ..., w_N , then *random sampling* can be written formally as the following algorithm:

$i \leftarrow 1$

$w_i \sim P(w_i \mid c)$

while w_i != EOS or i<=N:

$i \leftarrow i + 1$

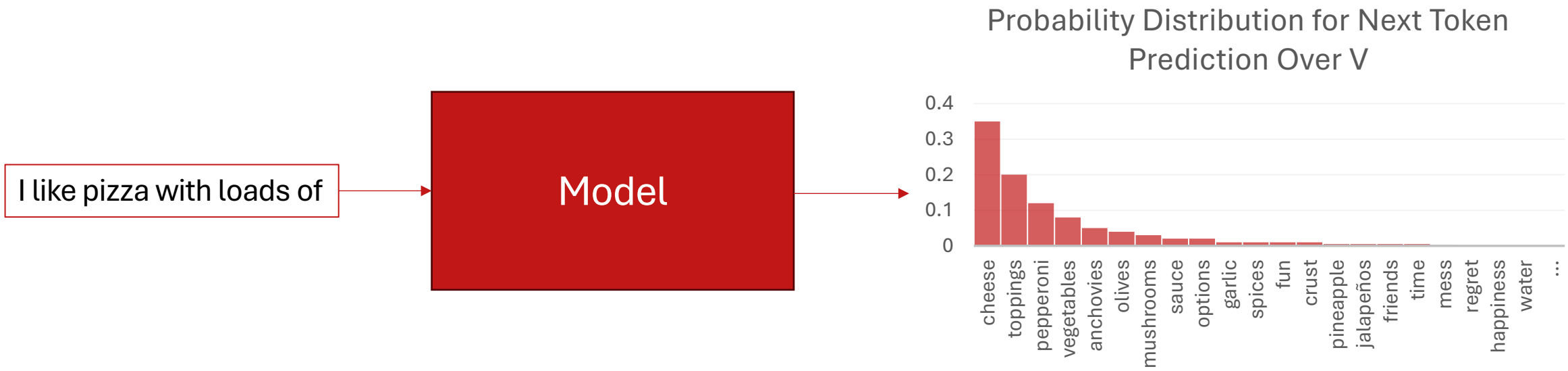
$w_i \sim P(w_i \mid c, w_{<i})$

Intuitively, if say P(*cheese* | c='I like pizza with loads of') = 0.35, then on giving c as input 100 times, on random sampling, 'cheese' is expected to be generated approximately 35 times as the next token, though the exact count may vary due to randomness.

Top-k Sampling

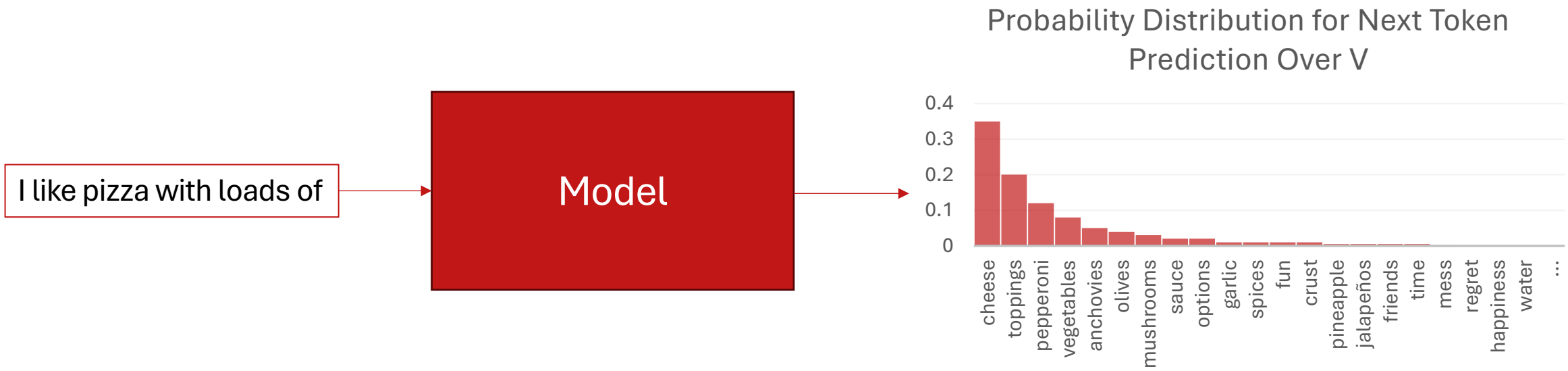
- In greedy decoding, the token with the highest probability was chosen at each step, making it deterministic. Top-k sampling is a stochastic generalization of greedy decoding.
- Top-k sampling involves the following steps:
  1. Choose in advance a number of tokens ***k***
  2. Given the probability $P(w_t | c, w_{<t})$ for all tokens in the vocabulary V, as generated by the model, sort the tokens by their likelihood, and throw away any token that is not one of the top ***k*** most probable tokens.
  3. Renormalize the scores of the ***k*** tokens to be a legitimate probability distribution.
  4. Randomly sample a token from within these remaining ***k*** most-probable tokens according to its probability.
- When k = 1, top-k sampling is identical to greedy decoding.

Top-k Sampling: Working Example



- Let's take **k=3**.
- Step-1: Sort the tokens by their likelihood, and throw away any token that is not one of the top ***k*** most probable tokens.
  - In our example, top-3 tokens with P(w|c) are: **cheese (0.35), toppings (0.20), pepperoni (0.12)**

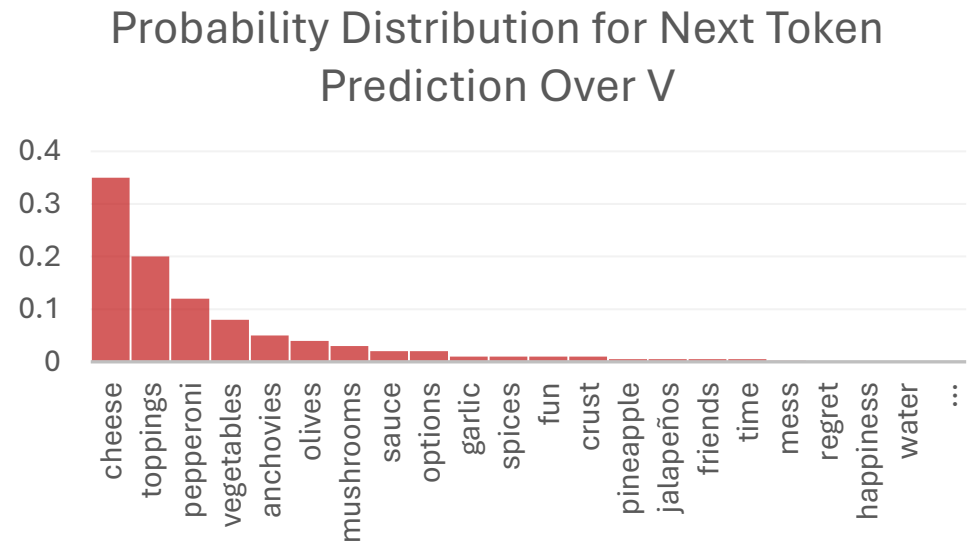
Top-k Sampling: Working Example

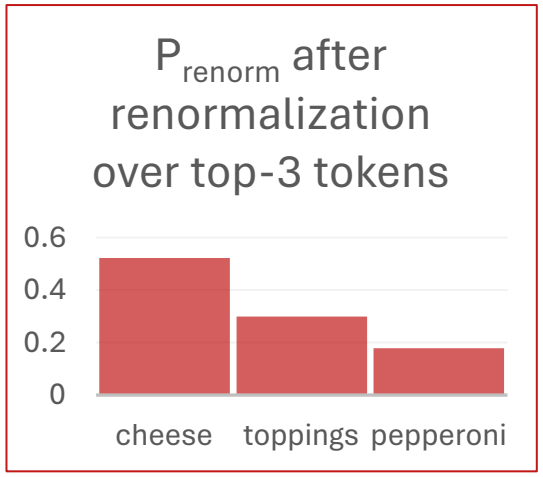


- Step-2: Renormalize the scores of the ***k*** tokens to be a legitimate probability distribution.
- After renormalization, $P_{renorm}(w|c) = \frac{P(w|c)}{\sum_{x \in top-k} P(x|c)}$.
  - Renormalized probabilities: **cheese (0.522), toppings (0.299), pepperoni (0.179)**

Top-k Sampling: Working Example

I like pizza with loads of → Model

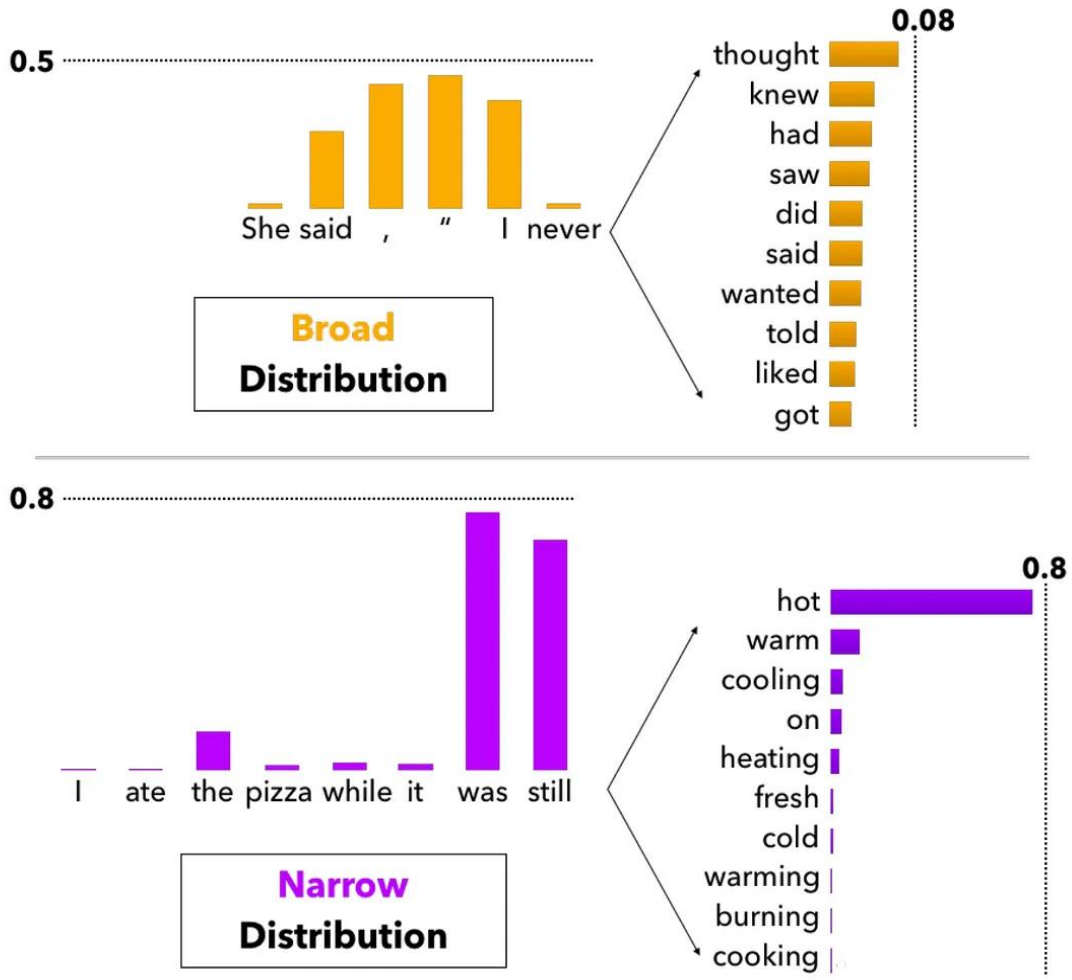




- Step-3: Randomly sample a token from within these remaining ***k*** most-probable tokens according to its probability after renorm.
- For generating the next token, $w_t \sim P_{renorm}(w_t | c)$

Problem With Top-k Sampling

- In top-k sampling ***k*** is fixed, but the shape of the probability distribution over tokens differs in different contexts.
- For example, if we set k = 10:
  - Sometimes the top 10 tokens will be very likely and include most of the probability mass.
  - But other times the probability distribution will be flatter and the top 10 tokens will only include a small part of the probability mass.



Solution: Top-p / Nucleus Sampling

Image source: https://towardsdatascience.com/how-to-sample-from-language-models-682bceb97277

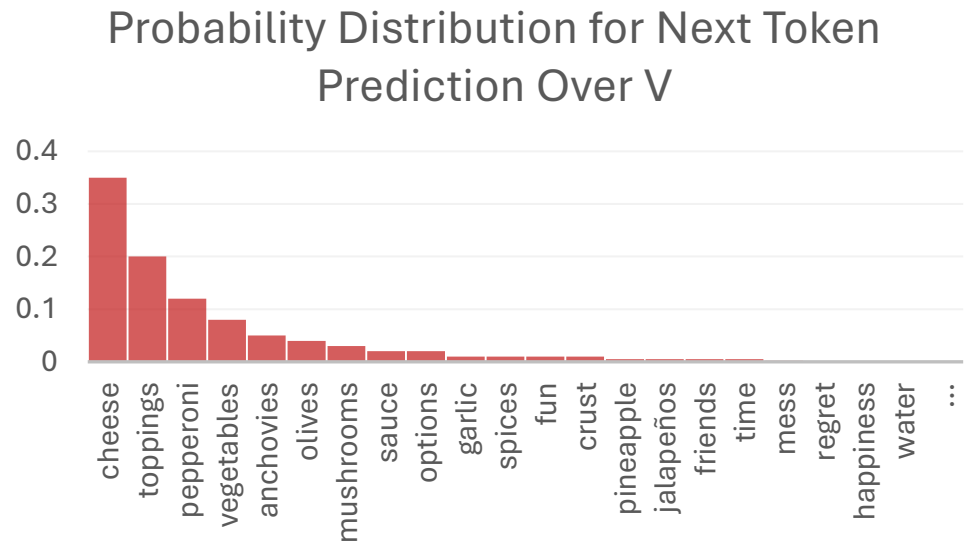
Top-p / Nucleus Sampling

- The goal of nucleus sampling is the same as top-k sampling, which is to truncate the distribution to remove the very unlikely tokens.
- Here, we keep the top ***p*** percent of the probability mass.
  - By measuring probability rather than the number of tokens, the hope is that the measure will be more robust in very different contexts, dynamically increasing and decreasing the pool of token candidates.
- Formally, given a distribution $P(w_t \mid c, w_{<t})$, the top-p vocabulary $\mathbf{V^{(p)}}$ is the smallest set of tokens such that:

$$\sum_{w \in V^{(p)}} P(w|c, w_{<t}) \geq p$$

Top-p Sampling: Working Example

I like pizza with loads of → Model →



Token (w)	P(w|c)	Cumulative P
cheese	0.35	0.35
toppings	0.20	0.55
pepperoni	0.12	0.67
vegetables	0.08	0.75
anchovies	0.05	0.80

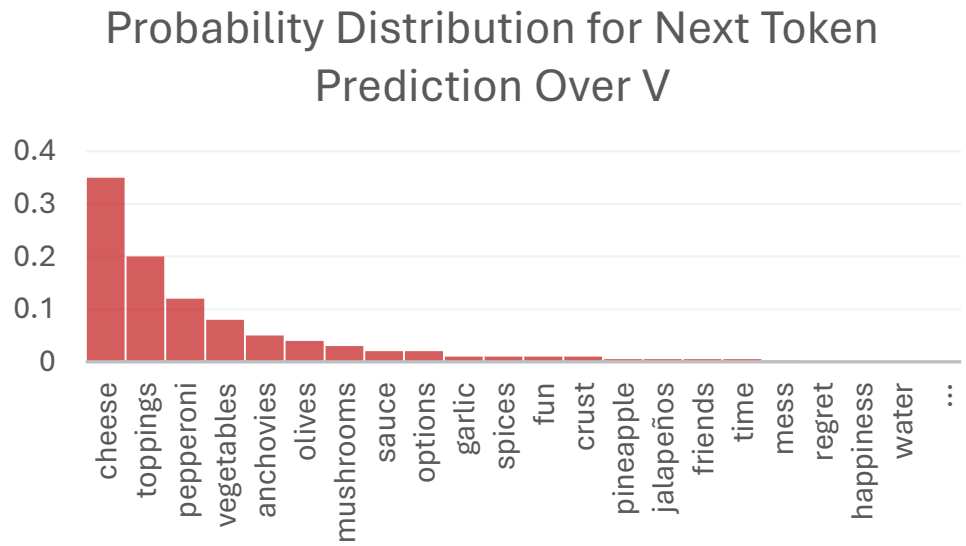
- Let's take **p=0.5**
- Step-1: Get the top-p vocabulary $\mathbf{V^{(p)}}$, which is the smallest set of tokens such that: $\sum_{w \in V(p)} P(w|c, w_{<t}) \geq p$

In our example, $\mathbf{V^{(p)}}$ consists of:

cheese (0.35), toppings (0.20)

Top-p Sampling: Working Example

I like pizza with loads of → Model →

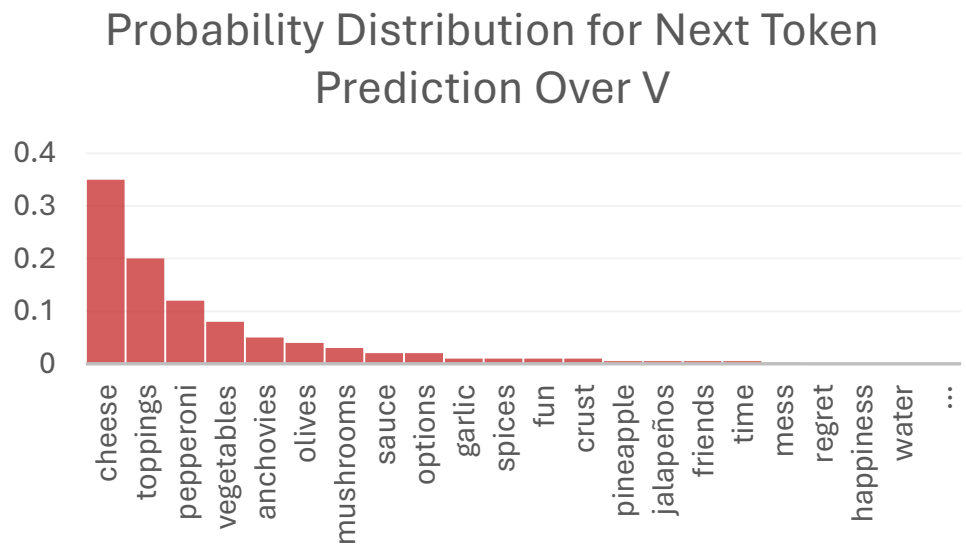


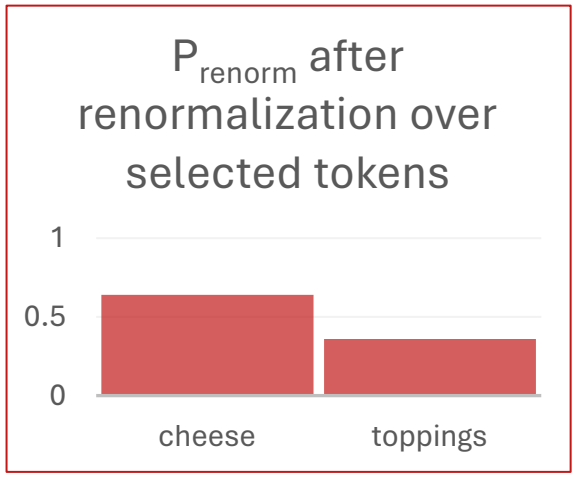
- Step-2: Renormalize the scores of the selected tokens (same procedure as top-k).

Renormalized probabilities: **cheese** $\left(\frac{0.35}{0.35+0.20} = 0.64\right)$, **toppings** $\left(\frac{0.20}{0.35+0.20} = 0.36\right)$

Top-p Sampling: Working Example

I like pizza with loads of → Model → Probability Distribution for Next Token Prediction Over V





- Step-3: Randomly sample a token from within these selected tokens according to its probability after renormalization.
- For generating the next token, $w_t \sim P_{renorm}(w_t | c)$

Temperature Sampling

- In temperature sampling, we don't truncate the distribution but instead reshape it.
- The **temperature parameter (τ)** is incorporated while computing ***softmax*** over the logits of the tokens (for next token prediction).

Normal Softmax

$$P(t_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

- $P(t_i)$: Probability assigned to token t_i
- z_i : Logit for token t_i

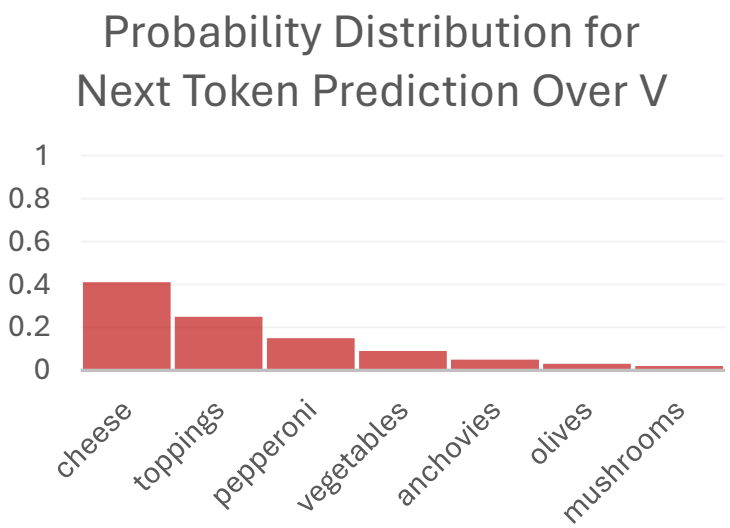
Softmax with Temperature

$$P(t_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

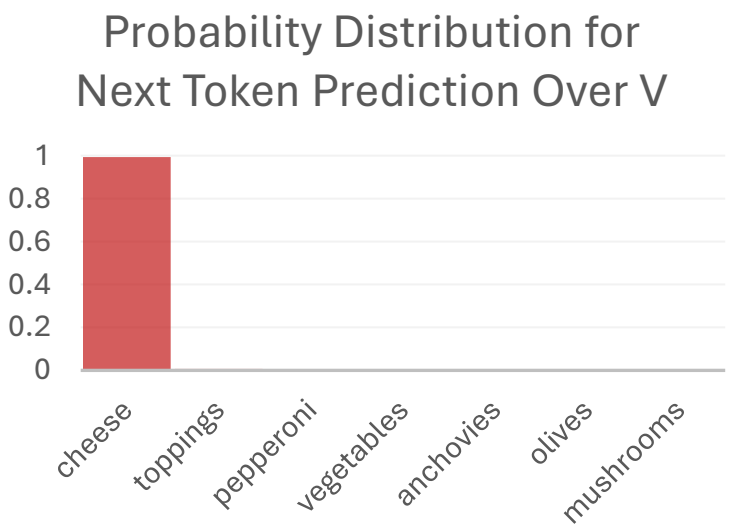
- τ : Temperature parameter
  - $\tau > 0$

Effect of Temperature

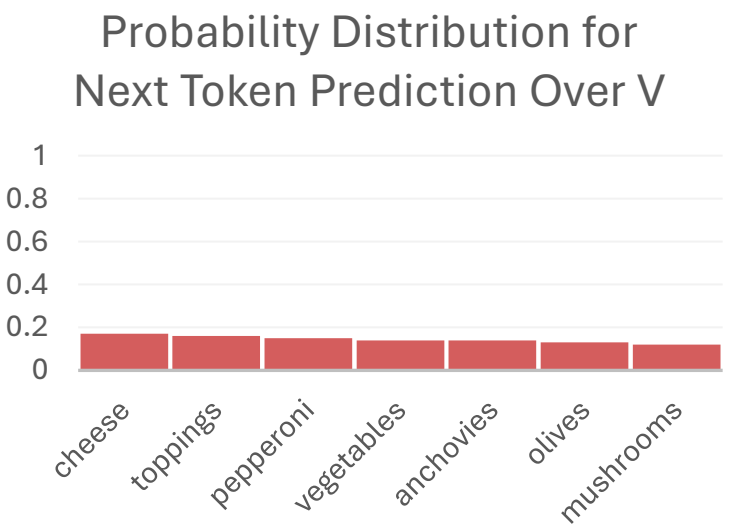
I like pizza with loads of



τ =1

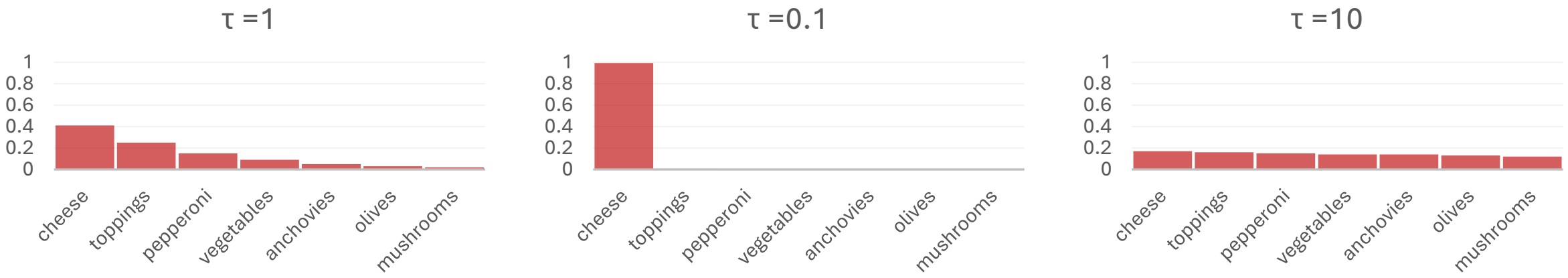


τ =0.1



τ =10

Effect of Temperature



- A low temperature sharpens the probabilities, making the model confident and more deterministic.
  - As τ approaches 0 the probability of the most likely word approaches 1.
  - Lower temperatures make the model more “confident” which can be useful in applications like question answering.
- A high temperature flattens the probabilities, encouraging diversity and creativity in outputs but with a risk of incoherence.
  - Thus, higher temperatures make the model more “creative” which can be useful when generating prose, for example.

Attention in Seq2Seq Models

Tanmoy Chakraborty
Associate Professor, IIT Delhi
https://tanmoychak.com/

Introduction to Large Language Models

NMT: The First Big Success Story of NLP Deep Learning

Neural Machine Translation went from a fringe research attempt in 2014 to the leading standard method in 2016

- 2014: First seq2seq paper published [Sutskever et al. 2014]
- 2016: Google Translate switches from SMT to NMT – and by 2018 everyone had
  - https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html

- This was amazing!
  - SMT systems, built by hundreds of engineers over many years, were outperformed by NMT systems trained by small groups of engineers in a few months

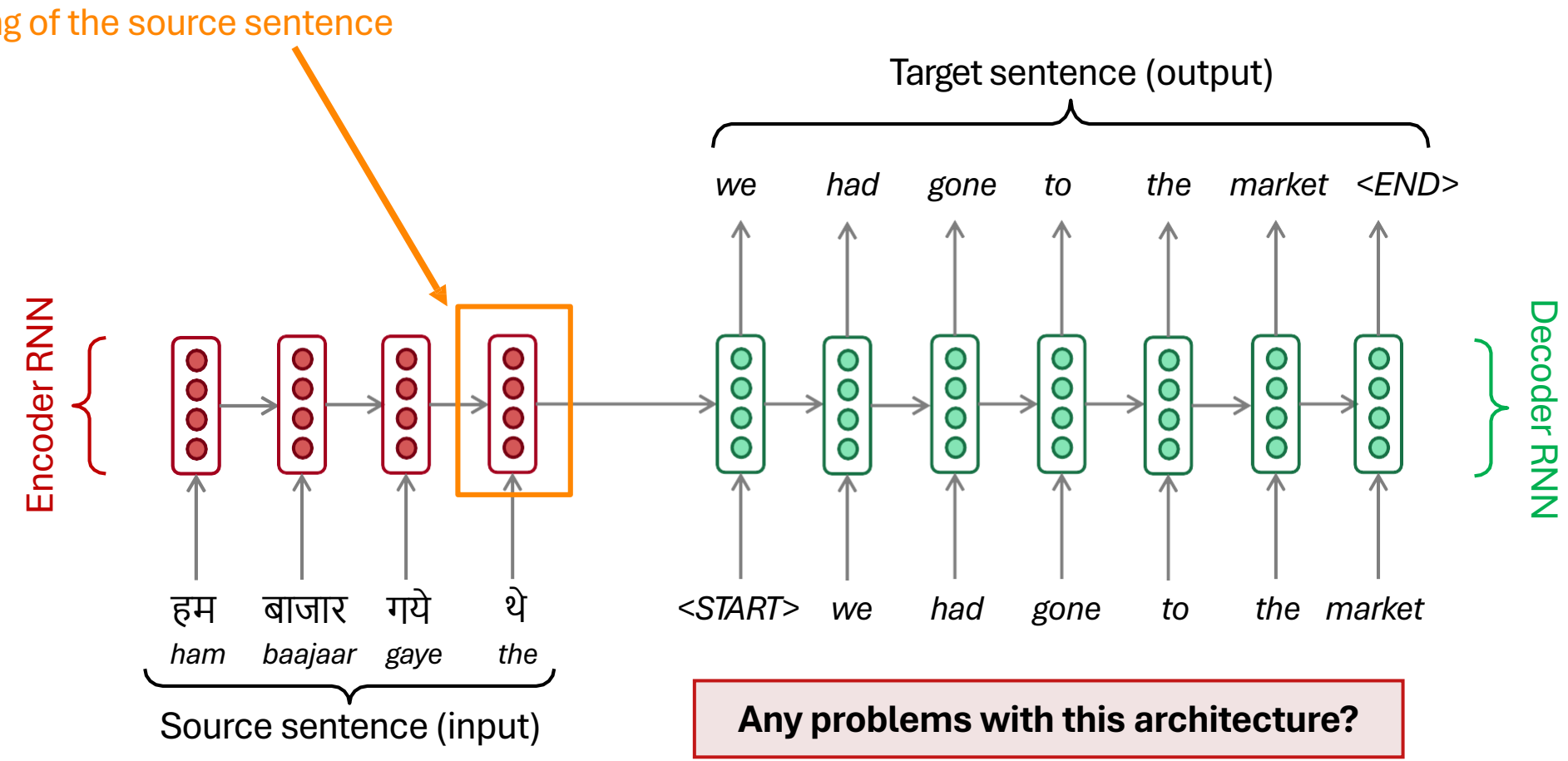
Issues With RNN

- Linear interaction distance
- Bottleneck problem
- Lack of parallelizability

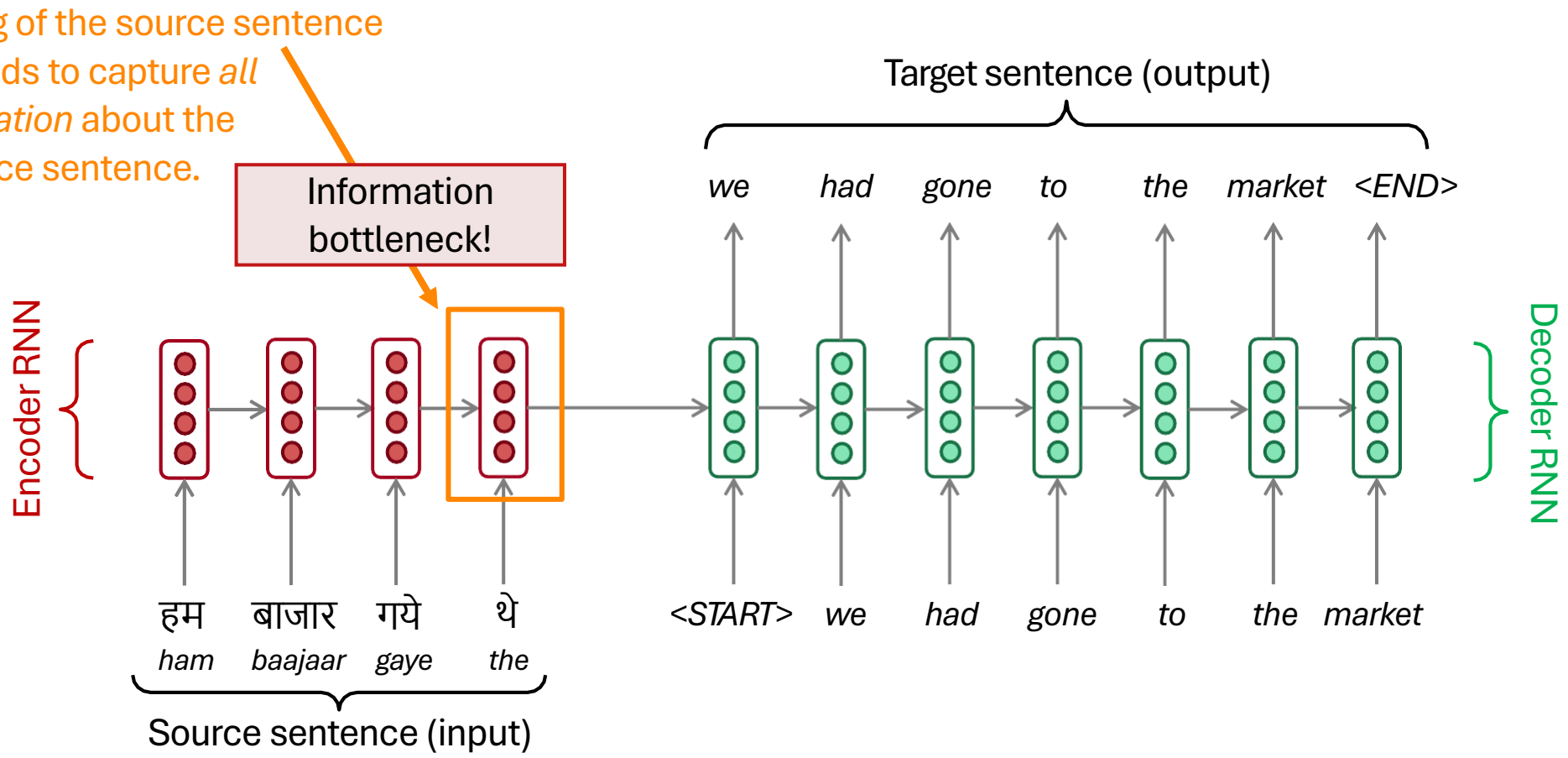
ATTENTION

Attention

Sequence-to-Sequence: The Bottleneck Problem



Sequence-to-Sequence: The Bottleneck Problem



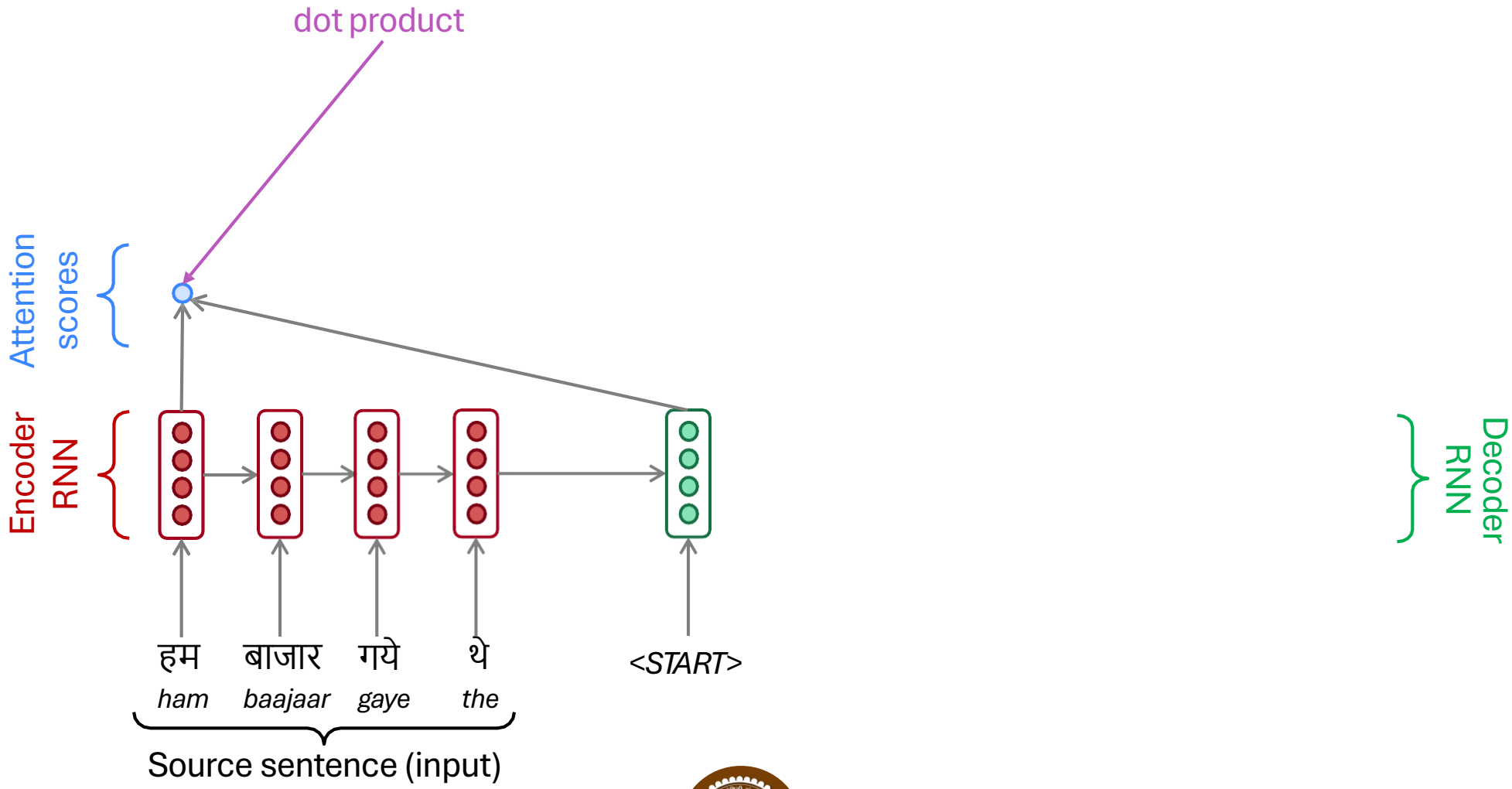
Attention

- **Attention** provides a solution to the bottleneck problem.

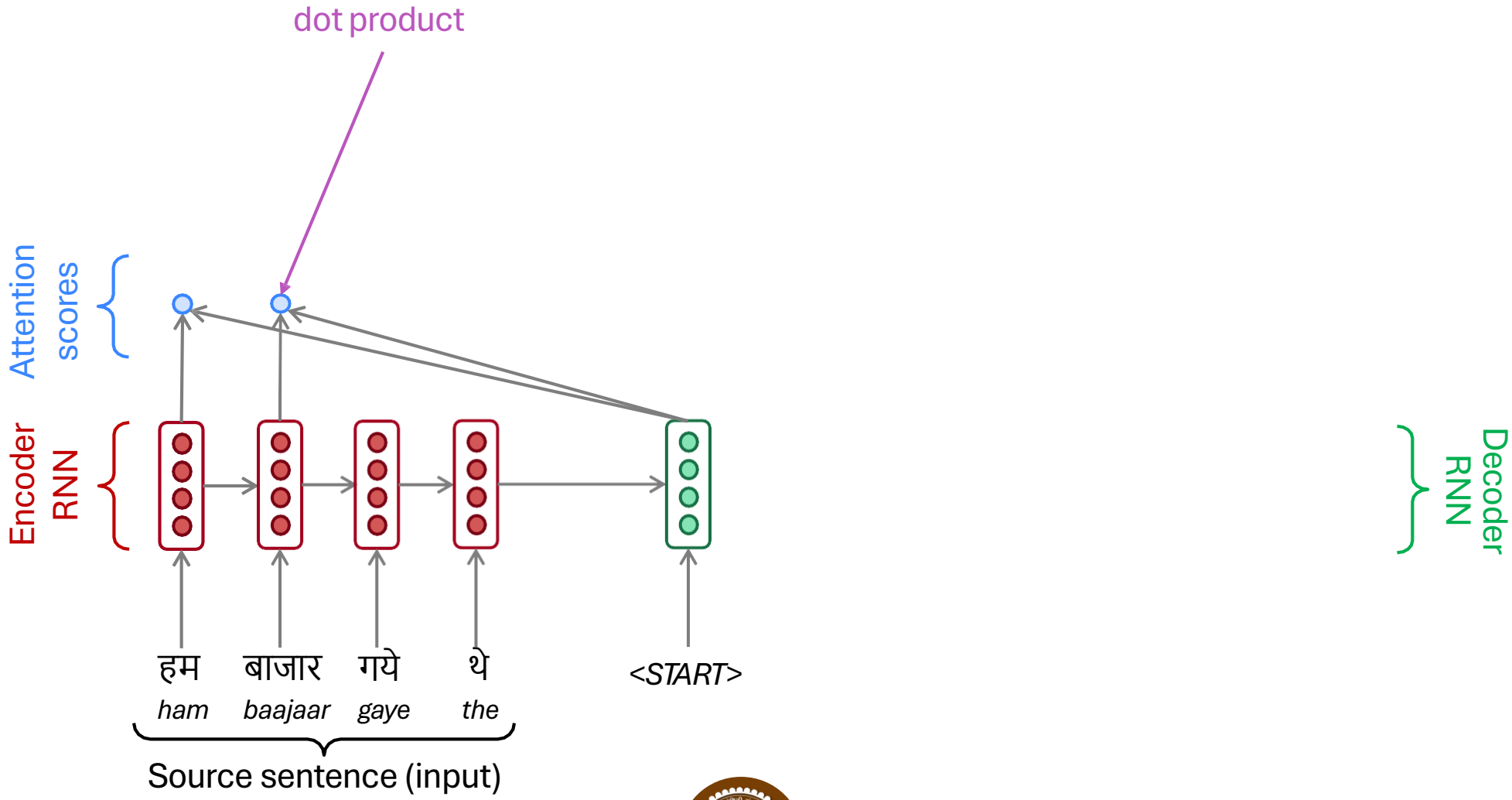
- **Core idea**: on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence

- Let's start with the visualization of the attention mechanism.

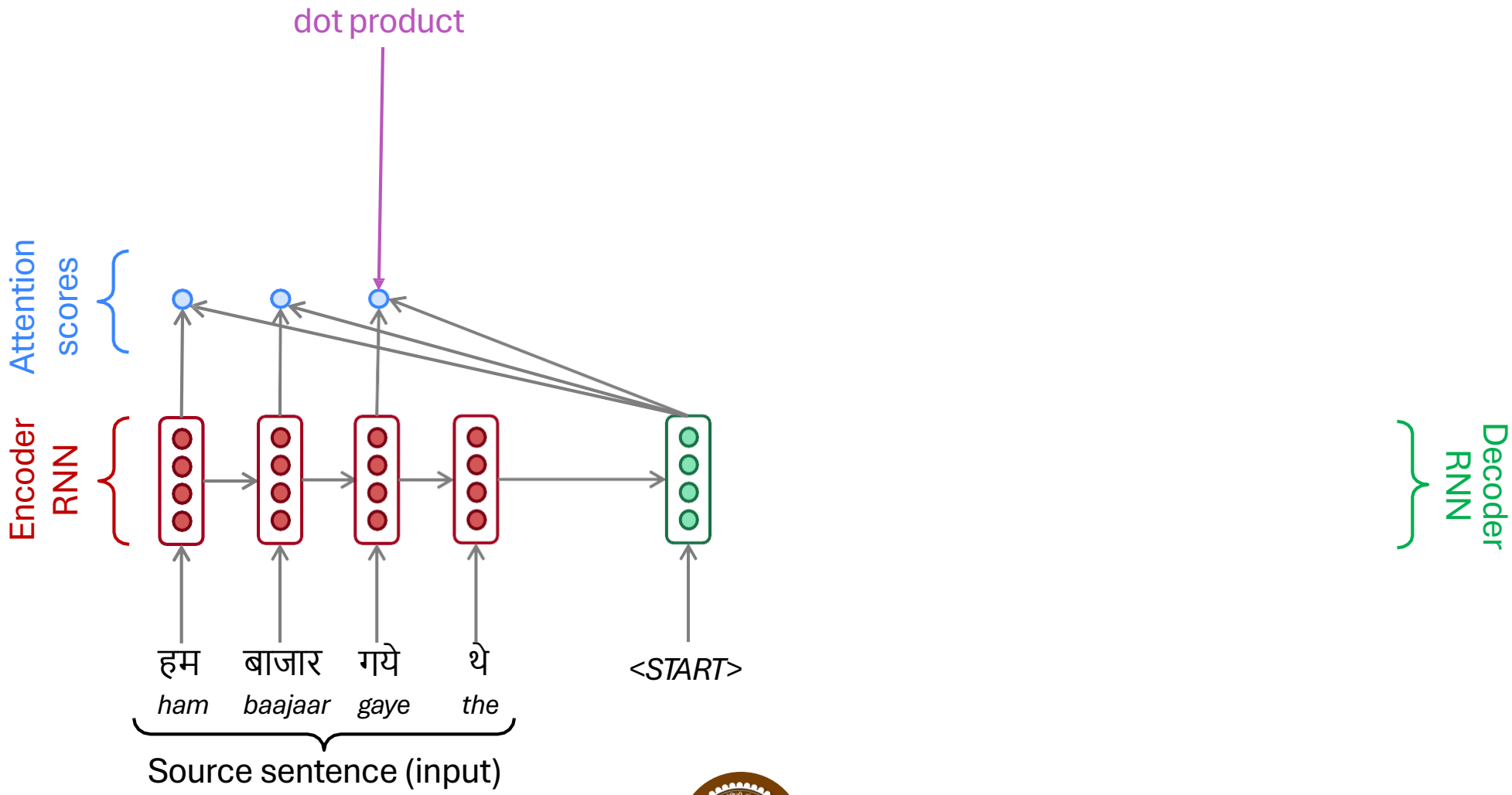
Sequence-to-Sequence With Attention



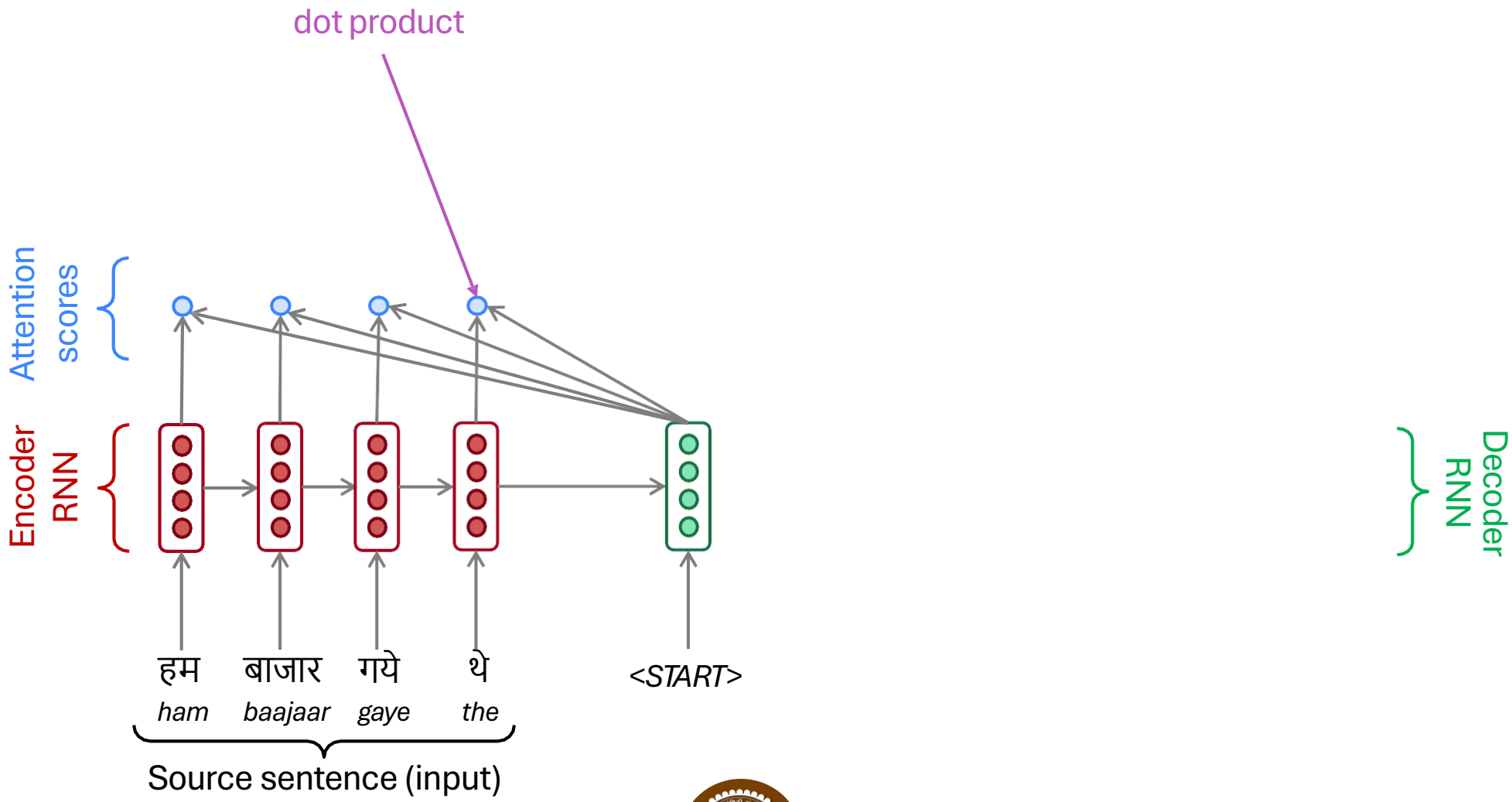
Sequence-to-Sequence With Attention



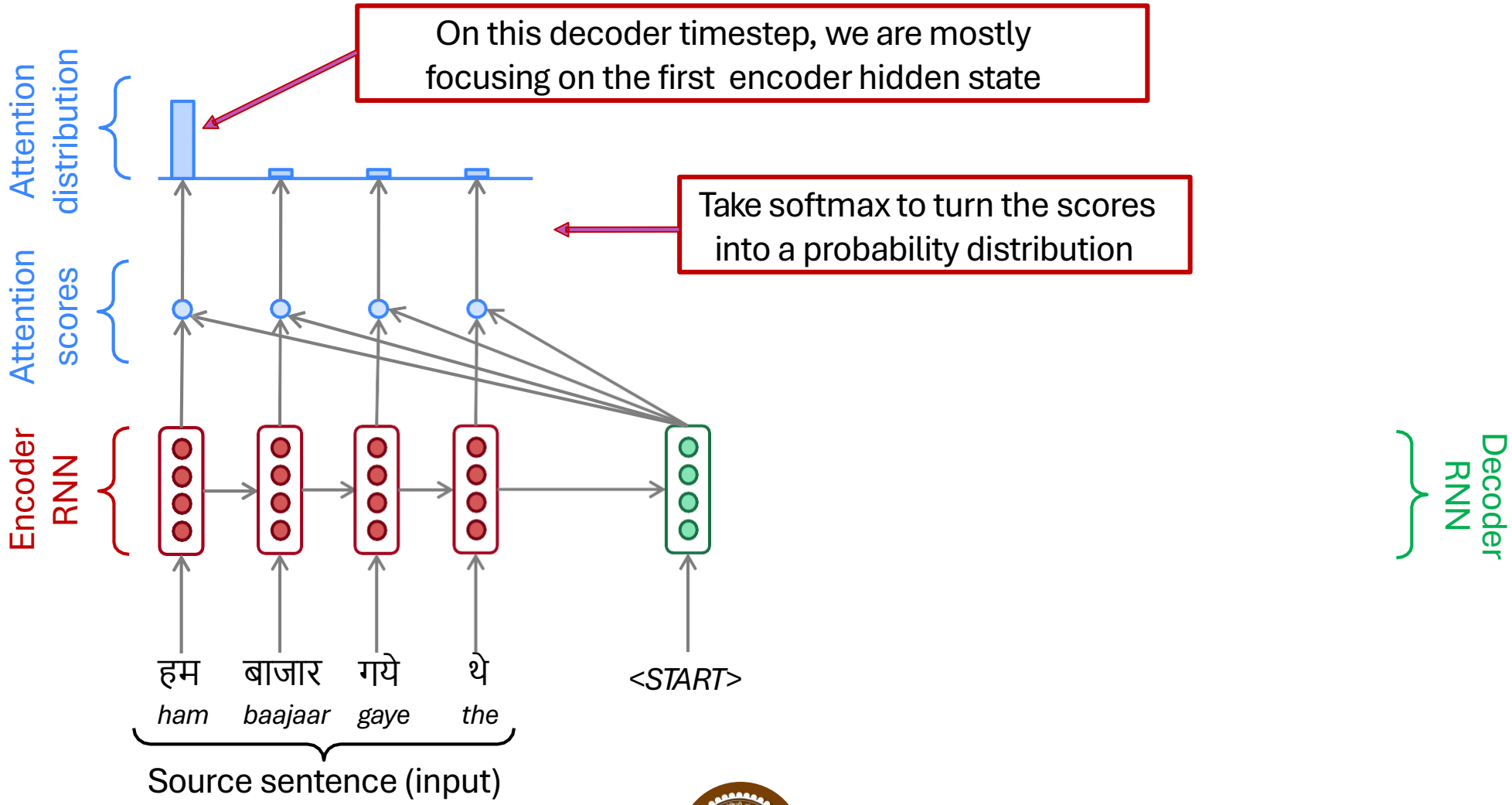
Sequence-to-Sequence With Attention



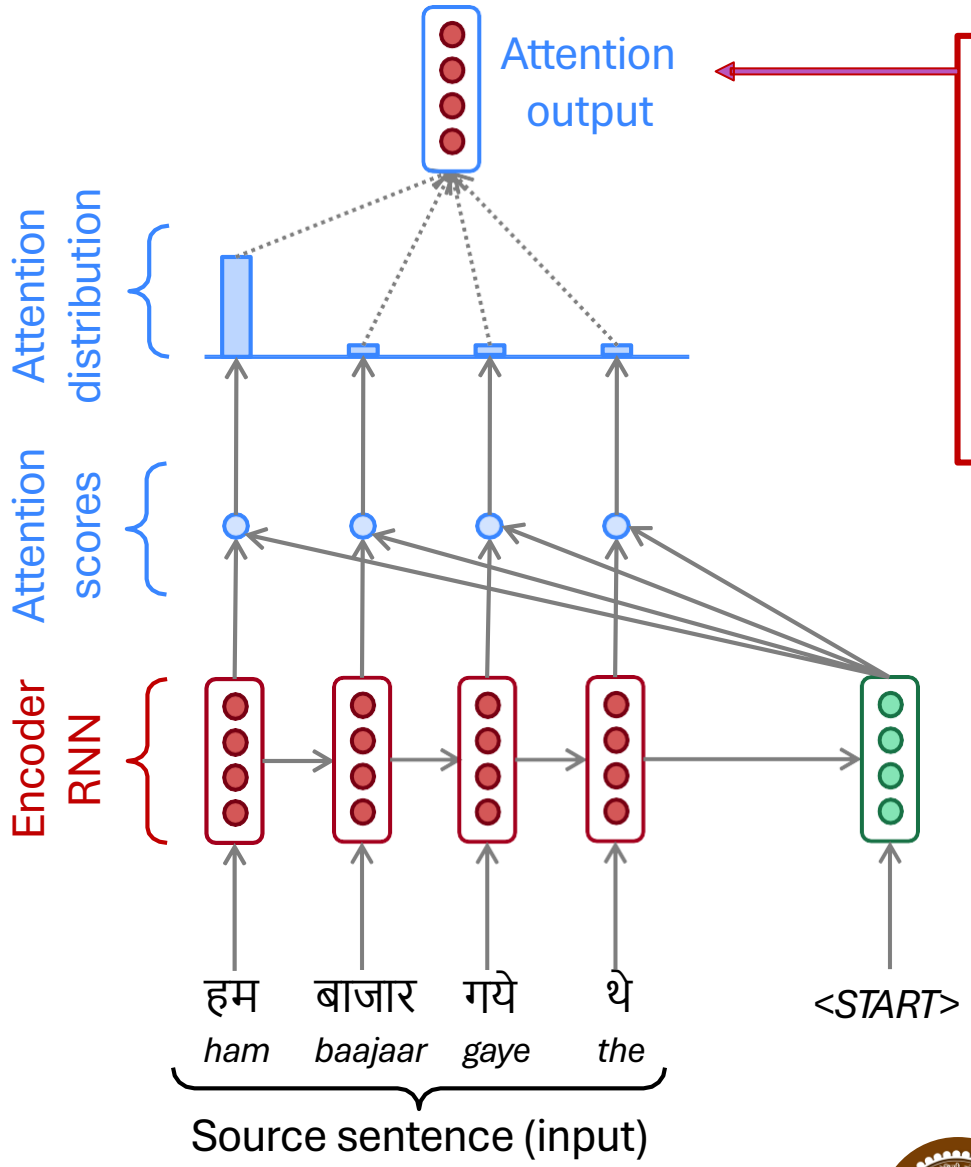
Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



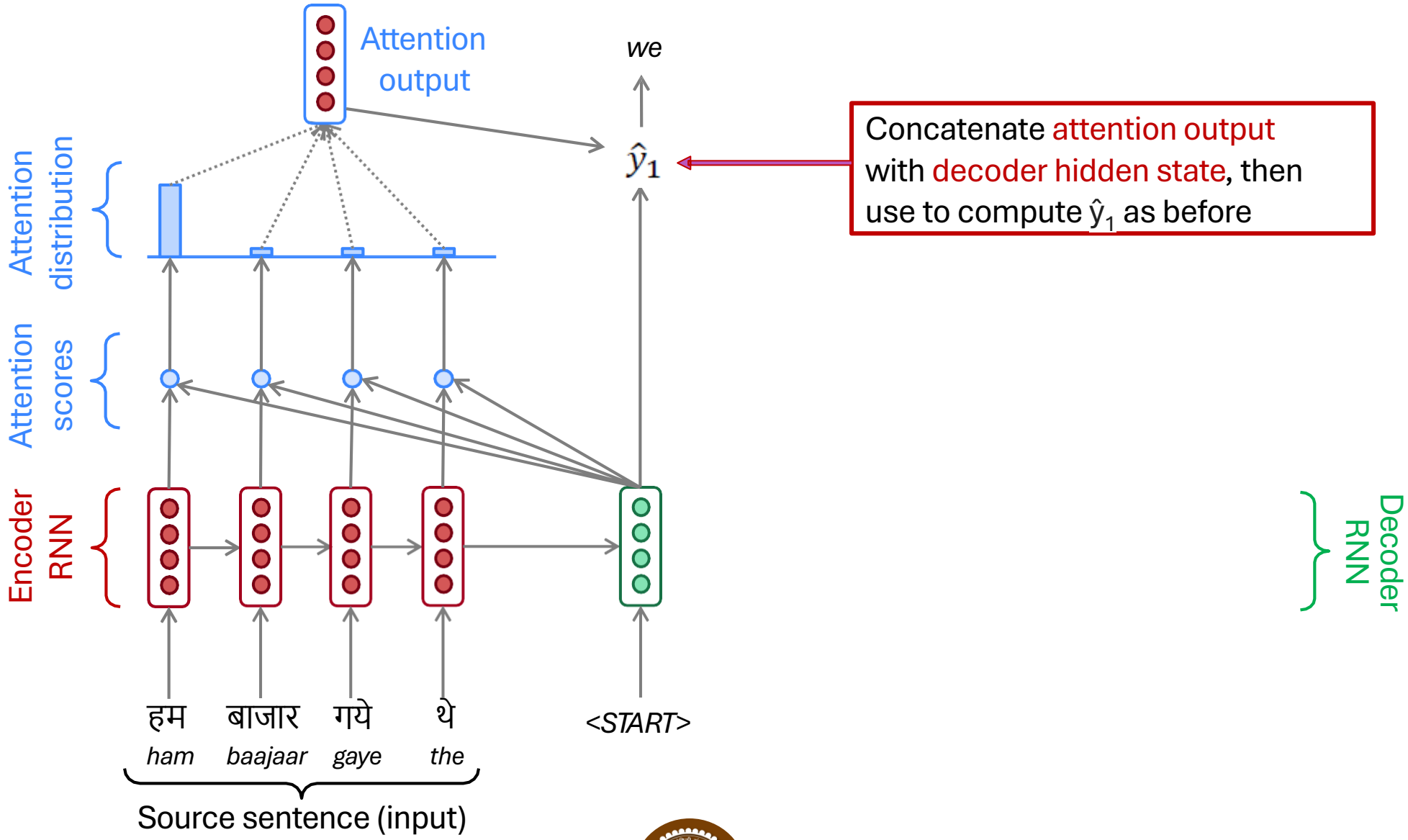
Sequence-to-Sequence With Attention



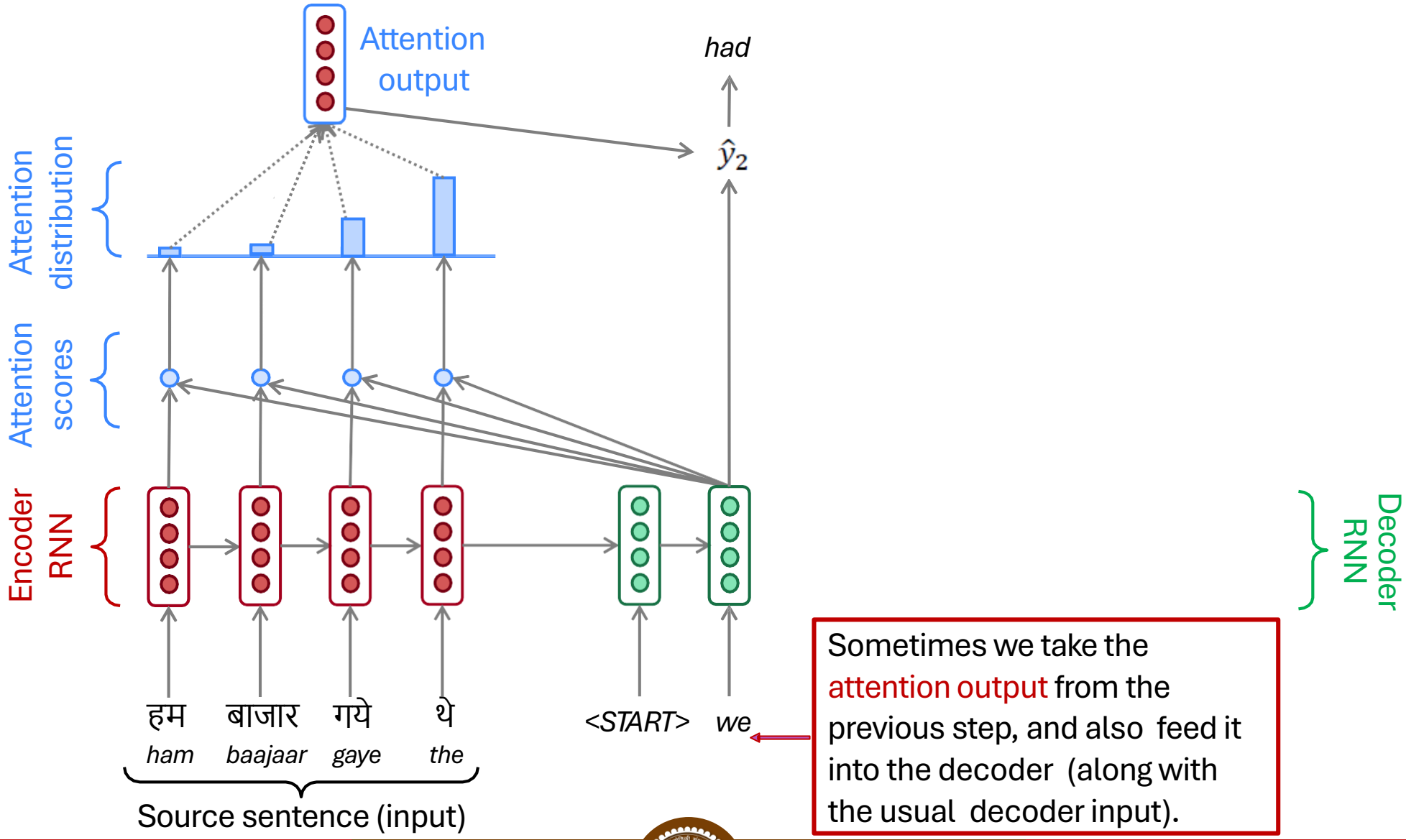
Use the attention distribution to take a **weighted sum** of the encoder hidden states.

The attention output mostly contains information from the hidden states that received high attention.

Sequence-to-Sequence With Attention

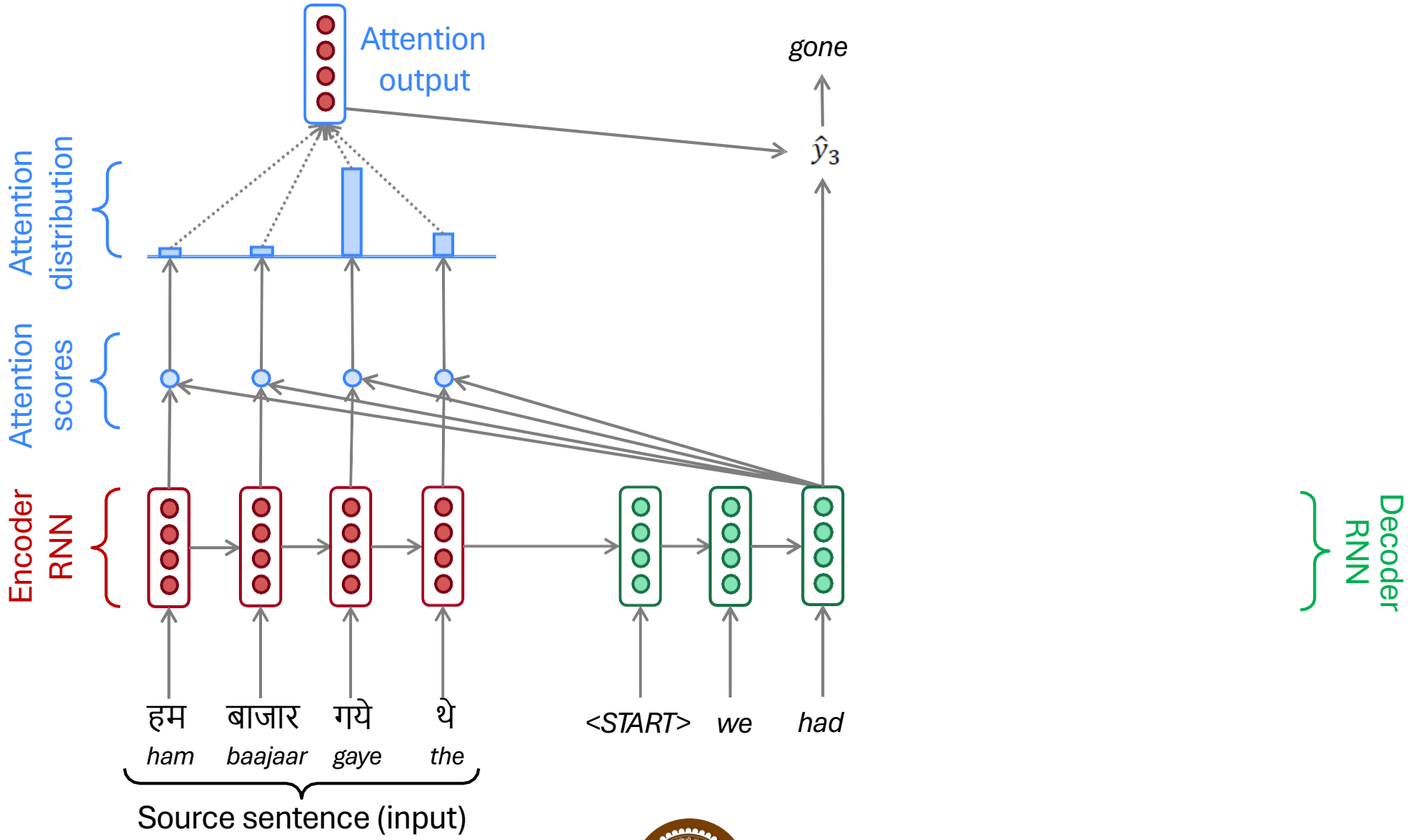


Sequence-to-Sequence With Attention

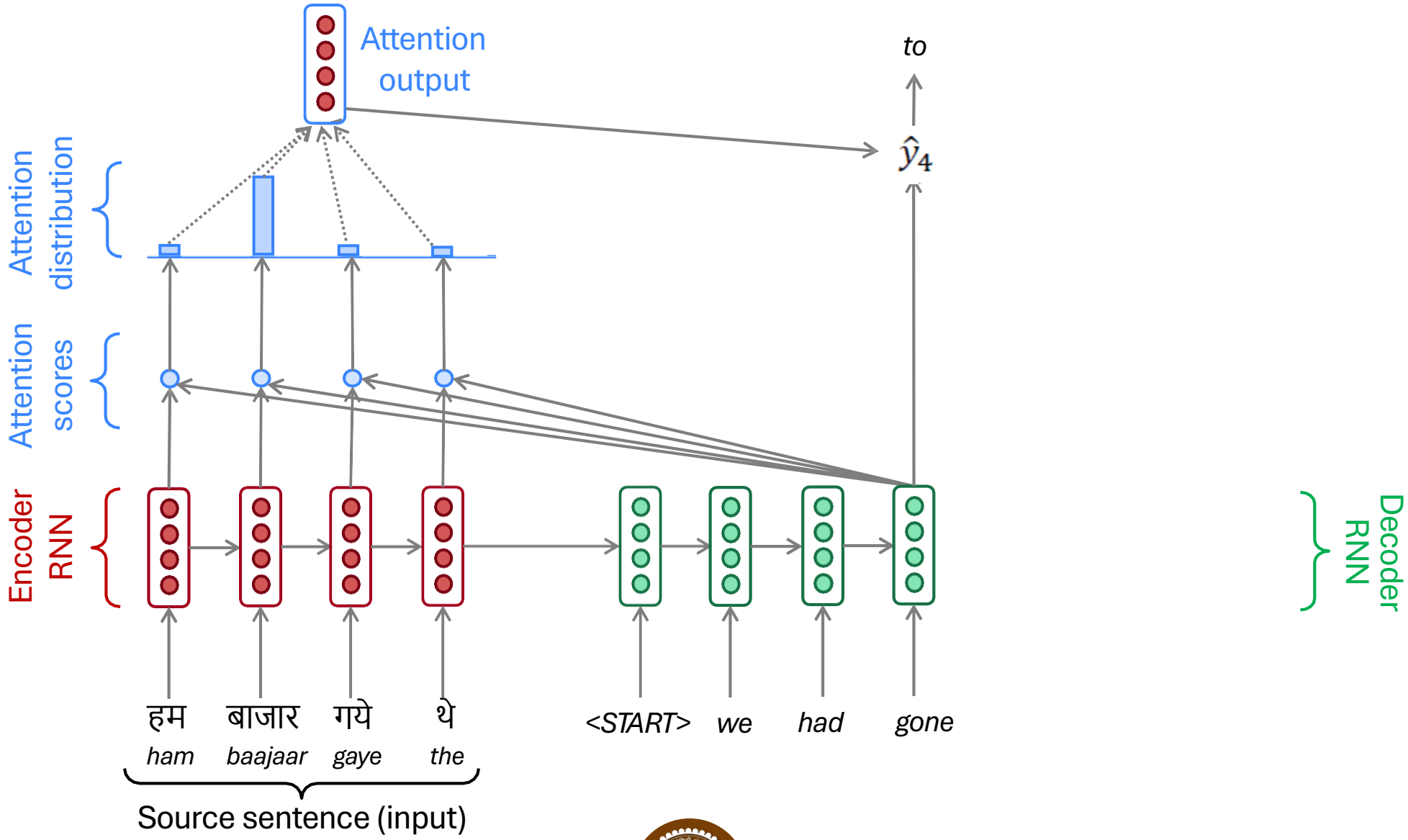


Sometimes we take the attention output from the previous step, and also feed it into the decoder (along with the usual decoder input).

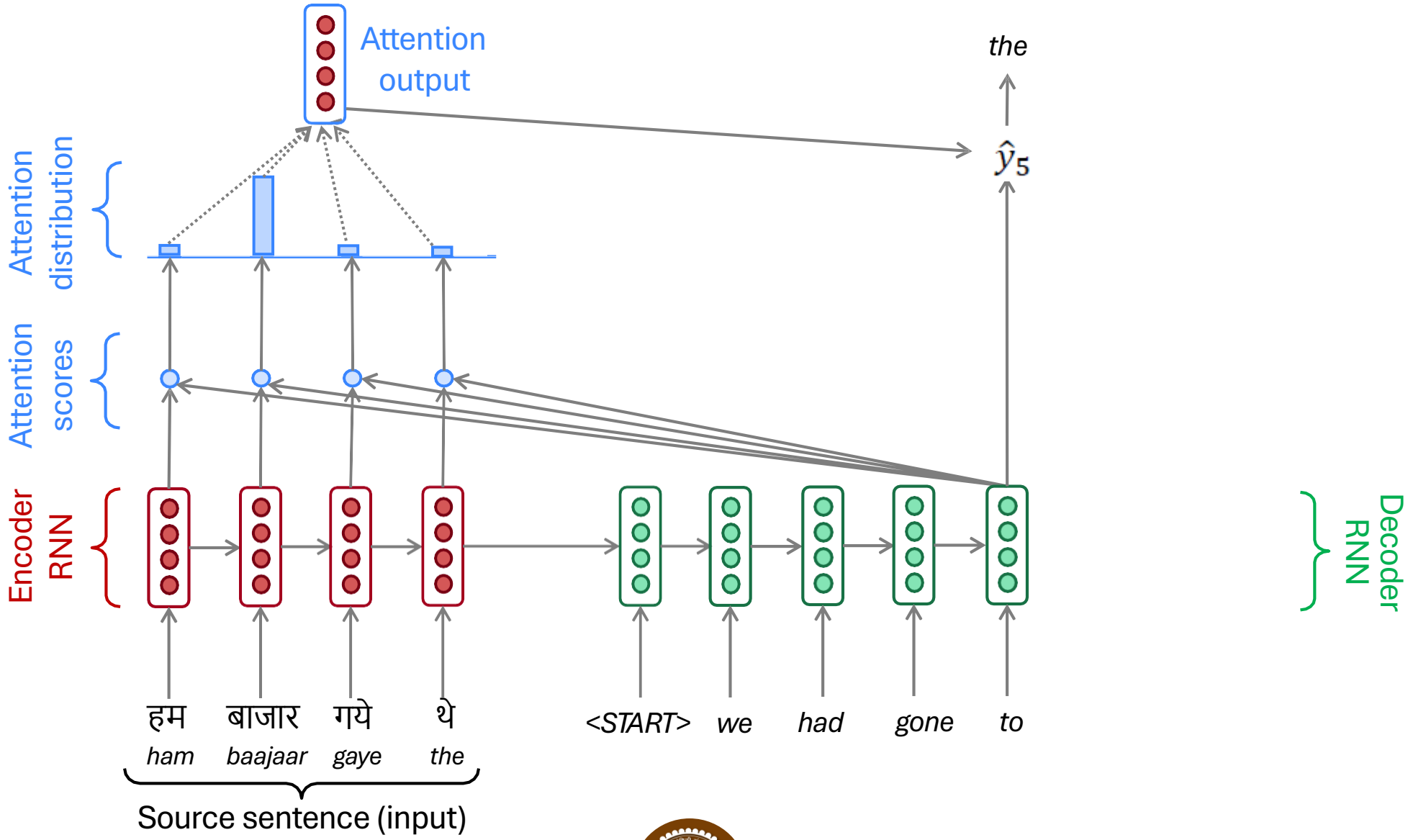
Sequence-to-Sequence With Attention



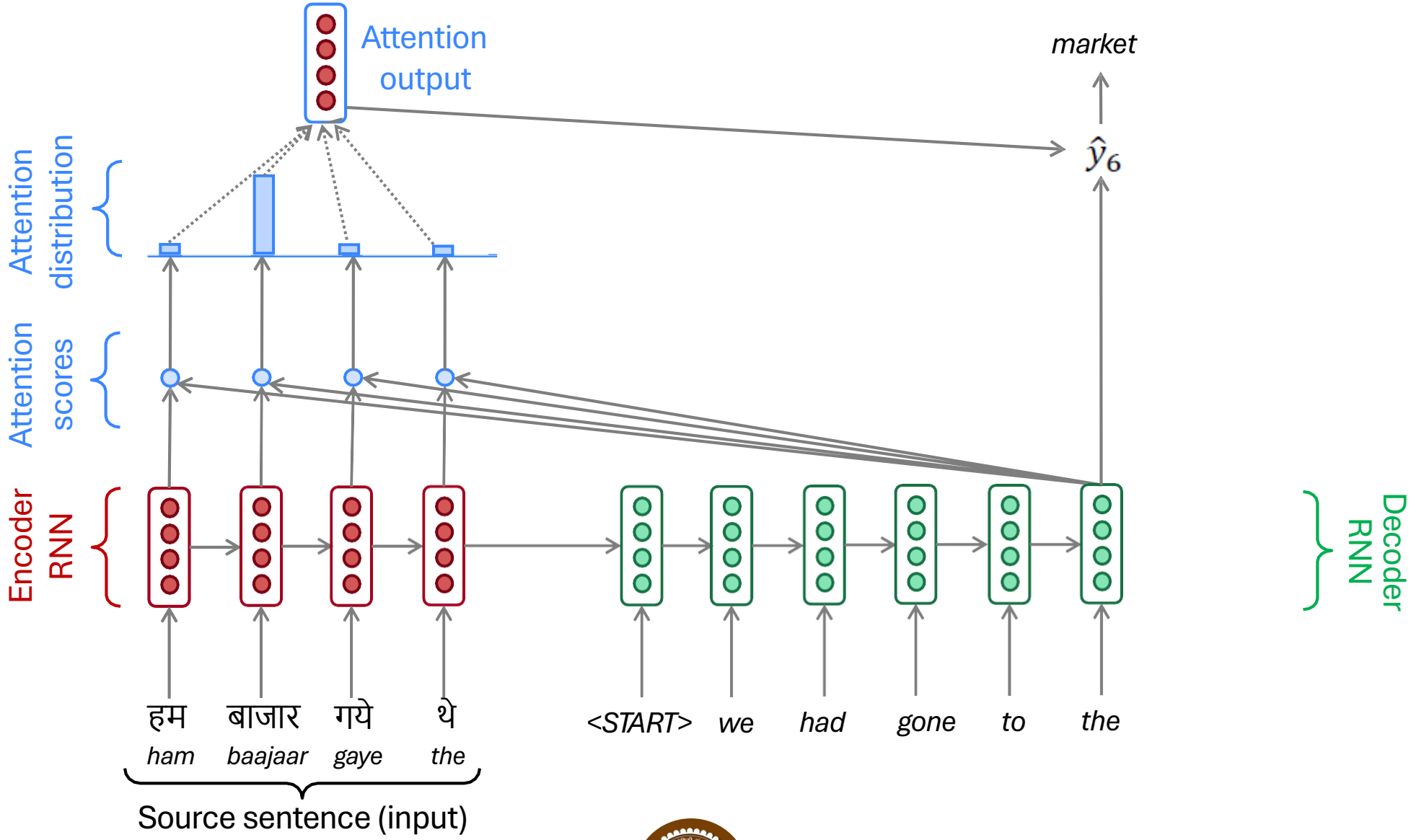
Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



Sequence-to-Sequence With Attention



Attention: In Equations

- We have encoder hidden states $h_1, \ldots, h_N \in \mathbb{R}^h$
- On timestep t, we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$\boldsymbol{e}^t = [\boldsymbol{s}_t^T \boldsymbol{h}_1, \ldots, \boldsymbol{s}_t^T \boldsymbol{h}_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution, sums to 1)

$$\alpha^t = \text{softmax}(\boldsymbol{e}^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output a_t

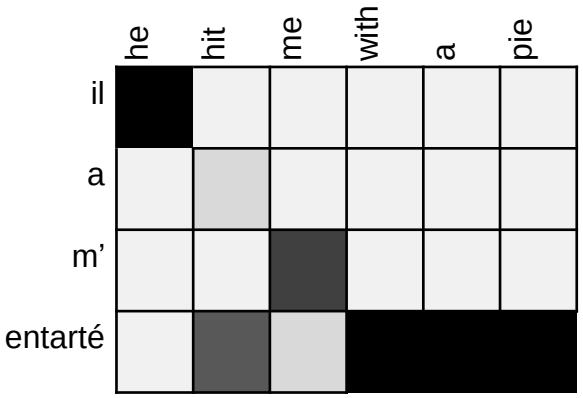
$$\boldsymbol{a}_t = \sum_{i=1}^{N} \alpha_i^t \boldsymbol{h}_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output a_t with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

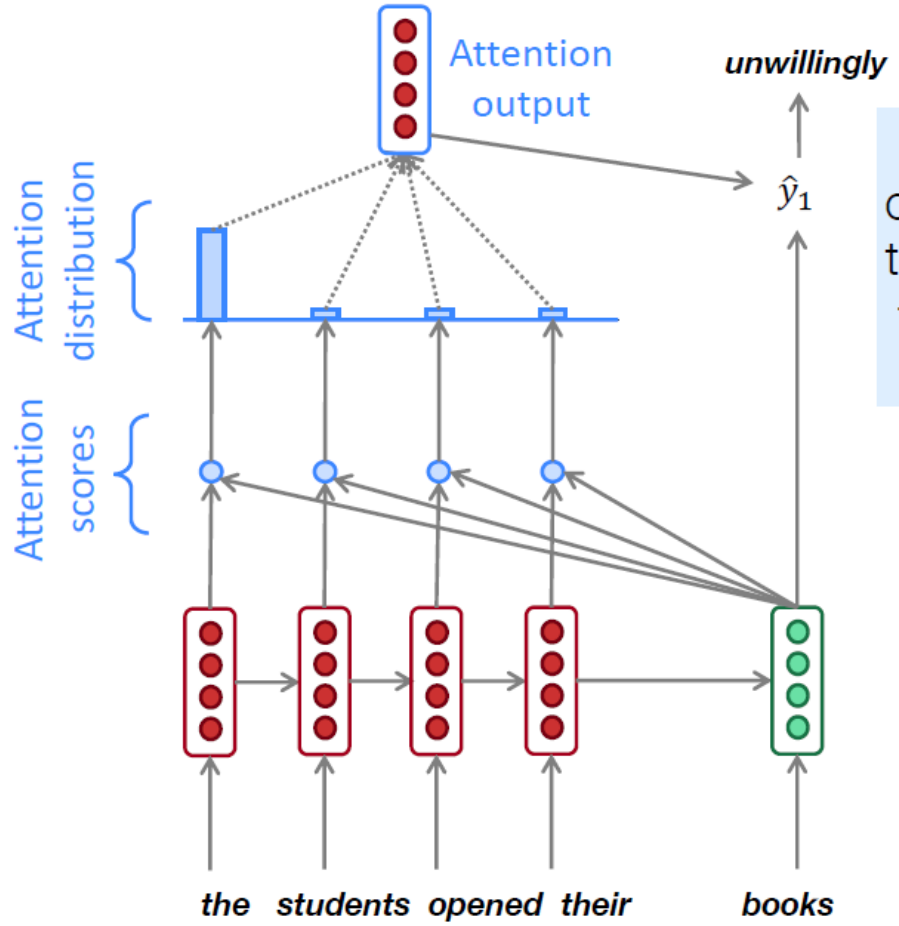
$$[\boldsymbol{a}_t; \boldsymbol{s}_t] \in \mathbb{R}^{2h}$$

Attention is Great

- Attention significantly improves NMT performance
  - It's very useful to allow decoder to focus on certain parts of the source
- Attention solves the bottleneck problem
  - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
  - Provides shortcut to faraway states
- Attention provides some interpretability
  - By inspecting attention distribution, we can see what the decoder was focusing on
  - We get (soft) alignment for free!
  - This is cool because we never explicitly trained an alignment system
  - The network just learned alignment by itself



Seq2Seq+Attention for LM

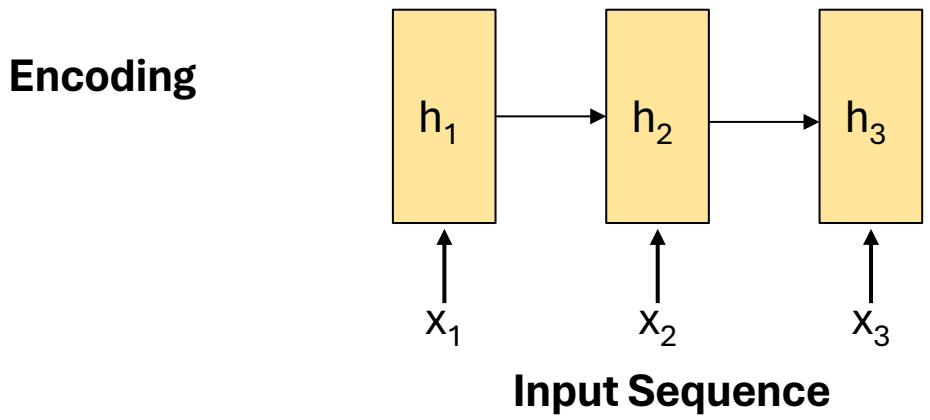


Concatenate (or otherwise compose) the attention output with the current hidden state, then pass through a softmax layer to predict the next word

Attention is a *General* Deep Learning Technique

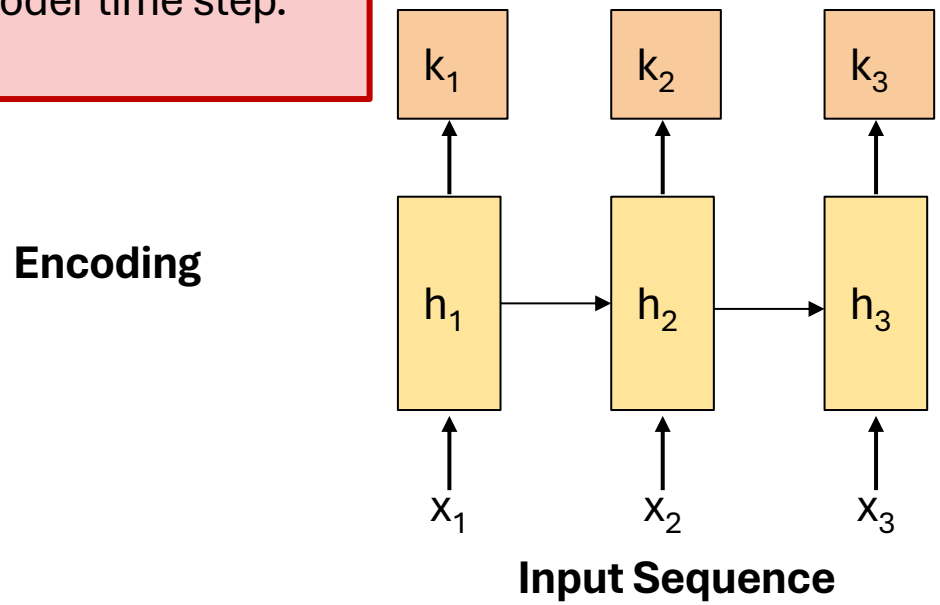
- We've seen that attention is a great way to improve the sequence-to-sequence model for Machine Translation.
- However: You can use attention in many architectures (not just seq2seq) and many tasks (not just MT)
- More general definition of attention:
  - Given a set of vector *values*, and a vector *query*, attention is a technique to compute a weighted sum of the values, dependent on the query.
- We sometimes say that the query *attends to* the values.
- For example, in the seq2seq + attention model, each decoder hidden state (query) *attends to* all the encoder hidden states (values).
- **Intuition**:
  - The weighted sum is a *selective summary* of the information contained in the values, where the query determines which values to focus on.
  - Attention is a way to obtain a *fixed-size representation of an arbitrary set of representations* (the values), dependent on some other representation (the query).

Attention

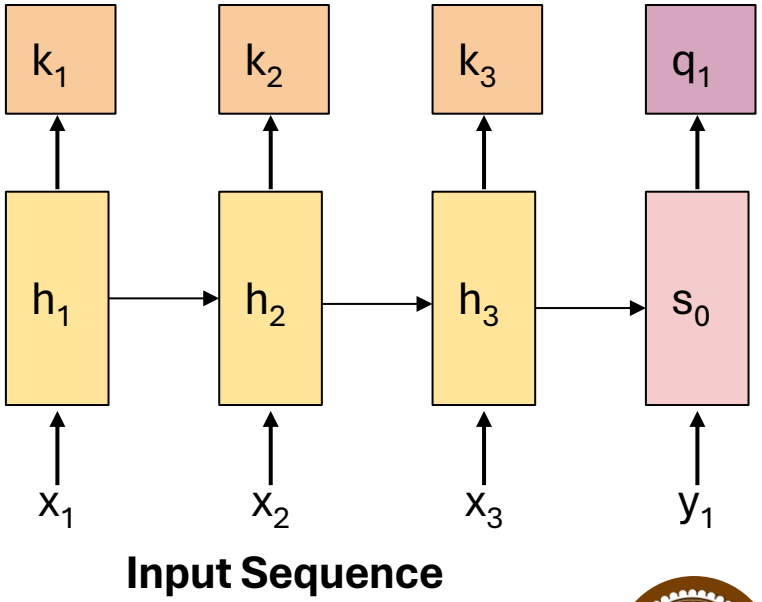


Attention

Key vectors represent what **information** is **encoded** at each encoder time step.

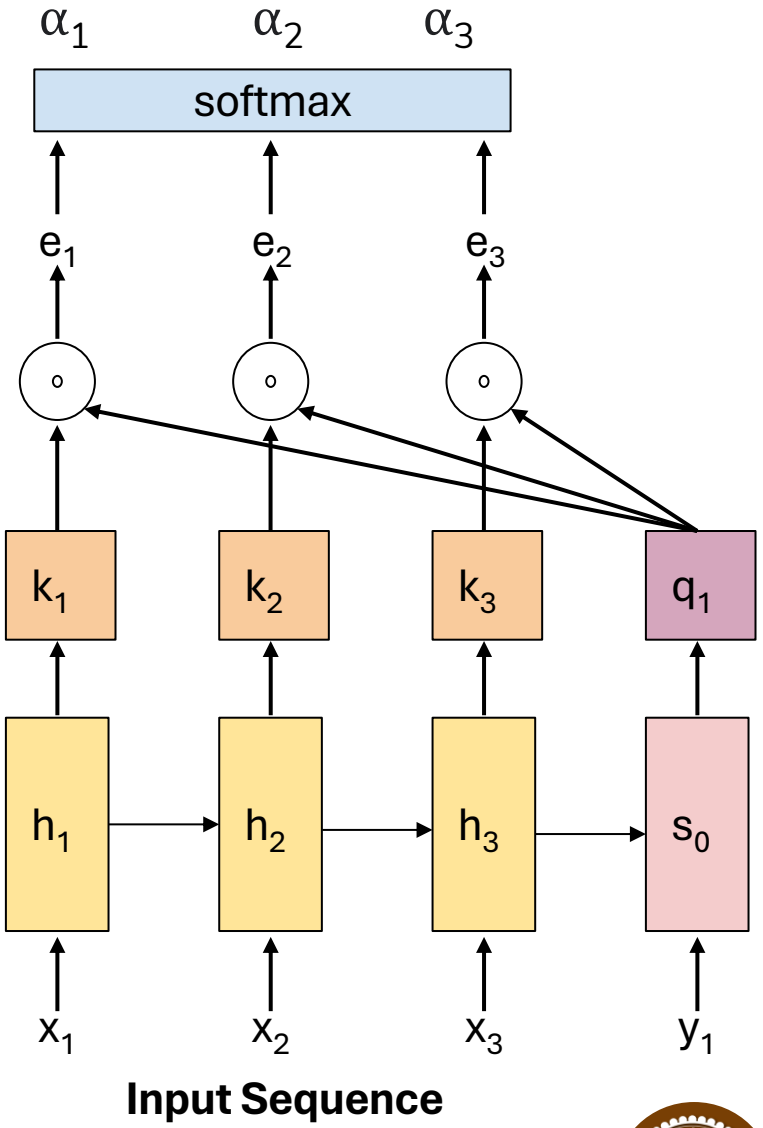


Attention



Query vectors represent what information we are **looking for** at each decoder time step.

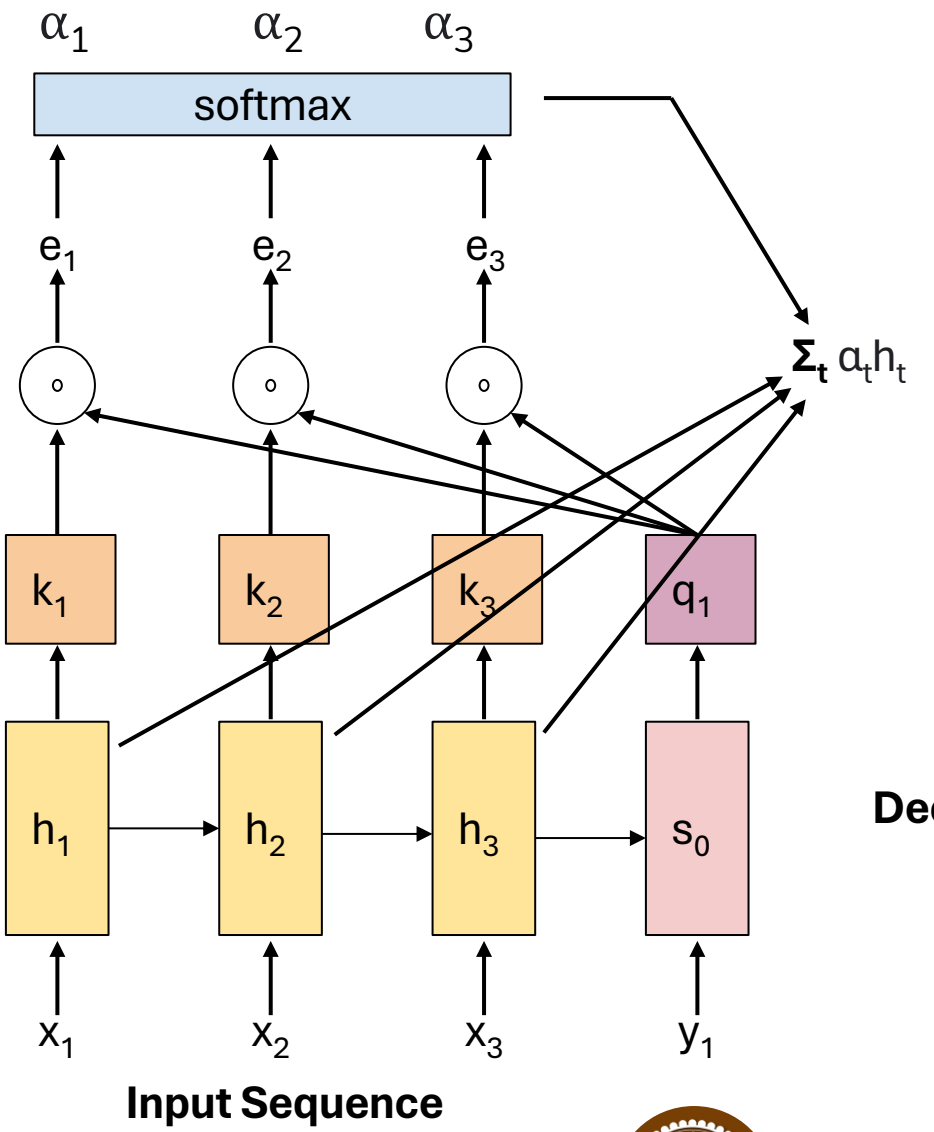
Attention



Softmax converts the similarity scores into a **probability distribution**.

Dot product between query vector and every key vector gives **similarity score**.

Attention



The output of attention mechanism is the **weighted sum** of hidden vectors.

Instead of simply summing up the hidden vectors, we can transform them using a learned function to generate **value vectors** and then compute a weighted sum.

Variants of Attention

- Original formulation: $a(\mathbf{q}, \mathbf{k}) = w_2^T \tanh(W_1[\mathbf{q}; \mathbf{k}])$
- Bilinear product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T W \mathbf{k}$ Luong et al., 2015
- Dot product: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^T \mathbf{k}$ Luong et al., 2015
- Scaled dot product: $a(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^T \mathbf{k}}{\sqrt{|\mathbf{k}|}}$ Vaswani et al., 2017

More information:
"Deep Learning for NLP Best Practices", Ruder, 2017. http://ruder.io/deep-learning-nlp-best-practices/index.html#attention
"Massive Exploration of Neural Machine Translation Architectures", Britz et al, 2017, https://arxiv.org/pdf/1703.03906.pdf

Self-Attention